



Evaluating Model Context Protocol (MCP)-Enabled Large Language Models for Natural Language User Interfaces to Big Data Systems

by

Ruoxi Rosa Yan

1272152

Supervisor: Professor Richard Sinnott

A thesis submitted in partial fulfillment for the
degree of Master of Computer Science

in the

School of Computing and Information Systems
THE UNIVERSITY OF MELBOURNE

June 2026

THE UNIVERSITY OF MELBOURNE

Abstract

School of Computing and Information Systems

Master of Computer Science

by Ruoxi Rosa Yan

1272152

This thesis evaluates MCP-enabled large language model agents as natural-language interfaces to the Australian Internet Observatory, a live social-data research system. It develops a 35-task execution-grounded benchmark with live-system auditing, repeated trials, process-level diagnostics, and qualitative trace analysis.

Across seven model configurations, agents completed many metadata, retrieval, and regular aggregation tasks successfully. However, valid tool calls did not consistently produce reliable analytical answers. Performance was less stable on tasks requiring interpretation, synthesis, contextual qualification, and final-answer construction. Failures often occurred after relevant evidence had been retrieved, including omitted limitations, incorrect aggregation, unsuitable comparisons, and undisclosed fallback retrieval.

The findings indicate that researcher-facing MCP agents require safeguards beyond prompt instructions and tool-schema validation. Structured evidence states that preserve provenance, coverage conditions, missingness, and derived calculations may improve the reliability of final research-facing answers.

Declaration of Authorship

I, Ruoxi Rosa Yan, declare that this thesis titled, ‘Evaluating Model Context Protocol (MCP)-Enabled Large Language Models for Natural Language User Interfaces to Big Data Systems’ and the work presented in it are my own. I confirm that:

- this thesis does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any university; and that to the best of my knowledge and belief it does not contain any material previously published or written by another person where due reference is not made in the text.
- the thesis is 25912 words in length (excluding text in images, table, bibliographies and appendices).

Acknowledgements

I would like to thank Professor Richard Sinnott, my supervisor, for his guidance throughout this project. I am especially grateful to Luca Morandini for his generous technical support, feedback, and assistance in understanding and working with the MCP server and related research infrastructure.

I also acknowledge the Australian Internet Observatory and AIReD project teams for providing the research setting for this study.

I am deeply grateful to my family and friends for their care and encouragement throughout this period. Their support made a meaningful difference in bringing this thesis to completion.

Finally, I am grateful to the books, games, and quiet company that made it possible to keep returning to this work.

Contents

Abstract	i
Declaration of Authorship	ii
Acknowledgements	iii
List of Figures	viii
List of Tables	ix
1 Introduction	1
1.1 Study Aim, Scope, and Approach	5
1.2 Research Questions	6
1.3 Contributions	7
1.4 Thesis Structure	7
2 Background and Related Work	9
2.1 Background	9
2.1.1 LLM Agents and Tool Calling	9
2.1.1.1 Evolution of Tool-Calling Mechanisms	10
2.1.2 Model Context Protocol	11
2.1.3 Australian Internet Observatory	12
2.2 Literature Review	15
2.2.1 Natural Language Interface: An Overview	15
2.2.2 Evaluation of LLM agents	17
2.2.3 Design Emphases in Existing MCP Benchmarks	18
Large-scale tool discovery and cross-server orchestration	19
Beyond one-shot success: process, reliability, and operational diagnostics	20
Execution-grounded evaluation in dynamic environments	21
2.2.4 Positioning of the present study	23
3 Benchmark Design	25
3.1 Design Principles	25
Research-oriented task realism.	25
Controlled evaluability.	26

	Capability-oriented coverage.	26
	Execution feasibility in a live system.	26
3.2	Benchmark Task Taxonomy	27
	Answer type:	29
	Source scope:	29
	Task demand profile:	29
3.3	Task Construction	31
	Feasibility against the current MCP server:	31
	Alignment with task category	31
	Temporal clarity	32
	Verifiable outcomes	32
	Natural-language tool selection.	32
3.3.1	Ground Truth Collection	32
3.3.2	Challenges in live environment	34
4	Methodology	36
4.1	System architecture	36
4.1.1	Remote MCP Server	37
4.1.2	Interface Adaptation	38
	Frozen tool snapshot:	38
	Schema adaptation for OpenRouter function calling:	38
4.1.3	Agent	39
	Execution Loop	39
	Prompting and Output Format	40
4.1.4	Trace Logging	40
4.2	Experimental Setup	41
4.2.1	Model selection	41
4.2.2	Execution Procedure	42
4.3	Evaluation Framework	43
4.3.1	Outcome correctness evaluation procedure	44
	4.3.1.1 Evaluator	44
	Judge-based Evaluator	44
	Deterministic Evaluator	46
	4.3.1.2 Reliability	46
4.3.2	Process-level diagnostics	47
	4.3.2.1 Tool-Call Appropriateness Classification	48
	4.3.2.2 Tool call quality	49
4.3.3	Qualitative Error Analysis Procedure	50
5	Results	52
5.1	Task Completion	52
5.1.1	Average Task Completion Rate	52
5.1.2	Task completion rate by Task demand profile	54
5.1.3	Reliability Across Repeated Runs	56
5.2	Process-Level Diagnostics	58
5.2.1	Tool-call generation	59
5.2.2	Tool Selection	61

5.3	Efficiency	63
5.4	Qualitative Error Analysis	66
5.4.1	General Findings	66
5.4.2	Example of task failures	69
5.5	Supplementary case study of Open-Ended Tasks	74
5.5.1	Study Design	75
5.5.2	Findings	77
6	Discussion	82
6.1	RQ1: Designing an Execution-Grounded Benchmark for Live Research Data Systems	82
	Beyond minimum-path evaluation.	83
6.2	RQ2: Task Performance and Repeated-Run Reliability	84
6.3	RQ3: Tool Use, Failure Patterns, and Exploratory Behaviour	87
6.3.1	Process-Level diagnostics are necessary but insufficient	87
6.3.2	Many Consequential Failures Occurred After Retrieval	88
6.3.3	Exploratory Behaviour	91
6.4	Implications for Research-Agent Design and Deployment	93
6.4.1	One-shot capability is insufficient for researcher-facing deployment	93
6.4.2	From prompt instructions to enforceable harness controls	94
6.4.3	Research agents require a structured evidence state beyond conversational history	95
6.5	Limitations and future work	95
	Benchmark Scope and Single-Annotator Bias	96
	Evaluation Framework Rigidity and Metric Tensions	96
	Operational Variance and the Researcher Experience	97
7	Conclusion	99
	Data and Code Availability	101
A	Benchmark Task Set	102
A.1	Open-Ended Supplementary Tasks	106
B	Prompt Templates	107
B.1	Agent Prompt Configuration	107
B.1.1	Base system prompt	107
B.1.2	Scalar-response schema prompt	108
B.1.3	Time-series-response schema prompt	108
B.1.4	Text-response schema prompt	109
B.1.5	Chart-response schema prompt	109
B.2	Judge Prompt Configuration	111
B.2.1	Semantic correctness evaluator prompt	111
C	Illustrative Evaluation Record	112
C.1	Task and audited reference answer	112
C.2	Abbreviated agent execution trace	113

C.3	Agent final response	113
C.4	Judge decision	114
D	Ground-Truth Audit Protocol for Live-System Evaluation	115
D.1	Purpose and Scope	115
D.2	Repository Dependencies and Implementation Context	116
D.3	Audit Workflow	116
D.4	Audit Record and Reference-State Control	118
E	Additional Results	119
E.1	Tool appropriateness by model and task demand profile, calculated over successful trials only.	119
E.2	Task-Level Completion Outcomes Across Repeated Trials	119
	 Bibliography	 121

List of Figures

2.1	Typical MCP workflow. Source: [1]	12
2.2	Screenshots illustrating current AIO design challenges.	14
4.1	System architecture	37
5.1	Distribution of task-level outcomes across 35 tasks for each model.	56
5.2	<i>pass</i> ³ heatmap by model and task demand profile. A = Analytic Interpretation; BR = Batch Retrieval; DR = Direct Retrieval; SY = Synthesis.	59
5.3	Average TCR vs Agent API Cost per Successful Task	65
5.4	Distribution of primary failure categories across failed task-model trials	66
5.5	Task D4	70
5.6	Task E7	71
5.7	Task D7	73
5.8	Distribution of tool-call codes across the three independent trials of each open-ended task. Bars show the number of individually coded tool calls; total coded calls are shown above each bar.	77

List of Tables

3.1	Task demand profiles used to classify benchmark tasks.	30
3.2	Composition of the benchmark task set.	30
4.1	MCP tool capabilities used in the benchmark.	38
4.2	Evaluated models and providers.	42
5.1	Mean task completion rate by model and task category.	54
5.2	Mean task completion rate by model and task demand profile.	54
5.3	Overall single-trial completion and repeatability across three independent trials.	56
5.4	Mean parameter schema validity and constraint compliance by model. . .	59
5.5	Mean tool-call composition and tool appropriateness by model.	61
5.6	Mean tool appropriateness by model and task demand profile.	62
5.7	Efficiency and resource use by model.	64
5.8	Error taxonomy for failed benchmark trials.	69
A.1	Full benchmark task inventory.	103
A.2	Derivation of the open-ended tasks used in the supplementary exploratory case study.	106
D.1	Repository components required for the ground-truth audit.	116
D.2	Ground-truth audit status categories.	117
E.1	Dashes indicate that no successful trials were available for that model–profile combination.	119
E.2	Task-level completion counts across three independent trials. Each cell reports the number of successful trials out of three. AI = Analytic Interpretation; BR = Batch Retrieval; DR = Direct Retrieval; SY = Synthesis.	119

Chapter 1

Introduction

Online platforms have become an important source of evidence for research on public discourse, social behaviour, political communication, crisis response, digital culture, and other forms of social interaction. Large-scale datasets derived from social media platforms can support the study of how people communicate, mobilise, react to events, and construct meaning in digitally mediated environments. They also create opportunities for computational methods that operate across large volumes of posts, including temporal analysis, keyword-based retrieval, sentiment analysis, topic modelling, and cross-platform comparison.

However, the growing availability of online-platform data has not necessarily made such data straightforward to use in research practice. Big-data systems commonly involve multiple heterogeneous collections, evolving datasets, platform-specific metadata, and specialised analytical services. Researchers may need to work across heterogeneous application programming interfaces (APIs), platform-specific data structures, query languages, authentication procedures, and analytical tools before they can answer a relatively bounded analytical question. These conditions create challenges of integration, reproducibility, and interpretation in addition to the basic challenge of data access.

For social-media research, these difficulties are particularly significant because meaningful analysis often depends on comparing data collected under different platform conditions. A seemingly ordinary question, such as whether discussion of an issue changed

across platforms over time, may require a researcher to identify relevant datasets, construct comparable filters, retrieve several outputs, assess collection coverage, and interpret whether observed differences reflect substantive variation or differences in data collection and processing. Such work can require substantial technical expertise and may shift effort towards repetitive data-access and engineering tasks rather than substantive social-scientific analysis [2].

These barriers create a gap between the analytical potential of large-scale online-platform data and its accessibility as a research resource. A researcher-oriented interface should therefore support more than data access alone. It should help users formulate analytical requests, retrieve relevant evidence, and present results with sufficient contextual information for those results to be interpreted responsibly. This need provides the motivation for investigating whether MCP-enabled LLM agents can operate as natural-language interfaces to heterogeneous social-data research systems.

The Australian Internet Observatory (AIO) provides a concrete case through which to examine the challenges of natural-language access to heterogeneous big-data systems. AIO supports research on online platforms by maintaining large-scale social-media collections and providing infrastructure for their exploration and analysis. Its data environment brings together collections with different platform characteristics, temporal coverage, data-processing histories, and analytical affordances. As a result, even relatively focused research questions may require users to combine evidence from several system functions and interpret results in light of differences in collection coverage or processing. As part of the AIO initiative, the University of Melbourne eResearch Group developed the AIReD project, which offers a graphical data dashboard through which researchers can inspect collections and construct queries. It also provides an API exposing programmatic access to social-media data and analytical functions for more flexible or automated use. This combination makes AIReD a useful setting for the present study. While the underlying AIO infrastructure and AIReD’s analytical tools are robust, they still present significant friction for users. Effectively leveraging these resources can require researchers to navigate separate interfaces, understand structured query parameters, and connect outputs from different analytical functions. An MCP-enabled LLM agent could potentially operate as an additional natural-language layer over these existing capabilities, translating researcher-oriented questions into the appropriate sequence of API-mediated operations. The key question, however, is whether such agents can

perform this role accurately, consistently, and with sufficient sensitivity to the evidential limitations of the underlying data system.

Agents as natural language interface

Recent advances in large language models create a potential way to reduce the interaction burden associated with systems such as AIReD. Rather than requiring a researcher to navigate individual dashboard modules or formulate API requests directly, an LLM-based interface could allow the researcher to express an analytical objective in natural language. The agent would then need to determine which available system capabilities are relevant, formulate the required parameters, execute the necessary operations, and translate the returned outputs into a researcher-facing response. The Model Context Protocol (MCP) provides a standardised mechanism through which such agents can access external tools and data services. In this setting, MCP does not replace the AIReD dashboard or API. Instead, it offers a layer through which an LLM agent can discover and invoke the analytical capabilities already exposed by the underlying system. For example, a request to compare discussion of an issue across two collections over time may require the agent to identify valid collections, issue comparable retrieval or aggregation calls, examine collection coverage, and integrate the resulting evidence into a coherent explanation. This potential extends beyond conversational convenience. An effective MCP-enabled agent could coordinate multiple analytical functions that are otherwise distributed across separate interfaces and reduce the technical effort required to move from a broad research question to an evidence-grounded result.

However, this additional interface layer also introduces new points of failure. The ability to invoke external tools does not, by itself, establish that an LLM agent can be a reliable research interface. The tasks involve several linked stages: interpreting the researcher's request, selecting relevant capabilities, generating appropriate parameters, executing calls, retaining intermediate evidence, and synthesising the returned results into a final response. An error at any stage can affect the validity of the answer presented to the user. Formal function-call validity provides only a limited indication of performance. An agent may generate arguments that satisfy the input schema of a tool while still selecting an unsuitable collection, applying an inappropriate filter, requesting an incomplete time range, or retrieving evidence that does not adequately address the analytical question.

Even when the retrieved outputs are relevant, further problems can arise during post-retrieval reasoning. Agents may aggregate values incorrectly, compare quantities using unsuitable baselines, interpret missing observations as zero values, or omit metadata and processing limitations that substantially affect the meaning of a result. These risks are particularly consequential in researcher-oriented systems. Analytical outputs often appear authoritative because they contain precise counts, dates, percentages, and references to underlying datasets. However, a numerically precise answer can still be misleading when its evidential basis is incomplete, its interpretation is unsupported, or important limitations are omitted. Similarly, an agent may attempt to recover from an unavailable tool response by using an alternative retrieval strategy. Such recovery may be useful, provided that the changed evidential basis is made explicit to the researcher. Accordingly, evaluation of MCP-enabled research interfaces requires more than checking whether an agent invokes available tools successfully. It must examine whether the final answer remains faithful to the retrieved evidence, communicates relevant constraints and limitations, and provides a sufficiently reliable basis for researcher interpretation. This motivates an end-to-end evaluation framework that considers outcome correctness, repeated-run consistency, trace-level tool-use behaviour, and operational efficiency.

Gap in Existing Evaluation

Recent benchmarks have substantially expanded the evaluation of LLM tool use beyond isolated function-call generation. Earlier work on function calling primarily assessed whether a model could select an appropriate API endpoint and produce structurally valid arguments. More recent agent benchmarks have introduced multi-step tasks, repeated trials, execution-based evaluation, and broader tool environments, reflecting the fact that tool-using agents must plan, act, observe, and revise their behaviour over multiple interactions. MCP-specific benchmarks have further examined whether agents can discover, select, and coordinate tools exposed through MCP servers in realistic execution settings. These developments provide important foundations for evaluating MCP-enabled agents. However, the settings examined in existing work do not fully capture the characteristics of researcher-oriented analytical use over heterogeneous big-data systems. Many benchmarks focus on general-purpose utility tasks, synthetic tool environments, software-oriented workflows, or broad ecosystem-scale orchestration. Even where tasks

are executed against live servers, evaluation commonly prioritises completion, tool selection, parameter validity, or trajectory alignment. These measures are necessary, but they provide only partial evidence of whether an agent can support defensible analytical interpretation. Research-data environments introduce additional requirements. A satisfactory response may depend on recognising differences in collection coverage, preserving the distinction between unavailable data and zero values, interpreting NLP-derived outputs cautiously, using comparable analytical conditions across sources, and disclosing when an answer depends on an alternative retrieval strategy. In such settings, a model may issue formally valid and apparently relevant tool calls while still producing a misleading final answer through incomplete synthesis, incorrect aggregation, unsupported comparison, or omitted contextual qualification. A further challenge is that live research infrastructures are not static benchmark environments. Underlying data may be updated, historical records may be backfilled, server behaviour may vary across collections or date ranges, and analytical outputs may change as processing pipelines are revised. Evaluation therefore requires reference answers that are grounded in actual tool execution and periodically audited, rather than assumed to remain permanently fixed. This study addresses these gaps by developing an execution-grounded benchmark for researcher-oriented analytical tasks on the Australian Internet Observatory. It evaluates MCP-enabled LLM agents across heterogeneous social-data collections using outcome correctness, repeated-run reliability, trace-level tool-use diagnostics, operational efficiency measures, and qualitative error analysis. In doing so, the study examines not only whether agents can invoke available tools, but whether they can produce reliable and interpretable research-facing answers from the evidence returned by a live big-data system.

1.1 Study Aim, Scope, and Approach

The aim of this study is to evaluate MCP-enabled large language model agents as natural-language interfaces for researcher-oriented analytical tasks on the Australian Internet Observatory. The study examines whether agents can translate bounded research requests into appropriate interactions with the analytical capabilities exposed by the AIREd data environment, and whether the resulting answers are accurate, repeatable, interpretable, and practically usable under live-system conditions. To address this aim,

the study develops an execution-grounded benchmark comprising researcher-oriented tasks derived from the analytical functions available through the AIO MCP server. The task set covers a range of research operations, including collection discovery, constrained retrieval, temporal aggregation, cross-collection comparison, and interpretation of NLP-derived outputs. Tasks are organised using a two-layer taxonomy that distinguishes their primary research intent from secondary execution-oriented properties, including source scope, answer type, and task demand profile. Each benchmark task is paired with a manually constructed reference execution plan and evidence-grounded expected outcome. Because the AIO environment is live rather than static, the reference plans and expected answers are audited against the active system before evaluation. This allows the benchmark to retain a close connection to the actual capabilities and constraints of the deployed research infrastructure while maintaining sufficient control for repeated comparison across models. Selected MCP-enabled LLM agents are evaluated through repeated independent trials using a common agent architecture and frozen tool definitions. The evaluation considers end-to-end task completion, consistency across repeated runs, trace-level tool-use behaviour, parameter and constraint compliance, resource use, and qualitative error patterns. A supplementary exploratory analysis also examines how a high-performing agent uses additional tools under less constrained research prompts, where useful analytical context may extend beyond a fixed reference pathway. The study focuses on bounded analytical tasks that can be supported through the available AIREd MCP toolset. It does not evaluate the full research lifecycle, autonomous formulation of new research questions, or the scientific validity of social-media methods themselves. It also does not conduct a participant-based usability study or compare researcher interaction with a deployed chat interface against the existing dashboard. Instead, it evaluates the technical and analytical reliability required before such an interface could be considered for practical researcher-facing deployment.

1.2 Research Questions

The following research questions guide the study.

1. Benchmark Design

How can an execution-grounded benchmark be designed to evaluate MCP-enabled

agents operating in a live, heterogeneous research data system such as the Australian Internet Observatory?

2. Task Performance and Reliability

To what extent can MCP-enabled LLM agents correctly and consistently complete natural-language social-data research tasks, and how does performance vary across task categories?

3. Tool-Use Behaviour and Deployment Implications

What do trace-level tool-use behaviour, error patterns, and operational costs reveal about the practical strengths, limitations, and deployment risks of MCP-enabled agents for researcher-facing analytical data access?

1.3 Contributions

1. An execution-grounded benchmark for live social-data research infrastructure.

The thesis develops a benchmark of researcher-oriented analytical tasks grounded in the actual capabilities and constraints of the AIO MCP server. It includes reference tool plans, structured ground truth, tool classifications, and an audit process for maintaining validity in a live environment.

2. A multi-level evaluation framework for MCP-enabled analytical agents.

The thesis combines outcome correctness and repeated-run reliability with process-level diagnostics of tool appropriateness, parameter schema validity, constraint compliance, efficiency, and trace-based error analysis.

3. Empirical evidence on the limits of end-to-end MCP agent reliability.

The study shows how high rates of formally valid tool calls can coexist with failures in interpretation, quantitative synthesis, contextual qualification, provenance preservation, and structured final-answer generation.

1.4 Thesis Structure

This thesis is organised as follows. Chapter 2 provides background on LLM agents, the Model Context Protocol, and the Australian Internet Observatory, followed by a

review of related work on natural-language interfaces, LLM-agent evaluation, and MCP benchmarking. Chapter 3 presents the benchmark design, including task taxonomy, construction process, reference-plan development, and live-system auditing procedure. Chapter 4 describes the experimental methodology, including the agent architecture, evaluated models, execution procedure, and evaluation framework. Chapter 5 reports the empirical results on task completion, repeated-run reliability, tool-use behaviour, efficiency, error patterns, and exploratory open-ended tool use. Chapter 6 discusses the implications of these findings for benchmark design and the practical deployment of MCP-enabled agents in research-data environments. It also outlines the study’s limitations and identifies directions for future work. Chapter 7 concludes the thesis.

Chapter 2

Background and Related Work

2.1 Background

2.1.1 LLM Agents and Tool Calling

Large Language Models (LLMs) are Transformer-based deep learning models [3]. They are trained on vast quantities of text data, enabling them to understand, generate, and reason about human language. Traditional LLMs are constrained by static parametric knowledge: Their knowledge is frozen at the time of training, leading to a "knowledge cutoff" that prevents them from accessing real-time information. They also lack the capacity to interact with external systems or APIs, which limits their ability to perform tasks beyond text generation.

To overcome these fundamental limitations, the research community has increasingly focused on developing "LLM agents" — systems that augment LLMs with capabilities for autonomous planning, tool use, and dynamic interaction with external environments [4]. Unlike traditional models that simply generate a response based on a prompt, LLM agents can decompose complex tasks into a sequence of intermediate steps. They actively engage with their environments through selecting and utilizing external tools to gather information, execute actions, and adapt their strategy based on observational feedback.

2.1.1.1 Evolution of Tool-Calling Mechanisms

A tool for an LLM is defined as an external program that the model can call through a function interface, passing inputs and receiving outputs[5]. The tool calling mechanisms in LLMs have evolved significantly. Early pioneering works like ReAct [6] and Toolformer [7] demonstrated that LLMs could be prompted to generate textual representations of tool calls. Through prompt engineering, these systems produced interleaved sequences of reasoning traces (i.e., “chain of thought”) and tool-specific actions. For instance, the model first verbalizes a reasoning step (e.g., ”I need to find the current weather in Melbourne”), then selects an appropriate tool and formulates an action (e.g., Action: search(”current weather in Melbourne”)), which was a textual command that required external code for parsing and execution. Then the model receives an observation from the tool, and iterates this process until the task is complete. This strategy enabled models to iteratively combine reasoning with external observations, and improved their ability to complete multi-step tasks involving dynamic information retrieval and planning. This approach falls under the category of tuning-free methods, as it does not require additional model training.

Building on these insights, the paradigm of “tool calling” shifted towards a more robust approach where the model is explicitly fine-tuned for native function calling. This tuning-based method involves training LLMs on high-quality datasets of function calls, which dramatically improves their ability to select the correct tool, construct parameters, and operate in multi-turn environment [8]. This native function-calling capability has been widely adopted and is a core feature of modern LLM APIs, including both commercial systems (e.g., GPT series[9]) and open-source releases (e.g., Llama[10]). In parallel, frameworks such as LangChain[11] and LlamaIndex[12] have emerged to provide developers with standardised interfaces and abstractions for managing tool use.

Despite these advances, there are remaining challenges: efficient coordination of multiple tools and the management of large toolsets, especially when dependencies are complex or incomplete [13]. This limitation underscores the need for a more sophisticated, standardised framework for managing the flow of information.

2.1.2 Model Context Protocol

While tool calling empowers LLMs to generate more reliable responses, the traditional implementation poses several significant challenges for large-scale deployment: Developing separate connectors for each API introduces redundant engineering effort and slows development, while the addition of new data sources or tools remains cumbersome, limiting scalability. The absence of standardised protocols further limits interoperability, preventing reuse across different models and leading to duplication. Moreover, inconsistent security practices in custom solutions increase the likelihood of misconfigurations and data breaches[14].

This challenge has been described as the “ $N \times M$ problem”[15]: Connecting N different LLMs to M external tools or data sources required developers to build a custom integration for each pair, which is both costly and difficult to maintain.

Model Context Protocol (MCP) seeks to address these limitations by offering a standardised framework for how LLM applications expose and consume tools, contextual data, and sampling instructions, often described as “a USB-C for AI”[1]. It provides a universal interface that addresses the $N \times M$ problem by replacing it with a more efficient $M + N$ model, in which each new LLM or tool integrates only once with the protocol.

MCP is designed around a structured client–server model with three core roles[16]:

- **Hosts:** Represent LLM applications, such as integrated development environments (IDEs) or platforms like Claude Desktop, which initiate communication and manage the user experience.
- **Clients:** Maintain one-to-one connections within host applications. Facilitate requests and responses instantiated by host applications.
- **Servers:** Expose specific capabilities to AI applications, including Tools (definition of executable functions that AI applications can invoke), Resources (Data sources), and Prompts (Reusable templates that help structure interactions).

In a typical workflow (see Figure 2.1), the server first exposes available tools to the client as context. When the host LLM receives a natural language query, the client sends a

function call to the server, and the server returns the result. This architecture enables a standardised and auditable channel for tool use across heterogeneous environments.

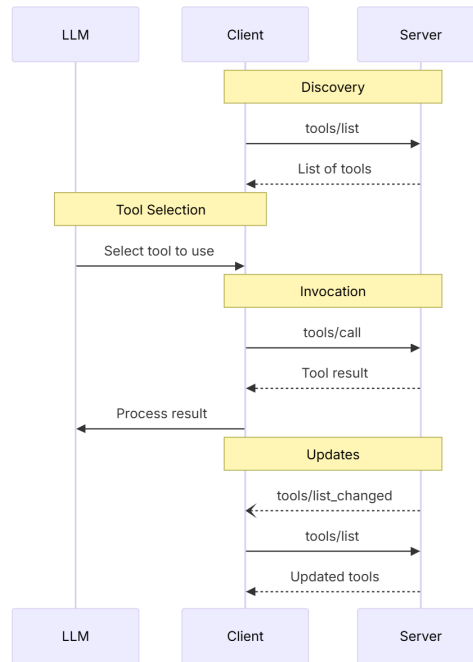


FIGURE 2.1: Typical MCP workflow. Source: [1]

2.1.3 Australian Internet Observatory

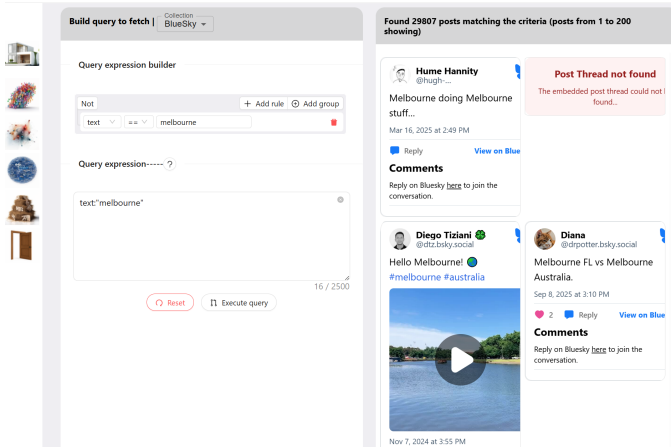
The Australian Internet Observatory (AIO) is a national-scale, multi-institutional digital observatory project that aggregates and analyses data from online platforms to support research on internet activity, social media ecosystems, and digital policy. Its goal is to provide access to large-scale, ethically governed social media datasets and analytical tools for researchers across disciplines[17].

AIO curates and maintains collections of posts from platforms including Mastodon, Reddit, Bluesky, YouTube, and Flickr, as well as a historical Twitter corpus. To enable researchers to interact with these datasets, the Melbourne eResearch Group has developed the AIReD (Australian Internet Research Data dashboard) and API services[18]. These resources allow retrieval of post identifiers, which can then be resolved to full post content via official platform APIs. The platform also provides NLP tools such as topic modelling, sentiment analysis and a range of visualisation tools.

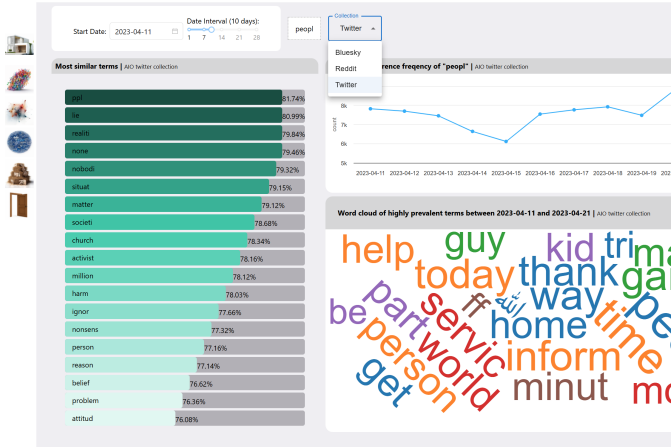
While this has significantly improved availability of social media data for research, several design challenges remain:

- **Complex Query Construction:** The query builder (2.2(a)) requires users to compose structured expressions (e.g., `text:"melbourne"`), select operators, and manage nested rules, which requires technical literacy in query syntax
- **Extensive API Documentation:** The API exposes powerful analytic functions, but interacting with them requires detailed knowledge of parameters (e.g. date formats, thresholds), authentication protocols, and post-processing steps. These demands can exclude researchers without technical training.
- **Fragmentation of analytic outputs:** Visualisations such as topic graphs, aggregation plots, and word clouds are siloed across different modules, each with separate menus and parameter settings (2.2(b), 2.2(c)). To compare results from different platforms, researchers must repeatedly configure settings or write API calls, breaking exploratory flow.

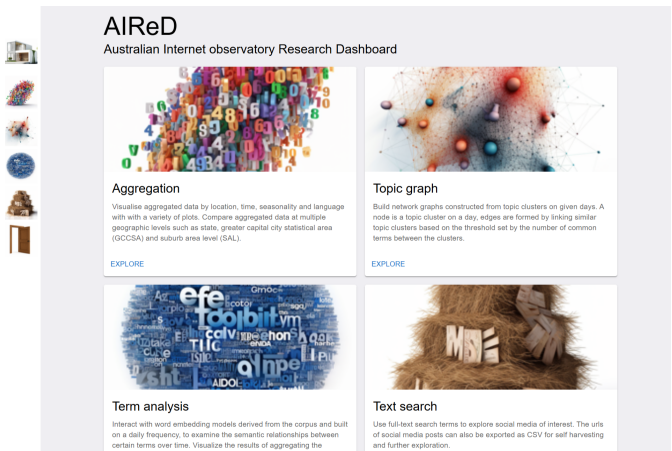
The challenges highlight the limitations of the current AIREd interface design in technical complexity, which restrict broader engagement with AIO datasets. MCP enabled agents offer a potential solution by providing a single standardised interface to all tools. Instead of navigating menus or writing queries, researchers could simply ask: “Compare sentiment around digital privacy on Reddit and Mastodon in July 2025”, leaving the LLM agent to orchestrate the necessary tool calls in the background. This integration could substantially lower technical barriers and make AIO resources more accessible to a wider community of researchers.



(a) Query builder interface requiring structured expressions and nested rules.



(b) Hard to compare results from different platforms.



(c) Analytic outputs siloed across modules (topic graphs, aggregations, word clouds).

FIGURE 2.2: Screenshots illustrating current AIO design challenges.

2.2 Literature Review

2.2.1 Natural Language Interface: An Overview

Natural language interfaces (NLI) are a type of user interface that allow users to interact with a system in a conversational manner using natural language rather than formal query syntax. It has been an active area of research for decades, driven by the goal of empowering non-technical users to access and query complex datasets. Over time, the development of NLIs can be traced through several stages: from database-focused text-to-SQL, through semi-structured data, toward LLM-mediated APIs and tools. Across these stages, a consistent set of challenges emerges—faithfulness, reproducibility, and robustness—which directly motivate the current study.

A large body of work on NLI is rooted in text-to-SQL systems[19], which translate natural language queries into executable SQL statements, allowing users to interact directly with databases without requiring expertise in SQL syntax. Large-scale benchmarks such as Spider[20], CoSQL[21], and BIRD[22] highlight persistent challenges, particularly in schema generalisation, multi-turn interactions, and domain adaptation. Despite the advances brought by neural models and now LLM-based approaches, two problems remain unresolved: ensuring that generated SQL faithfully captures user intent (query fidelity), and achieving robustness when models encounter queries or schemas from previously unseen domains, where they must generalise beyond the training distributions.[19]. These problems highlight the difficulty of aligning flexible human language with rigid, structured backends.

To extend NLIs to more diverse data environments, researchers developed systems [23] for NoSQL and graph databases. While SQL-focused systems struggled with schema generalisation, these systems face challenges in query expressivity and alignment with heterogeneous backends. Systems such as Text-to-NoSQL[24] and Text2Cypher [25] demonstrate that natural language can be mapped into heterogeneous query languages, while multi-agent frameworks like NAT-NL2GQL[26] explore schema alignment and iterative error correction. However, these approaches remain constrained to single-query translation and are often limited to one backend or domain. The harder problems of retrieving data across heterogeneous sources at scale remain largely unsolved.

Parallel efforts applied NLI to data visualisation, where the goal was to translate vague user queries into executable code for tools such as Vega-Lite[27] or matplotlib[28]. Tools like NL4DV[29] and LIDA[30] reduced usability barriers for non-technical users but also revealed problems of inconsistent query-to-visualisation mappings and poor reproducibility across different models. These findings mirror issues already documented in text-to-SQL systems, suggesting that reproducibility is not domain-specific but a fundamental challenge across NLI research.

Recent advances in NLI have shifted the focus from query translation toward orchestration, where LLMs coordinate multiple tools to execute complex tasks. LLMs can now act as mediators between users and APIs, selecting and invoking tools dynamically to execute open-ended tasks. The introduction of the Model Context Protocol (MCP) formalises this development by standardising how LLMs discover, invoke, and manage tools and resources.

Two recent case studies illustrate MCP’s potential across very different domains. In Attrach et al.’s study [31], a conversational system for clinical data repositories encapsulated endpoints as MCP tools and achieved an accuracy of 94% when expert evaluators compared outputs against ground truth. This demonstrated the opportunity for simplified access to sensitive structured data, while also underscoring risks such as hallucinated outputs and potential information leakage if orchestration was not tightly controlled.

A contrasting example comes from the humanities, where researchers implemented an MCP server[32] for the DraCor [33] drama corpus, enabling LLMs to autonomously query plays, corpora, and metadata. Evaluation criteria included quantitative metrics such as tool correctness, tool calling efficiency, and tool use reliability, revealing both strengths and weaknesses: while LLMs often selected appropriate tools and completed tasks, results varied across repetitions, and models frequently encountered maximum sequence length constraints when processing large files.

These studies demonstrate the potential of MCP-enabled LLMs to serve as domain-specific NLIs, from high-stakes medicine to exploratory research in humanities. Yet both highlight the same open problems that are central to this project: retrieval and integration across datasets, and efficient, reliable tool orchestration under real-world constraints. These cases suggest that challenges in reliable retrieval, evidence integration,

and tool orchestration may recur across different MCP application domains, although their severity and form are likely to depend on the specific system and task context.

Across this trajectory of NLI studies, progress has been made in translating queries and lowering usability barriers, but the same challenges persist: schema mismatch, query faithfulness, reproducibility, and orchestration. Existing studies demonstrate feasibility but few studies directly evaluate heterogeneous, large-scale social media environments like AIO, where researchers require reliable cross-platform retrieval and dynamic tool orchestration. This gap—evaluating MCP-enabled LLMs as NLIs for heterogeneous social media data—provides the central motivation for the present study.

2.2.2 Evaluation of LLM agents

The rapid evolution of large language models (LLMs) from static text generators into autonomous agents introduces new challenges for evaluation. Traditional NLP systems are assessed on static benchmarks for tasks such as text classification, summarisation, or question answering. Unlike traditional systems, LLM-based agents operate in dynamic environments where they must plan, act, and adapt. As such, there is a need for holistic, process-aware methods and metrics that can capture interactive, tool-using, stateful behaviour in dynamic environments[34].

Function calling emerged as a major area of agent evaluation. It is a multi step process, including intent recognition, function selection, parameter-value-pair mapping, execution, and response integration [34]. Early work has focused on one-step interactions, evaluating agents’ ability to generate the right function calls from a set of functions in an API on synthetic datasets[35]. Benchmarks such as ToolBench [36] and the Berkeley Function Calling Leaderboard (BFCL)[37] use structural accuracy metrics, such as pass rates and abstract syntax tree matching, to evaluate performance. However these metrics capture only syntactic correctness and overlook the complexity of real-world scenarios of implicit parameters, multi-turn dependencies, and long-context processing.

Recognising these limitations, successive benchmarks have moved toward richer and more realistic settings. Later versions of BFCL [37] incorporated organisational tools and multi-turn, multi-step evaluation logic, while tau-Bench[38] introduced a framework where LLMs simulate both users and environments to test tool-augmented agents in

multi-turn tasks. It also proposed the Pass@k metric, designed to measure reliability and consistency across repeated trials, which is more relevant for production systems than single-shot accuracy. Results revealed that even state-of-the-art models exhibited low consistency (e.g., Pass@8 below 25% on domain-specific tasks), highlighting the gap between laboratory accuracy and production reliability[38]. While initially designed for customer service agents, variants such as MedAgentBench[39] have been extended to medical settings, showing that the methodology is adaptable across domains like healthcare.

Taken together, these efforts trace a clear trajectory: from static, single-step synthetic benchmarks toward multi-step, multi-turn, and domain-sensitive evaluation frameworks. At the same time, they expose persistent blind spots. Most current benchmarks remain confined to narrow domains or synthetic environments, without engaging the heterogeneous and dynamic APIs that characterize real-world big data infrastructures. Addressing these blind spots requires benchmarks that capture not only correctness of tool calls but also reliability, efficiency, and reproducibility in complex, evolving tool ecosystems. More broadly, Rigorous agent evaluation depends on whether both the task setup and scoring procedure validly measure the intended construct. Weakness in either component can lead to substantial over- or underestimation of agent capability[35, 40].

2.2.3 Design Emphases in Existing MCP Benchmarks

While these general benchmarks represent progress, they remain insufficient for evaluating MCP-enabled systems. Unlike prior studies, which often assume static tool schemas in synthetic settings, MCP agents must operate across large, heterogeneous, and evolving tool spaces. This introduces unique benchmarking challenges: first, MCP agents must retrieve and integrate across diverse servers, which goes beyond single-domain reasoning. Second, it requires tool orchestration at scale, requiring agents to discover, select, and sequence unfamiliar tools in long-horizon workflows. Finally, in MCP environments there are trade-offs between accuracy, latency, cost, and reproducibility, which are rarely measured in earlier synthetic or domain-specific settings.

Large-scale tool discovery and cross-server orchestration A major strand of MCP evaluation addresses the challenge of navigating massive, unfamiliar tool environments. Pioneering this effort, MCP-Bench [41] tests agent capability across 28 live MCP servers and 250 tools, assessing whether models can map ambiguous task descriptions to appropriate tools and coordinate multi-step cross-server workflows. The scale of such evaluations was subsequently broadened by LiveMCPBench [42], which incorporates 70 servers and 527 tools across six domains to emphasize tool retrieval and composition. This trajectory of expansion culminates in MCPToolBench++ [43], which drastically scales the evaluation environment to over 4,000 community MCP servers spanning 40 categories, while introducing both single- and multi-step multilingual tasks.

Together, these studies establish tool discovery, routing, parametrisation, and cross-server coordination as important MCP-specific challenges. A standardised protocol makes tool interfaces more interoperable, but it does not remove the need for an agent to identify relevant capabilities from natural-language descriptions, map a user request to appropriate parameters, or sequence calls across heterogeneous services. Large tool inventories also create substantial context burden, since tool descriptions and parameter schemas themselves consume context-window capacity. In these environments, failures may arise before substantive analytical reasoning begins: an agent may retrieve an unsuitable tool, fail to recognise a relevant capability, use an incorrect parameter, or construct an incomplete execution plan.

These large-scale MCP benchmarks also reveal that agents do not necessarily make full or effective use of the tools available to them. LiveMCPBench reports that several evaluated models used close to one tool on average, suggesting that, once a model identified a usable tool, it often relied on that tool rather than continuing to retrieve and combine additional capabilities [42]. MCPGauge similarly frames proactive tool invocation and instruction compliance as distinct aspects of MCP-augmented behaviour, rather than assuming that the presence of available tools guarantees their use [44]. However, the appropriate extent of tool use is task-dependent. Additional calls may represent necessary cross-tool orchestration, but they may also reflect exploratory investigation, recovery from unavailable outputs, or redundant activity. Tool-call volume alone is therefore insufficient for determining whether an agent has used an MCP environment effectively.

These benchmarks provide important evidence about agents intended to operate across

broad MCP ecosystems. However, their aggregate outcomes often combine several distinct stages of failure, including tool discovery, routing, parameterisation, execution planning, and post-retrieval reasoning. As a result, they offer limited isolation of a different question that is central to researcher-facing analytical systems: once an appropriate evidence pathway has been identified and relevant tool outputs have been obtained, can an agent accurately interpret, integrate, and communicate that evidence in a final user-facing answer? This distinction motivates the bounded tool environment adopted in the present study.

Beyond one-shot success: process, reliability, and operational diagnostics

MCP benchmarking has moved away from one-shot final-answer accuracy towards process observability, repeated-run reliability, and deployment-relevant resource measures. A successful outcome may hide inefficient tool use or unstable execution, while failures may still involve largely valid interactions. To capture these differences, studies evaluate agent behaviour across multiple process-level dimensions.

For example, MCP-RADAR [45] emphasises a five-dimensional objective framework that measures Result Accuracy, Tool Selection Efficiency, Computational Resource Efficiency, Parameter Construction Accuracy, and Execution Speed, making trade-offs between speed and accuracy visible via radar charts. MCP-Bench similarly combines task fulfilment with schema compliance, execution success, dependency awareness, and parallelism efficiency [41]. MCP Eval evaluates tool-name, parameter, and call-order matching together with planning quality, requirement coverage, and answer completeness [46]. Such designs make agent behaviour more observable, helping distinguish malformed calls, unsuitable tool choices, inefficient trajectories, and incomplete final responses.

A recurrent pattern across these frameworks is that procedural competence does not necessarily translate into successful end-to-end completion. MCP Eval reports an "execution-completion gap": the average trajectory-matching score was 0.779, whereas the average completion score was 0.679, indicating that agents could follow substantial parts of a reference procedure without producing an adequate final answer [46]. MCP-Bench similarly finds that models can perform strongly on lower-level aspects of tool use, such as valid tool selection and schema compliance, while remaining less reliable on tasks requiring dependency-aware planning and grounded completion across multi-server workflows [41].

These findings suggest that correct tool invocation, parameter construction, and partial trajectory alignment should not be treated as sufficient evidence of answer quality.

Repeated-run evaluation has also become important for assessing whether an agent can be relied upon in practical use. MCPMark reports `pass@1`, `pass@4`, and `pass4`, separating tasks that can be solved at least once from those completed consistently across independent executions [47]. MCPToolBench++ likewise repeats tool-call evaluations to account for variation in live-tool success and multi-step execution [43]. These measures are particularly relevant for deployed interfaces, where a user typically receives one response and cannot be expected to identify whether it represents an intermittent success. Operational burden is increasingly treated as part of this deployment question. In addition to turns, tool calls, tokens, and estimated cost, MCPGauge examines whether MCP integration improves task performance sufficiently to justify its token overhead, and evaluates proactivity and instruction-following separately from outcome effectiveness [44]. Together, these studies show that stronger agent evaluation requires attention to consistency and resource use, as well as average task completion.

Taken together, recent MCP benchmarks establish that agent evaluation must examine process quality, repeated-run consistency, and operational cost alongside task success. However, these measures primarily characterise whether an agent followed a plausible or efficient execution pathway. They provide less direct guidance on the substantive consequences of post-retrieval failures when tool outputs are used as evidence for analytical claims. In researcher-facing data systems, this includes attention to data availability, collection coverage, processing context, comparison basis, and any changes in the evidential pathway used to produce an answer.

Execution-grounded evaluation in dynamic environments A characteristic of the Model Context Protocol ecosystem is its deployment across live, non-static data environments. Consequently, static benchmarks relying on frozen labels are no longer sufficient, prompting recent frameworks to adopt real-time evaluation strategies.

A growing body of MCP evaluation works seeks to assess agents through actual interaction with executable environments rather than through static ground-truth matching. In these settings, an apparently plausible tool call is insufficient: the agent must retrieve information from, or successfully alter, the relevant external state. MCP-AgentBench

explicitly instructs its LLM judge to look for conclusive evidence of tool use in the agent’s answer. The judge is told that the answer “must contain specific external data” (like live metrics, exact distances, or recent events) that could not possibly come from general training knowledge, making the absence of live data a primary failure criterion[48]. MCPMark takes a different approach: it executes tasks in sandboxed environments and uses purely programmatic verification evaluators to objectively check the final environment state[47]. For example when a task requires creating a Notion page or updating a PostgreSQL database, the script checks the actual API to confirm the data was successfully pushed, making it impossible for the agent to fake success through text generation

Other benchmarks extend execution-grounded evaluation to live MCP services, where the reference state itself may change during or between evaluations. Frameworks such as LiveMCP-101 implement a dual-execution mechanism where a reference agent follows a human-curated plan while the evaluated agent operates independently against the same live services[49]. MCP-Universe similarly introduces dynamic evaluators that are engineered to retrieve contemporaneous backend information for time-sensitive tasks[50].

These approaches recognise that in live environments, a benchmark reference cannot always be treated as a permanently fixed label. Tool outputs, data availability, service behaviour, and externally hosted resources may change over time, meaning that an answer may differ from an earlier reference for legitimate environmental reasons rather than because the agent acted incorrectly.

This requirement creates a substantial curation burden. Dynamic evaluators, reference agents, and LLM judges are themselves fallible components: an incorrect reference parameter, an invalidated reference pathway, or an ambiguous scoring criterion can produce misleading evaluations. Therefore reference agents and dynamic evaluators must be validated not only when tasks are first created, but also when services, schemas, or data outputs change. LiveMCP-101, for example, reports extensive manual work to validate and correct its reference agents, while MCP-Universe similarly relies on post-creation verification of executable tasks and evaluators[49, 50]. These costs reveal a practical tension in dynamic MCP benchmarking: automation can reduce the cost of repeated evaluation, but the validity of the automated process still depends on sustained human oversight. For researcher-oriented data systems, this suggests that live evaluation should combine automated re-execution with explicit audit records and manual review

of flagged discrepancies, rather than treating either static labels or dynamic evaluator outputs as self-validating.

2.2.4 Positioning of the present study

The literature reviewed above demonstrates a clear progression in the evaluation of natural-language and tool-using systems. Earlier natural-language interfaces primarily focused on translating user requests into executable database queries or visualisation specifications[19, 23]. More recent LLM-agent benchmarks extend this focus towards multi-step planning, tool selection, parameter generation, execution, and repeated-run reliability[34, 35, 38]. MCP-specific benchmarks further examine whether agents can operate across large and unfamiliar tool ecosystems, coordinate tools distributed across servers, and complete executable tasks under live or partially dynamic conditions[42, 43].

These developments provide important foundations for the present study. However, they leave limited evidence about a more specific deployment setting: researcher-facing analytical access to a live, heterogeneous data infrastructure. Large-scale MCP benchmarks are designed to test broad ecosystem navigation, including discovery of relevant tools, routing across servers, and orchestration over extensive tool inventories. Such designs are valuable for assessing general MCP-agent capability, yet their aggregate results can combine several distinct sources of difficulty, including tool discovery, parameterisation, execution planning, service availability, and post-retrieval reasoning. They therefore provide limited isolation of whether an agent can accurately interpret, integrate, and communicate evidence once an appropriate research-data pathway has been identified.

Research-data systems also impose requirements that are only partially represented by conventional tool-use metrics. Social-media data are shaped by collection, platform, and processing conditions that affect what can reasonably be inferred from an observed result [2]. A response may be formally well structured, supported by executable calls, and broadly aligned with a plausible trajectory while still failing to communicate the evidential conditions needed for responsible interpretation. These conditions can include the availability and completeness of returned data, differences in collection coverage, processing limitations attached to derived analytical outputs, and the relationship between a requested measure and the pathway used to obtain it. The central question is therefore not only whether an agent used tools successfully, but whether its final answer

remains appropriately grounded in the evidence and constraints associated with those tool outputs.

Live research infrastructures create a further challenge for evaluation. Their data, collection coverage, processing pipelines, and service behaviour may change after a benchmark item has been created. A static expected answer can therefore become outdated even when an agent follows a valid execution pathway. Recent execution-grounded MCP benchmarks recognise this problem by evaluating agents against executable environments or dynamically refreshed references[42, 49]. However, such approaches also introduce a continuing validation burden: reference procedures, evaluators, and scoring criteria require auditing when backend behaviour changes. For researcher-oriented systems, reliable evaluation therefore requires an auditable reference state that records both the expected outcome and the executable pathway through which that outcome was obtained.

The present study addresses these issues through a constrained-realism benchmark for the AIReD[18]. It deliberately evaluates a narrower setting than broad multi-server MCP benchmarks. The model-facing tool surface is held constant in order to reduce variation caused by changing tool descriptions or discovery conditions, while the underlying AIO backend remains live. This design makes it possible to focus on a different stage of agent performance: the transformation of live, heterogeneous tool outputs into accurate and interpretable research-facing answers.

Accordingly, the study combines execution-grounded reference plans and live-system auditing with repeated task trials, outcome-level correctness evaluation, trace-level tool-use diagnostics, operational efficiency measures, and qualitative error analysis. It also examines exploratory behaviour under less constrained prompts, recognising that additional retrieval may sometimes improve contextual interpretation while also increasing cost and opportunities for error. The study does not aim to measure general MCP ecosystem navigation, autonomous research-question formulation, or full multi-turn research dialogue. Instead, it evaluates the reliability required for MCP-enabled agents to function as evidence-grounded natural-language interfaces to a live social-data research system.

Chapter 3

Benchmark Design

3.1 Design Principles

This benchmark adopts a constrained-realism design for evaluating MCP-enabled agents in a live social-data environment. The task set is intended to reflect plausible research operations supported by the Australian Internet Observatory’s AIReD system, while retaining sufficient specification and execution stability for repeated and comparable evaluation. The benchmark therefore evaluates an agent’s ability to answer bounded, tool-mediated research questions. It does not aim to evaluate the full process of open-ended research design, such as negotiating ambiguous user requests, formulating entirely new research questions, or obtaining evidence outside the available MCP toolset.

Research-oriented task realism. Tasks were designed to correspond to plausible analytical operations that a researcher could perform through AIReD. These operations include collection discovery, metadata inspection, filtered counting, temporal aggregation, cross-collection comparison, and analysis of NLP-derived outputs. Prompts were written as natural-language research requests and did not name internal tools directly. A task was considered realistic when its goal could reasonably arise during exploration of social-media data in the AIReD environment and could be addressed through capabilities exposed by the deployed MCP server.

Controlled evaluability. Realistic tasks still require sufficiently clear boundaries for model performance to be evaluated consistently. Each task therefore specifies the collection, time period, filters, metric, and expected answer form to the extent required for a reference execution plan to be constructed. Relative temporal expressions such as “yesterday”, “last week”, and “recently” were excluded because their meaning changes with the execution date. Prompts that admitted multiple materially different interpretations were rewritten or removed. For text-based tasks, variation in wording was permitted, but the central conclusion and supporting evidence had to be traceable to retrieved outputs. This principle supports consistent evaluation without requiring agents to reproduce one fixed sequence of tool calls.

Capability-oriented coverage. Coverage was defined at the level of analytical capabilities and execution demands rather than as exhaustive coverage of every tool endpoint or parameter combination. The benchmark includes tasks requiring metadata retrieval, constrained search, temporal aggregation, cross-source comparison, and interpretation of NLP-derived outputs. It also includes tasks with different source scopes, answer formats, and execution patterns, ranging from direct retrieval to multi-step synthesis. The resulting task set is therefore intended to sample substantively distinct forms of MCP-enabled analysis, rather than to provide a balanced factorial test of all possible tool configurations.

Execution feasibility in a live system. Because AIReD is a live research infrastructure, a task can be realistic in principle while remaining unsuitable for benchmark evaluation in practice. Candidate tasks were retained only when their reference calls could be executed successfully through the available MCP server and produced outputs that were sufficiently stable and interpretable for validation. Tasks were excluded when they required unavailable capabilities, depended on external knowledge, produced unresolved server errors, or relied on data that could not be meaningfully interpreted. This decision deliberately narrows the scope of the benchmark: it evaluates agent reasoning, parameterisation, and tool orchestration under an executable service configuration, rather than robustness to every possible infrastructure failure.

Together, these principles define the benchmark as a constrained but realistic evaluation of tool-mediated social-data research. The task set preserves the substantive structure

of AIREd research workflows while restricting ambiguity and infrastructure instability that would otherwise make model comparisons difficult to interpret.

3.2 Benchmark Task Taxonomy

Benchmark tasks were organised using a two-layer taxonomy. The primary layer classifies the dominant analytical objective of the user request. The secondary layer records execution-oriented properties that may affect agent performance, including source scope, expected answer form, and the type of reasoning or tool coordination required.

Each task was assigned one primary category according to the dominant analytical operation required to produce a satisfactory answer. This category does not imply that a task uses only one type of tool or contains only one analytical element. For example, a comparative task may involve temporal aggregation, while an NLP task may also require metadata retrieval. The primary category captures the central research objective; the secondary labels capture the execution demands through which that objective must be achieved.

This distinction is necessary because research intent and execution difficulty do not always align. A conceptually simple task may require multiple calls when a requested period exceeds the capacity of a single tool invocation. Conversely, a task involving a complex analytical concept may be resolved through one specialised endpoint, while still requiring substantial interpretation of its output. The two-layer taxonomy therefore separates what the user is asking from what the agent must do to answer it.

Primary task categories

The primary categories are as follows.

- Category A, Discovery and Dataset Overview, covers tasks that ask what data is available in the database. These tasks involve listing collections, retrieving collection summaries, or identifying which collections satisfy a particular condition. They test whether an agent can identify the relevant metadata tools and avoid hallucinating unsupported collections.

- Category B, Filtered Counting and Lookup, covers tasks that ask for the number of records matching specific constraints. These constraints may involve a collection, keyword, hashtag, language, author, or fixed date range. These tasks test whether an agent can map natural-language filters into valid tool parameters and produce a grounded count rather than an unsupported estimate.
- Category C, Aggregation and Temporal Pattern Analysis, covers tasks that require trends, distributions, or temporal structure. Examples include daily or monthly post counts, sentiment aggregation, and seasonality patterns by day of week or hour of day. These tasks test temporal reasoning, date-range handling, granularity selection, and correct use of aggregation tools.
- Category D, Comparative Analysis, covers tasks that require comparison or synthesis across collections, subsets, metrics, or time periods. These tasks are important in AIO because the system contains heterogeneous social media collections, while many tools operate on one collection at a time. Comparative tasks therefore test whether an agent can repeat calls with consistent parameters and synthesise the outputs into a coherent answer.
- Category E, NLP Analysis, covers tasks where terms, topics, topic groupings, or other NLP-derived outputs are the main object of analysis. These tasks test whether the agent can identify and use specialised NLP tools, handle ranked or structured outputs, and avoid substituting simpler search tools when NLP-specific analysis is required.

Secondary execution labels

At the secondary level, each benchmark task was annotated with execution-oriented properties that describe the form of the expected answer, the scope of data access required, and the type of agent behaviour needed to complete the task. These annotations complement the primary task taxonomy: while the primary categories capture the analytical purpose of a task, the secondary labels capture properties of its execution pathway and output requirements. They therefore support subsequent analysis of whether particular task demands are associated with differences in task completion, tool-use behaviour, reliability across repeated runs, or execution cost.

Answer type: The expected form of the task outcome. Four answer types were used: scalar, for tasks requiring a single numeric value or discrete factual result; time-series, for ordered values indexed by time; chart, for tasks requiring a graphical representation of structured results; and text, for tasks requiring a qualitative or interpretive written response. This label is relevant to evaluation because different output forms require different validation procedures: scalar and time-series answers can generally be assessed through deterministic comparisons, whereas textual analytical responses may require rubric-based evaluation.

Source scope: Whether a task concerns a single AIO collection or requires information from multiple collections. Tasks labelled `single_source` can be answered using data from one collection, even when multiple calls within that collection are required. Tasks labelled `multi_source` require retrieval or comparison across two or more collections. This distinction enables analysis of whether cross-collection access introduces additional difficulty for MCP-enabled agents, particularly in tasks requiring consistent filtering or comparison across heterogeneous platform datasets.

Task demand profile: Characterises the principal execution pattern required to answer the task. This label was introduced because the number of calls in a reference execution plan alone does not fully represent task demands. For example, a single tool call may return a complex NLP output that requires substantive interpretation, while several tool calls may simply repeat the same retrieval operation across collections. Four task demand profiles were therefore defined, as shown in Table 3.1. Task demand profiles were assigned with reference to a minimum valid execution pathway rather than to the observed behaviour of any particular model. Direct Retrieval applies when one result can be reported directly. Analytic Interpretation applies when one complex result requires substantive interpretation. Batch Retrieval applies when the main challenge is repeated homogeneous retrieval followed by mechanical combination of results. Synthesis applies when the task requires non-trivial integration across multiple results, tools, or analytical stages.

Table 3.2 summarises the distribution of the final 35-task benchmark across the secondary execution labels. Most tasks concerned a single collection (23/35, 65.7%), while 12 tasks required access to multiple collections. In terms of task demand, the suite

TABLE 3.1: Task demand profiles used to classify benchmark tasks.

Label	Operational definition
Direct Retrieval	Tasks that require the agent to extract the answer from a direct tool call.
Batch Retrieval	The same retrieval operation must be repeated independently across a predefined set of parameter values, such as collections, dates, or terms. The returned results are then listed, sorted, or combined through simple aggregation.
Analytic Interpretation	Tasks that can be solved through a single core tool call with a complex output. The agent must interpret trends, themes, or implications from potentially messy results.
Synthesis	Tasks that require the agent to coordinate multiple dependent, parallel, or output-producing operations and integrate the resulting information into a final analytical response.

TABLE 3.2: Composition of the benchmark task set.

Secondary dimension	Label	Tasks, n	Percentage
Source scope	Single-source	23	65.7%
	Multi-source	12	34.3%
Task demand profile	DR: Direct Retrieval	7	20.0%
	BR: Batch Retrieval	7	20.0%
	AI: Analytic Interpretation	8	22.9%
	SY: Synthesis	13	37.1%
Answer type	Scalar	8	22.9%
	Time series	2	5.7%
	Chart	3	8.6%
	Text	22	62.9%
Total task set		35	

contains seven direct retrieval tasks and seven batch retrieval tasks, alongside eight tasks requiring interpretation of analytical outputs and thirteen synthesis tasks requiring coordination or integration across multiple operations or results. The benchmark therefore places substantial emphasis on tasks that extend beyond direct information retrieval. Textual answers constitute the largest output type, reflecting the inclusion of comparative and exploratory analytical questions, while scalar, chart, and time-series outputs provide structured cases suitable for deterministic evaluation. The primary taxonomy was deliberately balanced, with seven tasks assigned to each of the five primary categories.

3.3 Task Construction

The final task set was constructed in three stages: manual template design, LLM-assisted candidate generation, and human validation. This process was intended to balance linguistic diversity with feasibility and alignment with the AIREd MCP server.

First, task templates were developed for each primary category using AIREd documentation, available MCP tool descriptions, and common analytical operations supported by the system. For example, filtered-counting templates specified a collection, query condition, and fixed date range, whereas comparative templates required the same measure to be retrieved across collections or time periods.

Second, GPT-5 was used through one-shot prompting to generate candidate task phrasings from these templates. The generation prompt included the benchmark context, category definitions, examples of valid tasks, and explicit constraints on ambiguity, temporal references, and expected answer verifiability. The model was used only to expand the candidate pool and diversify phrasing; final task selection and revision remained manual.

Third, candidate tasks were manually reviewed against the task taxonomy, the available MCP toolset, and the requirements for reference-answer construction. The following criteria were applied:

Feasibility against the current MCP server: Each candidate task was validated through manual execution of its proposed reference pathway against the currently available MCP toolset. Tasks were excluded where the reference pathway could not be completed end-to-end or repeatedly failed because of unresolved server-side limitations. This criterion did not exclude tasks in which an unavailable, empty, or incomplete result formed a meaningful and reproducible part of the evidential state required for the task. This helped ensure that benchmark failures primarily reflect agent behaviour rather than known limitations of the server.

Alignment with task category When a task involved multiple analytical aspects, the primary category was assigned according to the dominant analytic operation required to answer the task, rather than the topic, metric, or output format. This rule avoids

duplicating tasks across categories while still allowing tasks to preserve realistic execution complexity.

Temporal clarity Relative temporal expressions such as “yesterday”, “last week”, or “recently” were intentionally avoided because they make the expected answer dependent on the execution date. This improves reproducibility and ensures that the same benchmark task can be evaluated consistently across repeated runs and different models.

Verifiable outcomes Tasks were revised or excluded when the wording allowed multiple materially different answers that could not be adjudicated against retrieved evidence. Textual tasks were retained only when a central conclusion and supporting evidence could be specified in the ground truth, even where acceptable answers could vary in wording or level of detail.

Natural-language tool selection. Task prompts did not directly name internal MCP tools. This restriction ensures that the benchmark evaluates whether an agent can infer an appropriate tool-use strategy from a natural-language research request, rather than merely follow an instruction that specifies the required tool.

Together, these review criteria ensured that the final benchmark tasks were realistic, executable, category-aligned, and suitable for evaluating MCP-enabled agents as natural-language interfaces to the AIO data system.

3.3.1 Ground Truth Collection

Annotation

After the benchmark tasks were finalised, each task was annotated with a set of reference fields used for both answer validation and process-level analysis. These fields were designed to preserve the link between the natural-language task, the MCP calls required to answer it, the raw system outputs, and the final expected answer. The main annotation fields are as follows:

`ref_tool_call` stores a list of JSON objects specifying the MCP tool calls that are sufficient to answer the task. Each object records the tool name and its argument

values. This field acts as the reference execution plan for the task and is used to derive the expected result.

`core_toolset` stores the set of tool names considered necessary for completing the task. This includes tools appearing in the reference tool call sequence, as well as tools that support valid alternative execution pathways. For example, if a task could be completed either by using `get_collection_summary` then `get_all_terms` or by using `get_collection_summary` then `analyze_terms_in_collection`, then all three tools would be included in the `core_toolset`. This design avoids treating the manually selected reference plan as the only acceptable solution path, while still allowing tool-use behaviour to be evaluated against known valid tool affordances.

`raw_output` stores the raw outputs returned by executing each MCP call in `ref_tool_call`. This field is retained as an audit trail of the manual execution process and supports later inspection when discrepancies occur.

`ground_truth` stores the final expected answer for the task, derived from the reference tool-call outputs and represented in a task-specific JSON format. The structure of this field depends on the expected answer type. Scalar tasks store a value, unit, and optional comment. Time-series tasks store the temporal granularity, value unit, series labels, and ordered data points. Chart tasks store the chart type, underlying data, labels, and the URL or file reference for the generated image. Text-based tasks store a summary and the supporting evidence for the final conclusion. This normalisation allows heterogeneous MCP outputs to be evaluated consistently across different task types.

Several validation steps were used to improve the reliability of the collected ground truth:

First, the initial output for each task outcome was independently cross-checked against the AIREd graphical dashboard. For each task, the equivalent query was reconstructed through the dashboard interface, and the returned result was compared with the MCP-derived result. Only tasks whose final outputs were consistent across both interfaces were retained as finalized benchmark items. Second, the manually specified calls were then executed through the same MCP execution module used by the final benchmark runner. This reduces the risk that differences between reference outputs and model outputs are caused by client-side implementation differences rather than agent behaviour. Finally,

tasks were revised or excluded when repeated executions produced unstable responses within a short time frame.

All task construction, reference-plan specification, and ground-truth annotation were conducted by a single researcher. Although reference outputs were validated through repeated MCP execution and, where available, cross-checked against the AIREd dashboard, the task-selection and annotation decisions were not independently double-coded. Consequently, some degree of researcher judgment may remain in the definition of acceptable execution pathways, task constraints, and expected-answer criteria.

3.3.2 Challenges in live environment

AIO is a live data infrastructure rather than a frozen benchmark dataset. Collection summaries, temporal aggregations, and NLP-derived outputs may change as new data are harvested, historical records are backfilled, or downstream processing is updated. The benchmark therefore treats ground truth as execution-grounded and periodically refreshed, rather than permanently static.

To manage live-system change, a semi-automated audit was performed immediately before each benchmark run. The semi-automated workflow followed predefined operational directives (provided in Appendix D) for re-executing stored reference calls, comparing outputs, and generating audit records; all flagged discrepancies were manually reviewed.

For every task, the stored reference tool-call sequence was re-executed and the newly returned output was compared with the currently stored reference record. Any returned-content difference or reference-call execution failure triggered manual review. Where the original reference pathway remained valid and the difference reflected an interpretable update in the live AIO environment, the corresponding ground-truth record was refreshed using the same task-specific schema. Reference states could therefore differ between benchmark runs, particularly for collection-metadata tasks and some Bluesky-related tasks whose outputs changed over time. The audit process did not substitute alternative tool calls to repair a failing reference pathway. This preserved the distinction between a valid pathway whose returned evidence had changed and a pathway that

had become invalid. Accordingly, repeated-run results represent agent performance relative to the audited live-system state immediately preceding each run, rather than exact reproducibility against a permanently fixed answer key.

Chapter 4

Methodology

4.1 System architecture

The benchmark system was designed to evaluate how effectively large language models can operate as natural language interfaces to the analytical capabilities exposed by the AIReD project. This experimental framework consists of four principal components: a remotely hosted MCP server, a local MCP client, an OpenRouter-hosted language model, and a benchmark runner responsible for task execution, prompting, and trace logging.

For each benchmark task, the runner sends the natural-language query, system prompt, answer-format instruction, and model-facing tool definitions to the selected language model through the OpenRouter API. The model may either produce a final answer or request one or more tools. Requested tool calls are resolved locally and executed against the remote MCP server through the MCP client. The resulting observations are then appended to the model conversation history, allowing the model to continue the task iteratively until it produces a final answer or reaches the configured step limit. Throughout this process, the runner records both final outputs and intermediate tool-use traces for later evaluation. Figure [4.1](#) illustrates the overall architecture and data flow of the benchmark runtime.

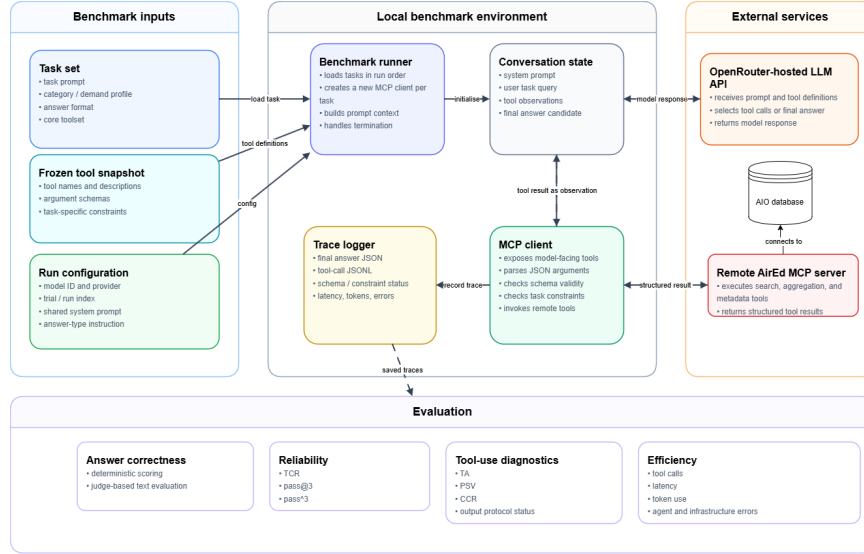


FIGURE 4.1: System architecture

4.1.1 Remote MCP Server

The benchmark operates against a remotely hosted AIO MCP server [18]. The local agent harness acted as an MCP client and connected directly to the server through its Streamable HTTP endpoint. The server exposes analytical capabilities over AIREd’s heterogeneous social media collections through a set of 15 tools, summarised in Table 4.1. They are grouped into five functional categories: system and collection metadata tools, temporal and seasonal aggregation tools, text and term analysis tools, topic-modelling and NLP metadata tools, and a chart-generation tool.

System and collection metadata tools provide basic information about the server and available datasets, including collection names, harvesting date ranges, and total post counts. Aggregation tools supported count- and sentiment-based analysis over temporal or seasonal groupings. Text and term tools supported full-text result counting, prominent-term retrieval, and term-level temporal analysis. Topic-related tools exposed topic clusters, topic similarity networks, and information about the associated NLP models. Finally, the visualisation tool (QuickChart API) allowed the agent to construct visual outputs from structured chart specifications.

The tool interface also imposed operational constraints relevant to benchmark difficulty. Temporal and seasonal aggregation tools accepted date ranges of up to 365 days, whereas most term- and topic-analysis tools restricted requests to intervals of at most 28 days. These differences mean that some long-horizon questions can be answered through a

TABLE 4.1: MCP tool capabilities used in the benchmark.

Capability category	Tools	Role in the benchmark
System and collection metadata	<code>get_api_version</code> , <code>get_collection_summary</code> , <code>get_collections</code>	Identify available collections and basic coverage information.
Temporal and seasonal aggregation	<code>aggregate_by_time</code> , <code>aggregate_seasonality</code>	Retrieve count or sentiment aggregates over time, week-day, or hour-of-day patterns.
Text and term analysis	<code>text_search</code> , <code>analyze_terms_in_collection</code> , <code>get_all_terms</code> , <code>get_term_daily_counts</code> , <code>get_nlp_terms_for_day</code> , <code>get_nlp_term_analysis</code>	Support keyword-count queries, prominent-term identification, and term-level temporal analysis.
Topic modelling and NLP metadata	<code>get_nlp_topics</code> , <code>get_nlp_metadata</code> , <code>get_topic_groupings</code>	Retrieve topic clusters, topic similarity networks, and metadata about NLP models.
Visualisation	<code>generate_chart</code>	Produce chart images from structured analytical results.

single aggregation call, while comparable term- or topic-oriented questions require repeated calls over multiple time windows followed by result integration. The constraints therefore create realistic differences in orchestration demand across benchmark tasks.

4.1.2 Interface Adaptation

Frozen tool snapshot: Although MCP supports tool discovery from a connected server, the benchmark did not dynamically rediscover the active tool definitions at the beginning of each experimental run. Freezing the tool surface ensured that all tested models and repeated trials were evaluated under the same interface conditions, preventing changes to the remotely hosted server or its tool descriptions from introducing uncontrolled variation into the benchmark.

Schema adaptation for OpenRouter function calling: Model inference was conducted through OpenRouter’s function-calling interface. The remote AIO MCP server exposed tool definitions in MCP format, whereas the OpenRouter API expected tool specifications in an OpenAI-compatible function-calling format [51]. The local client therefore adapted the frozen MCP tool definitions into model-facing function definitions before each inference request. Tool names and descriptions were preserved, and each

MCP `inputSchema` was represented as the corresponding function-parameter schema supplied to the model.

During execution, the model returned a structured function-call request containing a tool name and JSON arguments. The local client matched this request to the corresponding retained MCP tool definition, parsed and validated the arguments, issued the associated MCP tool call through the existing remote session, and returned the resulting observation to the model. The client therefore served as an interface adapter between OpenRouter’s model-facing function-calling protocol and the remote MCP-based analytical backend.

4.1.3 Agent

Execution Loop The agent uses an iterative tool-calling loop inspired by the action–observation structure of the ReAct framework. At the beginning of each task, the benchmark query is inserted into the conversation history together with the system prompt, and the frozen model-facing tool definitions.

At each iteration, the current conversation history is submitted to the selected language model through OpenRouter. The model reasons over the user query and the previous observations, then either returns a final answer or emits one or more tool calls. When a tool call is returned, the client parses its arguments, validates the call against the stored tool schema, and executes the requested tool through the active MCP session. The server response is serialised as a tool observation and appended to the conversation history using the tool-call identifier supplied by the model-facing API. The updated conversation is then passed back to the model in the following iteration.

This loop continues until the model produces an assistant message with no tool calls, which is treated as the final answer. Following standard practices in agent benchmarking, we restrict each task to a maximum of 30 agent steps. This constraint is enforced to prevent degenerate behaviours such as repetitive action loops and ensure evaluations remain computationally tractable [40]. If the agent reaches the step limit without producing a final answer, the system performs a final summarisation call without tools, forcing the model to answer based on the evidence collected so far.

Prompting and Output Format The agent system prompt consists of a shared system-level instruction and an answer-type-specific output instruction. The shared prompt defines general expectations for evidence-grounded tool use, including the requirement that answers be based on retrieved AIO results rather than unsupported model knowledge. The answer-type-specific instruction is appended according to the task’s expected answer type and specifies the required response structure for scalar, time-series, textual, or chart-generation tasks. The answer-format instruction specifies the required JSON schema for evaluation, but does not expose the benchmark taxonomy labels or task-demand labels used for later analysis. The full prompt can be found in Appendix [B.1](#).

This modular prompt design allows the same agent architecture to be reused across different task types, while preserving output structures suitable for evaluation. For example, scalar tasks require a numeric value and unit, time-series tasks require dated series points. By requiring structured JSON outputs, the benchmark supports automated comparison with reference answers while retaining enough descriptive content for qualitative error analysis.

4.1.4 Trace Logging

The benchmark runner records both answer-level and process-level traces for every experimental run. Run-level logs include the task identifier, model identifier, trial number, toolset identifier, token usage, number of steps, tools invoked, termination status, and final answer. Tool-call-level logs record the tool name, raw and parsed arguments, schema-validation outcome, constraint-compliance status, execution latency, execution success, returned observation, and any transport- or server-level errors.

All traces are stored as JSONL artefacts under the corresponding experimental run directory. These logs make each model trajectory inspectable and provide the evidence required for later evaluation of answer correctness, tool selection, parameter validity, constraint compliance, orchestration efficiency, latency, token cost, and failure modes. In particular, retaining intermediate calls makes it possible to distinguish an incorrect final answer caused by faulty reasoning from one caused by invalid parameters, temporary server failure, or incomplete tool exploration.

4.2 Experimental Setup

4.2.1 Model selection

We evaluated seven language models, consisting of four proprietary models and three open-weight models. The proprietary group includes Claude Opus 4.6, Claude Sonnet 4.6, GPT-5.5, and Qwen3.6-Plus. These models were selected to represent high-performing commercial systems that are likely candidates for practical deployment in LLM-based research interfaces.

The open-weight group includes Kimi-K2.6, DeepSeek-V4-Pro, and Llama-Scout. These models were selected to provide a contrasting set of alternatives that may be more relevant to future self-hosted or institutionally controlled deployments. This distinction is important in the context of research infrastructure such as the AIReD, where issues such as data governance, cost control, and deployment flexibility may make open-weight models attractive despite potential performance trade-offs. Kimi-K2.6 and DeepSeek-V4-Pro were included as recent, high-capability open-weight models, while Llama-Scout was retained as a widely available open-weight baseline from the Llama model family. Together, these models allow the study to examine how the selected open-weight configurations perform on MCP-based tool-use tasks relative to the selected proprietary configurations under a common agent architecture and benchmark setting.

All models were accessed through OpenRouter using the same local agent implementation. Models were run with the following generation parameters: `temperature: 0.0`, `top_p: 1.0`, `tool_choice: auto`, `stream: false`.¹ Reasoning-control parameters were not manually configured; each model therefore used the endpoint’s default reasoning configuration. Provider routing was fixed for each model with fallbacks disabled in order to reduce variation caused by automatic provider substitution. The detailed providers are documented in table 4.2.

¹The GPT-5.5 endpoint did not support the temperature parameter; it was therefore omitted.

TABLE 4.2: Evaluated models and providers.

Model	Provider
GPT-5.5 (2026-04-23) [9]	openai
Claude Sonnet 4.6 [52]	anthropic
Claude Opus 4.6 [52]	anthropic
Qwen 3.6 Plus [53]	alibaba
Kimi K2.6 [54]	deepinfra/fp4
Llama 4 Scout (17B-16E-Instruct) [10]	google-vertex/us-east5
DeepSeek V4 Pro [55]	deepinfra/fp4

4.2.2 Execution Procedure

The experiments were executed using a custom benchmark runner developed for this study. The runner was designed to support switching between task sets, model configurations, prompts, and evaluation settings. This implementation was used to ensure that each model was evaluated under the same task definitions, tool schemas, and logging procedure. The source code and configuration files are made available in the project repository.

A complete execution of the benchmark task set for a given model is referred to as a run. For each selected agent–model configuration, the full task set was executed three times, producing three independent trials for each task–model pair. Within each run, tasks were loaded in the order specified by the task CSV file. The runner also supported concurrent execution of task trials to reduce wall-clock runtime. During the reported experiments, up to 4 task trials were executed concurrently.

For each task trial, the runner instantiated a fresh MCP client and constructed the agent prompt by combining the shared system prompt with answer-type-specific instructions. The model was then allowed to interact with the MCP tools through the local client until it either produced a final answer, reached the configured stopping condition, or encountered an execution failure. During this process, the runner recorded both the final response and the full tool-call trace, including tool names, raw arguments, parsed arguments, schema validation results, execution status, returned payloads, token usage, latency, and error metadata. Because the runner was executed against a live remote MCP environment and externally hosted model inference APIs, some observed failures may arise from infrastructure conditions rather than agent decision-making. Provider-level failures include rate limiting, transport errors, authentication errors, connection failures, network errors, and timeouts. When such an error prevented the model from

completing a task trial, the trial was marked as failed and the error type and message were stored in the task-level result.

Tool-level infrastructure evidence was recorded separately in the per-tool-call JSONL trace. When an MCP tool response contained an HTTP-style error or server-side failure message, the runner extracted diagnostic fields such as `mcp_is_error`, `payload_has_error`, `server_error_message`, `http_status_code`, and `failure_stage`. If the agent still produced a final answer after receiving one or more problematic tool responses, the trial remained eligible for normal ground-truth evaluation. This distinction avoids treating every tool-side warning as an automatic benchmark failure while still preserving evidence for later error analysis.

After each experimental sweep, the runner generated a summary of completed, failed, and infrastructure-affected trials. Trials flagged as infrastructure-affected were inspected separately from ordinary model failures. Where a failure was attributable to provider or tool infrastructure rather than agent behaviour, the affected trial was rerun at a later time using the same configuration. Both the original failed trial and the rerun were retained in the trace archive, but only the rerun was included in the main task-completion analysis.

4.3 Evaluation Framework

The evaluation framework was designed to assess MCP-enabled agents at three complementary levels: outcome correctness, process-level execution behaviour, and operational efficiency. Outcome correctness measures whether an agent produced an accurate final answer to a benchmark task. Process-level diagnostics examine how the agent interacted with the available tools, including whether its calls were relevant to the task and whether generated arguments were formally valid and compliant with documented tool constraints. Operational efficiency measures the resources consumed during execution, including tool-call count, step count, token usage, and latency.

4.3.1 Outcome correctness evaluation procedure

Task completion rate is the primary metric we used to evaluate whether LLM agents can correctly answer natural-language research queries through MCP tools. It measures the proportion of tasks for which the model produces a correct outcome, scored as a binary pass/fail under task-specific criteria. Formally, for task (i) and trial (j), the result is denoted as:

$$r_{i,j} \in \{0, 1\}$$

For each model, the overall task completion rate across (N) tasks and (K) repeated trials was calculated as:

$$\text{TCR} = \frac{1}{NK} \sum_{i=1}^N \sum_{j=1}^K r_{i,j} \quad (4.1)$$

where $r_{i,j} = 1$ indicates that trial (j) for task (i) was evaluated as correct, and ($r_{i,j}=0$) otherwise. In this study, each task-model pair was executed three times, so ($K=3$). TCR therefore estimates the expected success rate of a single benchmark execution under the configured experimental conditions. In other words, it answers the question: if a researcher issued one benchmark query to the agent once, how likely would the agent be to return a correct answer?

4.3.1.1 Evaluator

Because the benchmark contains different answer types, correctness could not be evaluated using a single matching procedure. Instead, a hybrid approach was used. Each agent execution produced a structured JSON answer conforming to the expected output format associated with the task's answer type, as discussed in the earlier section. The final answer was then evaluated using either a judge-based evaluator or a deterministic evaluator.

Judge-based Evaluator Textual analytical tasks require agents to summarise or interpret retrieved results in natural language. These answers may be expressed through

multiple valid phrasings and levels of detail; exact string matching is therefore unsuitable. Textual responses were instead evaluated using a judge-based semantic consistency procedure.

We used Gemini-2.5-pro [56] as the judging model. It is a frontier-level reasoning model from a provider family distinct from the main evaluated agent models, reducing evaluator-family overlap and the risk of direct self-preference or family-preference bias. The model was accessed through OpenRouter using the google-vertex/global provider route. To improve reproducibility, the judge was run with deterministic decoding settings: temperature was set to 0.0 and top_p was set to 1.0. The judge is instructed to return a structured JSON object containing an ordinal correctness score, diagnostic indicators, and a brief explanation. The full prompt and an example can be found in Appendix B.2.

For each textual task run, the judge was provided with the following inputs: the original benchmark query, an evidence-grounded reference answer constructed from manual tool execution, and the final answer produced by the agent. The evaluator was instructed to judge factual and semantic consistency, assessing whether the agent preserved the main analytical conclusion and any interpretation-relevant limitations reported in the reference answer. This treatment was important for live-system tasks in which unavailable data, incomplete comparisons, or tool-returned errors formed part of the correct interpretation. For example, an answer that reported unavailable data as zero was treated as substantively incorrect where the reference answer identified the data as missing.

The evaluator assigned one of four ordinal scores: 2 for a correct answer, 1 for a partially correct answer, 0 for an incorrect answer, and -1 for an invalid or non-interpretable answer. This ordinal score and the evaluator’s structured diagnostic fields were retained for error analysis, while for consistency with the binary outcome evaluation used across all answer types, the score was collapsed into a binary task-completion result:

$$r = \begin{cases} 1, & \text{if judge score} = 2 \\ 0, & \text{if judge score} \in \{-1, 0, 1\} \end{cases}$$

Only fully correct textual answers were therefore counted as successful task completions. This strict threshold reflects the intended use of the system as an analytical interface: an

answer that communicates the broad conclusion while omitting a material limitation may still mislead a researcher and was consequently not treated as a successful completion. To ensure evaluation quality, each task trial was judged once, and random samples of passed agent answers alongside all text-answer failures were manually inspected for error analysis.

Deterministic Evaluator For scalar tasks, the structured answer value was compared directly against the manually established reference value. Numeric values were compared using a small tolerance to allow negligible rounding or representation differences.

For time-series and chart-based tasks, both the agent’s answer and the reference answer were normalized into a shared intermediate representation of ordered time-value pairs. Chart answers were evaluated purely through the structured data encoded in the chart payload rather than by inspecting the rendered visual image. Correctness was determined by verifying that the returned series matched the expected series across field names, date formatting, numeric representation, and (in the case of charts) the expected chart type and labels.

This approach was used because structured quantitative tasks have explicit expected values and can therefore be scored reproducibly without relying on model-based judgment. At the same time, the evaluator retained detailed diagnostics, including parse errors, missing fields, unit mismatches, structural mismatches, ordering errors, and numerical mismatches. These diagnostics were used to support later error analysis by distinguishing formatting failures from substantive analytical errors.

4.3.1.2 Reliability

By default, we report the average single-run success (TCR) as the main metric for comparing agents. Reliability was also an important dimension for the real-world user-facing system. Accordingly, $pass^k$ and $pass@k$, introduced in tau-bench [38], were adopted.

$$pass^k = \mathbb{E}_{\text{task}} \left[\frac{\binom{c}{k}}{\binom{n}{k}} \right] \quad (4.2)$$

$$\text{pass}@k = 1 - \mathbb{E}_{\text{task}} \left[\binom{n-c}{k} / \binom{n}{k} \right] \quad (4.3)$$

where n represents the total number of independent trials executed per task, c is the number of successful trials among those n attempts, k is the specific sample size of trials being evaluated, and $\mathbb{E}_{\text{task}}$ denotes the expected value (or average) computed across all tasks in the dataset.

Pass@k measures whether the agent succeeds in at least one of k independent trials. With our experiment setting of 3 trials per task, $\text{pass}@3$ is calculated as:

$$\text{pass}@3 = 1 - \mathbb{E}_{\text{task}} \left[\binom{3-c}{3} \right]$$

This metric captures recoverability: whether an agent is capable of completing a task correctly when given repeated opportunities.

pass^k measures whether the agent succeeds consistently across all independent trials. With three trials per task, pass^3 is calculated as:

$$\text{pass}^3 = \mathbb{E}_{\text{task}} \left[\binom{c}{3} \right]$$

This metric captures consistency: whether an agent can be relied upon to produce a correct result repeatedly rather than occasionally.

Together, these measures distinguish between agents that are capable of solving many tasks at least once and agents that solve the same tasks reliably across repeated executions. This distinction is particularly important for analytical interfaces, where researchers require repeatable results rather than occasional successful completions.

4.3.2 Process-level diagnostics

In addition to outcome-level task completion, this study evaluates agent behaviour through a set of process-level diagnostics derived from the execution trace. This design is motivated by the broader shift in agent evaluation toward trace-level assessment. Recent MCP-agent benchmarks, especially MCP-Bench[41], emphasize that evaluating tool-using agents requires attention to tool-level schema understanding and usage, trajectory-level

planning, and task completion, rather than relying only on final-answer correctness. Following this perspective, three complementary diagnostics are introduced: Tool Appropriateness Classification (TA), Parameter Schema Validity Rate (PSV), and Constraint Compliance Rate (CCR). Together, these metrics examine whether the agent selected relevant tools, generated formally valid tool arguments, and respected tool-specific constraints. These diagnostics are intended to explain how agents interacted with the MCP tool environment, rather than to replace final-answer correctness as the primary outcome measure.

4.3.2.1 Tool-Call Appropriateness Classification

For each benchmark task, a task-specific `core_toolset` is defined. This set contains the tools that are considered directly relevant to obtaining the task result when the task is completed manually. Additionally, a helper tool family (`get_api_version`, `get_collections`, and `get_collection_summary`) is defined. These tools are not always result-producing tools in themselves, but may reasonably be used by an agent to inspect the environment before issuing task-specific analytical calls. CoreToolSets were assigned during ground-truth construction based on the tools used to manually obtain the reference answer and, where applicable, other tools capable of producing equivalent evidence. Each tool call in an agent trajectory was then classified into one of three categories:

- Core call: the called tool belongs to the task’s CoreToolSet;
- Helper call: the called tool is one of the predefined helper-capable tools.
- Non-relevant call: any remaining tool call that is neither core nor helper.

For cases in which a metadata-oriented tool, such as `get_collection_summary`, is itself required to obtain the answer, its first relevant invocation is treated as a core analytical call. Additional invocations may be classified as helper calls where they serve only repeated verification or inspection. This classification does not require an agent to reproduce a single gold-standard tool trajectory. Different valid call sequences may still be recognised as task-relevant if they use tools capable of producing the required evidence. The resulting call composition is used to identify patterns such as irrelevant exploration, repeated metadata checking, or reliance on task-relevant analytical tools.

Tool Appropriateness (TA) hence can be computed as a weighted call-relevance indicator. At the task level, TA is computed as a weighted average over all tool calls:

$$\text{TA} = \frac{\alpha N_{\text{core}} + \beta N_{\text{helper}} + \gamma N_{\text{irrelevant}}}{N_{\text{total}}}, \quad \text{where } \alpha = 1, \beta = 0.5, \gamma = 0 \quad (4.4)$$

where N_{core} is the number of core analytical calls. N_{helper} is the number of permissible helper calls. $N_{\text{irrelevant}}$ is the number of non-relevant tools. N_{total} is the total number of tool calls.

These weights reflect the intended interpretation of the categories: core analytical calls receive full credit, helper calls receive partial credit, and non-relevant calls receive no credit. Under this formulation, a score of 1 indicates that all tool calls were core task-relevant calls, whereas lower scores reflect increasing reliance on helper calls or the use of tools that are not considered relevant to the task. Helper calls receive partial credit rather than being treated as fully appropriate or fully inappropriate, reflecting their intermediate role in practical agent behaviour.

Because the weighting of helper calls necessarily involves a modelling decision, TA is interpreted as a supplementary descriptive indicator of tool selection quality rather than a primary performance metric. The underlying numbers of core, helper, and non-relevant calls are retained for interpretation.

4.3.2.2 Tool call quality

Two more diagnostic scores are computed to help assess lower level tool calling quality

Parameter Schema Validity Rate (PSV) measures whether generated tool-call arguments conform to the formal input schema of the invoked MCP tool. For a query q , for each executed tool call, a binary schema-validity indicator is assigned:

$$s_i = \begin{cases} 1 & \text{if the tool arguments parse successfully} \\ 0 & \text{otherwise} \end{cases}$$

For a query containing (C) schema-checkable tool calls, PSV is calculated as:

$$\text{PSV} = \frac{1}{|C|} \sum_{c \in C} s_c \quad (4.5)$$

PSV is interpreted as an execution-validity diagnostic. It identifies failures caused by malformed or structurally invalid function arguments.

Constraint Compliance Rate (CCR) measures whether tool-call arguments satisfy documented operational restrictions that are not fully represented by structural schema validity. These include restrictions such as permitted date-window lengths for temporal, term, or topic-analysis tools. For each call to which a documented operational constraint applies, a binary compliance indicator q_c is assigned:

$$q_c = \begin{cases} 1 & \text{if the call satisfies all documented operational constraints checked for that tool} \\ 0 & \text{otherwise} \end{cases}$$

Let $C_{\text{con}} \subseteq C$ denote the subset of tool calls to which at least one documented operational constraint applies. CCR is calculated as:

$$\text{CCR} = \frac{1}{|C_{\text{con}}|} \sum_{c \in C_{\text{con}}} q_c \quad (4.6)$$

Calls to tools without an identified operational constraint are excluded from the denominator. Where a run contains no constraint-applicable calls, CCR is recorded as not applicable. CCR is treated as a diagnostic measure. A high CCR indicates that an agent generally respects documented tool restrictions, but does not by itself establish that the agent selected the correct tool, collected sufficient evidence, or produced an accurate final answer.

4.3.3 Qualitative Error Analysis Procedure

An inductive coding approach, informed by grounded-theory principles, was used to develop the error taxonomy for failed trials. For each failed trial, the researcher inspected the evaluator decision, final answer, tool-call trace, tool parameters, and relevant tool responses. Open analytic notes were recorded to identify what went wrong, the point in the trace at which the error first became evident, whether the agent attempted or

achieved recovery, and why the final answer was judged non-passing. Where several errors occurred within a single trace, the analysis assigned one primary failure category. This category was defined as the dominant reason for the non-pass judgment. The initial notes were then compared across failed trials to identify recurrent patterns. Similar observations were iteratively grouped, refined, and merged into higher-level error categories. The coding scheme was revised during this process until the remaining traces did not produce additional distinct error categories. Because the coding was conducted by a single researcher, the taxonomy is intended as an interpretive descriptive analysis rather than an estimate of inter-rater reliability.

Chapter 5

Results

This chapter presents the results of evaluating seven LLM agents on the AIREd MCP benchmark. It reports overall task completion, process-level diagnostics, and operational efficiency, followed by a qualitative analysis of recurring failure modes.

5.1 Task Completion

5.1.1 Average Task Completion Rate

Table 5.1 reports Task Completion Rate by model and task category. This is the expected probability of task completion on a single attempt. To account for model stochasticity, this was calculated as the mean success rate across $n=3$ independent trials per task.

Claude Opus 4.6 achieved the highest mean TCR (0.971), corresponding to successful completion in 97.14% of task-trial executions across the three runs. Claude Sonnet 4.6 and DeepSeek V4 Pro formed a second tier at 91.43%, followed closely by Kimi K2.6, Qwen3.6 Plus, and GPT-5.5, which achieved a TCR of 0.895, 0.886, and 0.876 respectively.

Llama4 was an outlier, performing substantially below all other evaluated models, with TCR of 0.067 and no successful trials in the three analysis-oriented categories. Given the magnitude of this gap, this result is best interpreted as a failure to operate effectively as a tool-using agent in this experimental setup, rather than as a general statement about llama models.

The performance gap between the best-performing model (Claude Opus, 0.971) and the second tier (0.914) was only 5.7%. In contrast, the gap between this second tier and Llama4 exceeded 84%. This indicates that, within this benchmark setting, most frontier models converged to a relatively similar level of capability, whereas the primary separation was between models that successfully functioned as MCP agents and those that did not.

The category-level results suggest that direct retrieval and simple lookup were not the main bottlenecks for most models.

Breaking down the TCR by research intent, a clear pattern has emerged: Except for DeepSeek and Llama, performance was generally highest on Category A Discovery and Dataset Overview tasks, with a perfect completion rate of 1. This suggests that when the task required identifying available collections or retrieving directly exposed metadata, most frontier and strong open-weight models could select the relevant tools and report the result correctly.

Filtered counting and lookup tasks remained highly tractable. The small decline for GPT-5.5, DeepSeek, and Qwen in these tasks is consistent with the additional difficulty of translating natural-language constraints into precise tool parameters (For example, translating natural language to Lucene query when performing text-search). Trace-level inspection identified parameterisation as one observed failure mode, although the aggregate TCR results alone do not isolate it as the sole explanation.

Completion rates were generally lower for the more analytical task categories. Comparative Analysis tasks produced the clearest separation between models. Claude Opus remained highly reliable, while other evaluated models showed larger declines relative to their performance on simpler retrieval-oriented tasks. Within this benchmark, these tasks required agents to coordinate multiple MCP calls, retain intermediate results, compare outputs across sources, and construct a synthesised final response. The lower completion rates are therefore consistent with multi-step comparison and synthesis introducing additional opportunities for error beyond basic evidence retrieval. However, because these tasks also differed in source scope, answer complexity, and data characteristics, the results do not isolate orchestration as the sole cause of lower performance.

TABLE 5.1: Mean task completion rate by model and task category.

Model	A	B	C	D	E	Overall
Claude Opus 4.6	1.000	1.000	1.000	0.952	0.905	0.971
Claude Sonnet 4.6	1.000	1.000	0.905	0.762	0.905	0.914
DeepSeek V4 Pro	0.905	0.905	0.905	0.905	0.952	0.914
GPT-5.5	1.000	0.905	1.000	0.714	0.762	0.876
Kimi K2.6	1.000	1.000	0.952	0.762	0.762	0.895
Qwen 3.6 Plus	1.000	0.952	0.857	0.905	0.714	0.886
Llama 4	0.286	0.048	0.000	0.000	0.000	0.067

Categories are: A = Discovery and Dataset Overview; B = Filtered Counting and Lookup; C = Aggregation and Temporal Pattern Analysis; D = Comparative Analysis; E = NLP Analysis. Bold values indicate the highest score within each task category, including ties. Overall task completion is computed across all benchmark tasks and repeated trials.

Exploratory NLP analysis showed a different pattern from comparative analysis. DeepSeek achieved the highest completion rate of 0.952, while Claude Opus and Sonnet remained strong. GPT-5.5, Kimi, and Qwen were less reliable. This pattern is consistent with challenges arising after tool execution, particularly when agents must interpret intermediate NLP outputs, avoid over-generalisation, and align the final response with the expected qualitative claim. The qualitative error analysis later in this chapter provides task-level examples of these failure modes.

5.1.2 Task completion rate by Task demand profile

TABLE 5.2: Mean task completion rate by model and task demand profile.

Model	AI	BR	DR	SY	Overall
Claude Opus 4.6	0.958	1.000	1.000	0.949	0.971
Claude Sonnet 4.6	0.917	1.000	1.000	0.821	0.914
DeepSeek V4 Pro	0.917	0.905	1.000	0.872	0.914
GPT-5.5	0.792	1.000	0.905	0.846	0.876
Kimi K2.6	0.833	1.000	1.000	0.821	0.895
Qwen 3.6 Plus	0.792	1.000	0.952	0.846	0.886
Llama 4	0.000	0.143	0.190	0.000	0.067

Bold values indicate the highest score within each task demand profile, including ties. Overall task completion is computed across all benchmark tasks and repeated trials. AI = Analytic Interpretation; BR = Batch Retrieval; DR = Direct Retrieval; SY = Synthesis

Table 5.2 reports mean task completion rate by task demand profile. While task categories describe the user-facing research intent of each task, the demand profile captures the main execution pattern required to answer it. Within this benchmark, observed

task completion did not vary simply with the number of tool calls required by the reference pathway. Batch retrieval tasks achieved very high completion rates across most non-Llama models, with Claude Opus, Claude Sonnet, GPT-5.5, Kimi, and Qwen all reaching 1.00. Within the Batch Retrieval tasks in this benchmark, the required operation was regular and explicitly specified. This pattern indicates that repeated calls alone were not sufficient to account for differences in observed task completion, although the profile also differed from other task types in prompt structure and answer requirements. This also explains the relatively high performance in task categories A and B, which mainly involve reporting the results retrieved from repeated calls to the same tool at different time periods or sets.

In contrast, analytic interpretation and synthesis tasks produced greater variation between models, which is consistent with the lower TCR observed in comparative and NLP-oriented task categories. Analytic interpretation required the model to explain or summarise intermediate analytical outputs, and completion rates dropped for GPT-5.5, Kimi, and Qwen. Synthesis tasks were also challenging, as they required the model to combine multiple intermediate results into a coherent answer. The lower scores for Sonnet, Kimi, GPT-5.5, and Qwen suggest that cross-result integration was a plausible contributor to failure in these benchmark tasks. The lower completion rates are consistent with difficulties arising after evidence retrieval, particularly during interpretation of tool outputs and construction of a final analytical response. This interpretation is supported by the later qualitative error analysis, which identifies several failures involving evidence interpretation, quantitative aggregation, and final-answer construction after relevant data had already been retrieved.

Claude Opus was the most robust model across demand profiles, remaining close to or above 0.95 in all categories. Sonnet and DeepSeek achieved the same overall TCR, but their profile-level outcome patterns differed in this benchmark: Sonnet obtained higher scores on the retrieval-oriented profiles, whereas DeepSeek obtained higher scores on the Synthesis profile. This illustrates that similar aggregate scores can conceal differences in where models succeeded or failed across the evaluated task set. For deployment decisions based on this benchmark, workload composition should therefore be considered alongside overall averages. Finally, Llama4 remained a clear outlier, with near-zero performance across most profiles, suggesting a broader failure of the tested configuration to function as a reliable tool-using agent.

5.1.3 Reliability Across Repeated Runs

TABLE 5.3: Overall single-trial completion and repeatability across three independent trials.

Mean TCR is calculated over 105 trials per model (35 tasks $\times$ 3 trials). Pass@3 is the proportion of tasks completed at least once across three trials. pass^3 is the proportion of tasks completed successfully in all three trials.

Model	Mean TCR	pass@3	pass^3
Claude Opus 4.6	0.971	1.000	0.914
Claude Sonnet 4.6	0.914	1.000	0.800
DeepSeek V4 Pro	0.914	1.000	0.771
Kimi K2.6	0.895	0.971	0.771
Qwen 3.6 Plus	0.886	0.971	0.771
GPT-5.5	0.876	1.000	0.771
Llama 4	0.067	0.114	0.029

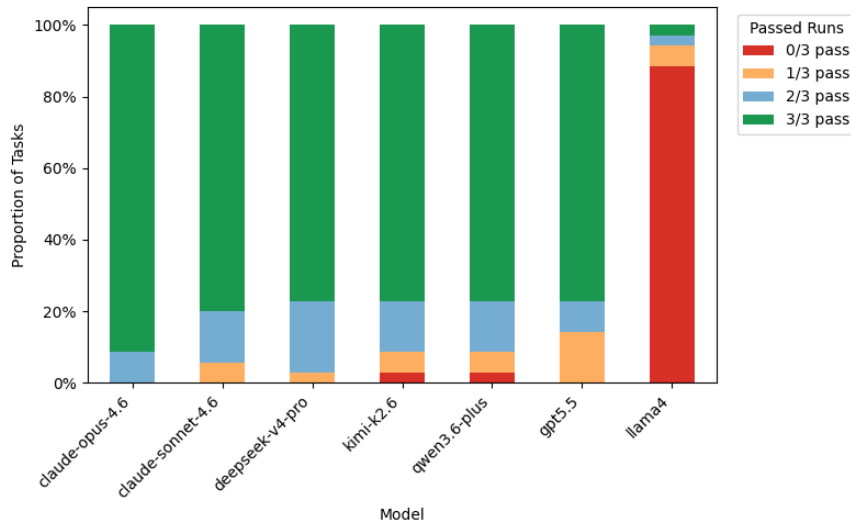


FIGURE 5.1: Distribution of task-level outcomes across 35 tasks for each model.

Each stacked bar represents the 35 benchmark tasks and shows the proportion of tasks completed successfully in zero, one, two, or all three trials. A larger green segment therefore indicates greater repeatability, whereas orange and blue segments indicate tasks with mixed outcomes across runs.

Each task–model pair was evaluated in three independent trials. Mean TCR reports the proportion of successful individual trials across the 105 trials conducted per model. Pass@3 reports the proportion of 35 tasks completed successfully in at least one of the three trials, whereas pass^3 reports the proportion completed successfully in every trial.

Pass@3 therefore indicates whether a task was solvable at least once under repeated evaluation, while *pass*³ provides the stricter measure of repeatability.

The results show a clear difference between at-least-once success and repeatable success. All non-Llama models achieved Pass@3 values of 0.971 or 1.000, meaning that 34 or 35 of the 35 tasks were completed successfully in at least one trial. Their *pass*³ rates were lower, ranging from 0.771 to 0.914. Claude Opus was the most repeatable model: it achieved a mean TCR of 0.971 (102 successful trials out of 105), completed all 35 tasks at least once, and completed 32 of 35 tasks successfully in all three trials. Thus, three tasks showed mixed outcomes across runs despite being solved at least once. Across the non-Llama models, between three and eight tasks per model had at least one failed trial. These results show that mean TCR or Pass@3 alone can overstate reliability when reproducibility across repeated executions matters.

The task-level distribution further clarifies this pattern, as shown in 5.1. For the non-Llama models, most tasks were completed successfully in all three trials, while a smaller group succeeded in only one or two trials. These mixed outcomes indicate that some failures were intermittent at the task level: the same task could be completed successfully in one independent run but fail in another. The subsequent error analysis examines the mechanisms associated with these inconsistent outcomes.

Repeatability also varied across demand profiles, as shown in Figure 5.2.

Batch Retrieval tasks were the most repeatable for most models: Claude Opus 4.6, Claude Sonnet 4.6, Kimi-K2.6, Qwen 3.6 Plus, and GPT-5.5 all achieved perfect *pass*³ on this profile. Direct Retrieval tasks were also highly repeatable for the stronger models, although Kimi, Qwen, and GPT-5.5 showed slightly lower stability than on Batch Retrieval. This suggests that direct retrieval was not always the easiest profile in practice. One possible interpretation is that some Direct Retrieval tasks involved more sensitive parameter choices, whereas batch retrieval tasks often provide a more explicit repeated structure. This may help models maintain consistent tool-use behaviour across trials.

Performance degradation is concentrated in more complex tasks. Analytic tasks show lower repeatability for most models. These tasks can often be solved with a single core tool call, but the returned output is complex and requires interpretation of trends,

themes, or implications. The lower *pass*³ scores indicate that this variance in repeatability is not solely driven by the number of tool calls. Instead, instability in these cases is more likely to arise post-retrieval—specifically during evidence interpretation and answer construction. Synthesis tasks also show reduced repeatability, with the strongest model, opus-4.6 achieving a score of 0.85 while the rest is below 0.70. Although similar to BR tasks, which also require multiple structured calls, these tasks differ in requiring the agents to integrate multiple tool outputs into a final analytical response. This structural difference corresponds to a significant drop in consistency performance (e.g. GPT-5.5, Kimi, Qwen decreases from 1 to 0.69).

Note that because *pass@3* is computed over a limited set of tasks per demand profile, a few highly unstable tasks can skew the aggregate score. Consequently, these findings should be read in conjunction with the subsequent task-level error analysis.

Overall, within this benchmark, repeatability was highest for profiles involving regular and explicitly specified retrieval sequences, and lower for profiles requiring interpretation of analytical outputs or integration of multiple retrieved results. The subsequent error analysis examines the concrete failure mechanisms associated with these less repeatable outcomes.

5.2 Process-Level Diagnostics

This section examines process-level properties of MCP tool use that are not captured by task completion alone. Two levels of behaviour were assessed. Parameter Schema Validity (PSV) and Constraint Compliance Rate (CCR) evaluate whether generated calls were structurally executable and complied with tool-specific constraints. Tool Appropriateness (TA) evaluates how closely the overall tool-use trajectory aligned with the benchmark’s coded core and supporting retrieval requirements. These measures diagnose different aspects of behaviour and should not be interpreted as direct measures of final-answer correctness.

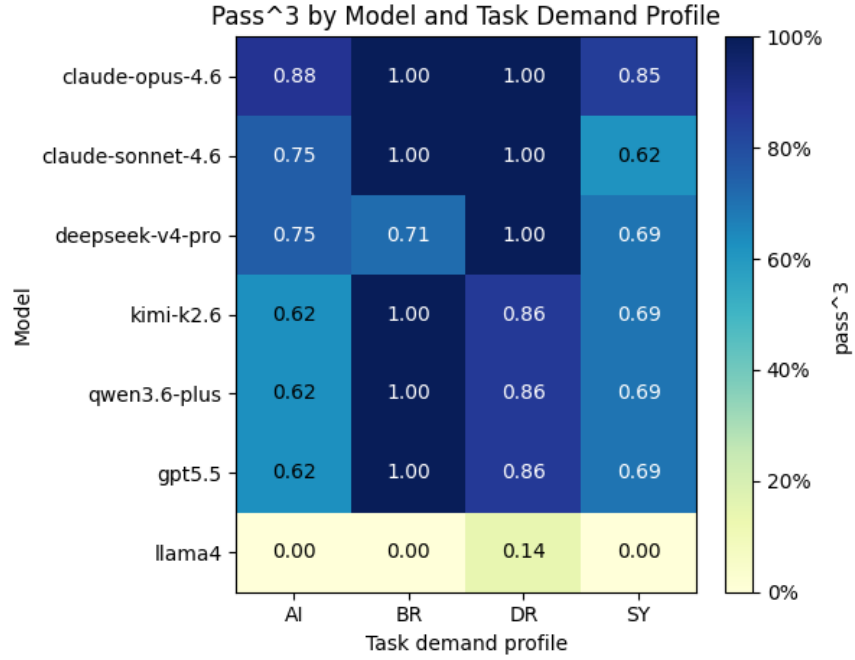


FIGURE 5.2: $pass^3$ heatmap by model and task demand profile. A = Analytic Interpretation; BR = Batch Retrieval; DR = Direct Retrieval; SY = Synthesis. Each cell shows the proportion of tasks within a demand profile that were completed successfully in all three independent trials. Demand profiles contain different numbers of tasks: AI (8), BR(7), DR (7), and SY (13).

5.2.1 Tool-call generation

TABLE 5.4: Mean parameter schema validity and constraint compliance by model.

Model	Mean PSV	Mean CCR
Claude Opus 4.6	1.000	0.917
Claude Sonnet 4.6	1.000	0.913
DeepSeek V4 Pro	1.000	0.901
GPT-5.5	0.998	1.000
Kimi K2.6	1.000	0.913
Llama 4	0.986	0.770
Qwen 3.6 Plus	1.000	0.919

Parameter Schema Validity (PSV) and Constraint Compliance Rate (CCR) were used to assess the formal validity of generated MCP calls. PSV measures whether a call satisfied the input schema of the selected tool. CCR measures whether schema-valid calls respected documented operational restrictions of the invoked tool that were not fully

represented in the formal input schema. These restrictions included, for example, maximum permitted date-window lengths for temporal, term, or topic-analysis endpoints. CCR therefore evaluates whether an agent operated within tool-level limits. CCR was calculated only for calls to which explicit tool constraints applied. Calls without assessable tool-specific constraints were excluded from the CCR denominator. Within each run, both metrics were calculated at the task level and macro-averaged across tasks, ensuring that tasks with many tool calls did not dominate the aggregate result. Table 5.4 reports the mean of these run-level scores across the three independent runs.

PSV exhibited a pronounced ceiling effect. Claude Opus, Claude Sonnet, DeepSeek, Kimi, and Qwen achieved mean PSV values of 1.000, while GPT-5.5 and Llama4 achieved 0.998 and 0.986 respectively. Structural function-call validity was therefore rarely a limiting factor in this benchmark. Even Llama4, which performed poorly on task completion, generally produced calls that conformed to the formal schemas exposed by the MCP server.

CCR showed greater differentiation. GPT-5.5 achieved perfect constraint compliance, while the other non-Llama models clustered between 0.901 and 0.919. Llama4 achieved a substantially lower mean CCR of 0.770. This difference between PSV and CCR indicates that the more common observable issue was not malformed syntax or missing required parameters, but selection of values that departed from tool constraints, such as an unsupported date range, filter, or level of temporal aggregation.

These results narrow the explanation for failures among the non-Llama models. Their lower TCR values cannot primarily be attributed to malformed calls or directly observable violations of stated retrieval constraints. However, high PSV and CCR should not be interpreted as evidence that the retrieval process was substantively adequate. These measures do not assess whether the selected tool was appropriate for the analytical goal, whether the retrieved evidence was sufficient, whether relevant calls were omitted, or whether the final answer accurately interpreted the returned data. The following TA analysis therefore examines tool selection at the trajectory level, while the qualitative error analysis investigates failures in evidence interpretation, quantitative reasoning, and final-answer construction.

5.2.2 Tool Selection

TABLE 5.5: Mean tool-call composition and tool appropriateness by model.

Model	Calls/task	Core/task	Helper/task	Inappr./task	Mean TA
GPT-5.5	4.829	4.105	0.600	0.124	0.917
Kimi K2.6	7.171	4.429	1.219	1.524	0.849
Claude Opus 4.6	4.781	3.724	0.391	0.667	0.840
Qwen 3.6 Plus	6.038	3.991	1.086	0.962	0.836
Llama 4	1.819	1.305	0.295	0.219	0.815
Claude Sonnet 4.6	5.181	3.667	0.619	0.895	0.793
DeepSeek V4 Pro	6.838	4.162	1.191	1.486	0.725

Tool Appropriateness (TA) provides a complementary measure of how tool-use trajectories aligned with the benchmark’s coded core and supporting retrieval requirements. Table 5.5 reports the mean number of calls per task, their composition, and the resulting TA score for each model.

GPT-5.5 achieved the highest overall TA score (0.917) and made the fewest calls coded as inappropriate (0.124 per task). Its tool-use trajectories were therefore comparatively concentrated around calls that directly supported the benchmark tasks. DeepSeek V4 Pro and Kimi K2.6 made more calls per task and also made more calls coded as inappropriate. This pattern indicates lower alignment with the benchmark’s coded core/helper pathway, although the aggregate counts alone do not establish why those additional calls occurred. They may reflect redundant retrieval, recovery behaviour, broad exploratory investigation, or genuinely unsuitable tool selection.

The TA ranking did not mirror the TCR ranking. GPT-5.5 achieved the highest TA and CCR, yet its overall TCR remained below that of Claude Opus, Claude Sonnet, and DeepSeek. Conversely, DeepSeek achieved the lowest TA score among the non-Llama models while retaining a relatively high TCR. TA should therefore be interpreted as a measure of tool-use concentration and task-pathway alignment, rather than as a direct proxy for answer correctness or overall agent capability.

Llama4 further illustrates this distinction. It achieved a mid-range TA score of 0.815, despite an overall TCR of only 0.067 and an average of just 1.819 calls per task. Because TA is calculated over calls that were actually issued, it cannot directly penalise agents

that omit necessary retrieval steps, terminate prematurely, or fail during final-answer generation. Its TA score should therefore not be interpreted as evidence of comparable end-to-end tool-use competence.

TABLE 5.6: Mean tool appropriateness by model and task demand profile.

Model	AI	BR	DR	SY	Overall Mean TA
GPT-5.5	0.899	0.988	0.845	0.928	0.917
Kimi K2.6	0.840	0.989	0.843	0.783	0.849
Claude Opus 4.6	0.688	0.986	0.960	0.791	0.840
Qwen 3.6 Plus	0.875	0.976	0.750	0.783	0.836
Llama 4	0.904	0.967	0.872	0.626	0.815
Claude Sonnet 4.6	0.656	0.988	0.879	0.726	0.793
DeepSeek V4 Pro	0.532	0.900	0.767	0.722	0.725

Bold values indicate the highest tool appropriateness score for each task demand profile.

TA was also computed by task-demand profile, as shown in Table 5.6 .

TA varied systematically across task demand profiles. Batch Retrieval tasks produced high TA scores for every model, ranging from 0.900 to 0.989. These tasks involved relatively regular and explicitly specified retrieval sequences, suggesting that most models could identify the relevant tool families and maintain an appropriate sequence of calls when the workflow was tightly constrained. TA was also generally high for Direct Retrieval tasks, although the range was wider, from 0.750 for Qwen to 0.960 for Claude Opus.

In contrast, Analytic Interpretation showed a different relationship between process alignment and task success. Claude Opus, Claude Sonnet, and DeepSeek achieved comparatively high Analytic TCR values of 0.958, 0.917, and 0.917, while their TA scores for the same profile were lower at 0.688, 0.656, and 0.532. Manual trace inspection suggests that this pattern was partly associated with additional chart-generation and analytic calls beyond the minimal benchmark pathway. Some of these calls may have supported richer interpretation, even where they reduced strict alignment with the coded core/helper plan. TA in this setting should therefore be read primarily as an indicator of reference-pathway conformity, rather than as an unqualified measure of tool-use quality.

The converse pattern also occurred. GPT-5.5 achieved high TA scores in both Analytic Interpretation (0.899) and Synthesis (0.928), while its TCR for these profiles was lower at 0.792 and 0.846. A focused and constraint-compliant tool-use trajectory was therefore insufficient to guarantee successful interpretation or synthesis of the retrieved evidence. Together, these patterns reinforce the conclusion that tool-call validity, constraint compliance, and tool appropriateness capture important but incomplete components of MCP agent performance.

To examine whether Tool Appropriateness was mainly affected by failed trials, TA was recomputed using successful trials only. The overall profile-level pattern remained similar for the six models with successful trials across all four demand profiles. Batch Retrieval and Direct Retrieval tasks continued to show comparatively high TA, whereas Analytic Interpretation and Synthesis tasks showed lower and more variable scores. For most comparable model-profile cells, outcome filtering produced only small changes in TA. The largest change occurred for Kimi K2.6 on Synthesis tasks, where TA increased from 0.783 to 0.830. More moderate changes were observed for Claude Sonnet on Synthesis tasks (0.726 to 0.759) and DeepSeek V4 Pro on Synthesis tasks (0.722 to 0.699). The persistence of the broader profile pattern indicates that lower TA in Analytic Interpretation and Synthesis tasks was not solely an artefact of unsuccessful trials.

Overall, PSV and CCR were compressed near the upper bound for most frontier models, while TA revealed meaningful differences in how models structured their tool-use trajectories. None of these process-level measures, however, fully explained the observed variation in task completion. The qualitative error analysis in the following section examines the downstream mechanisms that remained consequential after valid and broadly appropriate retrieval had occurred.

5.3 Efficiency

This section examines the operational burden associated with MCP-enabled tool use. In addition to task completion, a researcher-facing system must provide usable answers within an acceptable time and resource budget. The analysis therefore considers token consumption, estimated API cost, end-to-end latency, and tool-call volume. End-to-end

latency includes model inference, sequential MCP tool calls, server-side processing, network communication, and provider-level variation. Estimated API costs were calculated from recorded token usage using the listed OpenRouter pricing for each evaluated model and provider, retrieved in June 2026. These estimates exclude MCP hosting, database processing, network, and other infrastructure costs.

TABLE 5.7: Efficiency and resource use by model.

Model	Tokens/success	Latency	Calls/task	Avg. TCR
Claude Opus 4.6	19,620.81 (680.82)	24.54 (0.55)	4.78 (0.24)	0.971
Claude Sonnet 4.6	23,102.77 (2,135.67)	23.34 (0.76)	5.18 (0.02)	0.914
DeepSeek V4 Pro	30,729.43 (3,319.50)	52.96 (4.53)	6.91 (0.48)	0.914
Kimi K2.6	24,796.74 (1,340.10)	83.54 (10.13)	7.17 (0.49)	0.895
Qwen 3.6 Plus	26,467.47 (674.24)	53.42 (3.91)	6.04 (0.41)	0.886
GPT-5.5	11,802.98 (1,336.63)	22.46 (0.94)	4.83 (0.48)	0.876

Values in parentheses report standard deviation across three runs. Latency is measured in seconds.

Table 5.7 presents the token per successful task, mean latency, and mean tool calls per task alongside the overall TCR. Figure 5.3 presents average TCR vs Agent API Cost per Successful Task. Tokens per successful task and estimated API cost per successful task were calculated as total observed resource use across all trials divided by the number of successful trials. Latency and tool-call counts are mean values per trial. Latency is measured in seconds. Llama $\tilde{4}$ was excluded because its very small number of successful trials made per-success estimates unstable.

The results show that task completion and operational efficiency did not follow a single ranking. Claude Opus achieved the highest average TCR (0.971) while also using a comparatively compact tool-use trajectory, averaging 4.78 calls per trial. Its token consumption per successful task, at 19,621 tokens, was lower than that of Claude Sonnet, DeepSeek, Kimi, and Qwen. However, Figure 5.3 shows that this performance was associated with the highest estimated API cost per successful task among the evaluated configurations.

Claude Sonnet and DeepSeek achieved identical average TCR values of 0.914, but their operational profiles differed substantially. DeepSeek used 30,729 tokens per successful task, averaged 6.91 calls per trial, and required 52.96 seconds per trial. These values

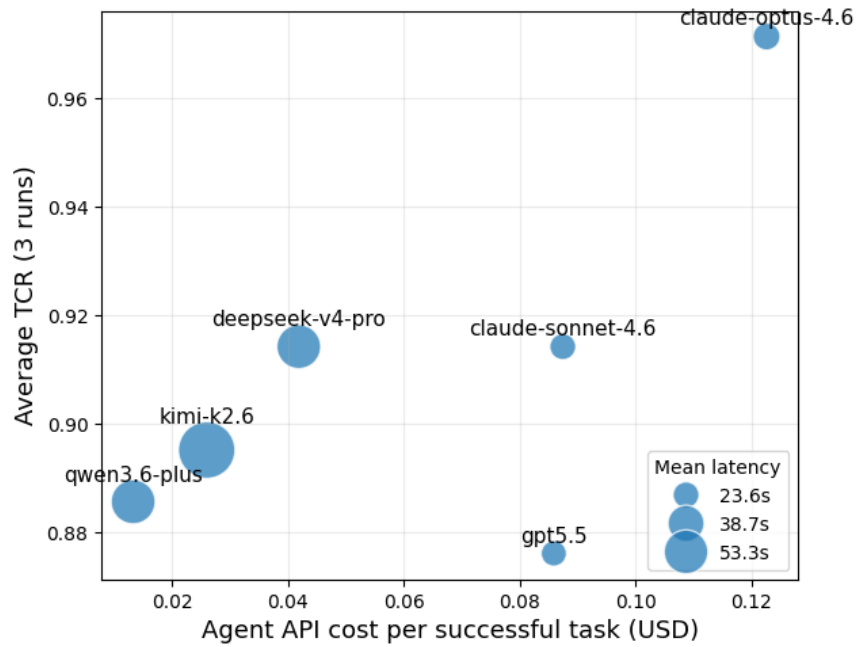


FIGURE 5.3: Average TCR vs Agent API Cost per Successful Task
Marker size encodes mean end-to-end latency

were substantially higher than Claude Sonnet’s corresponding values of 23,103 tokens, 5.18 calls, and 23.34 seconds. DeepSeek therefore achieved comparable task completion at a lower estimated API cost, while imposing a markedly greater latency and interaction burden.

GPT-5.5 showed the lowest token consumption per successful task, at 11,803 tokens, and the lowest mean latency, at 22.46 seconds. It also used one of the shortest tool-call trajectories, averaging 4.83 calls per trial. Its average TCR, however, was 0.876, below the strongest-performing configurations. This illustrates that a compact and fast trajectory can still yield lower end-to-end completion when later stages of interpretation, synthesis, or output construction fail.

Kimi and Qwen occupied less favourable positions on the observed latency–completion trade-off. Kimi produced the highest mean latency at 83.54 seconds and the highest average number of tool calls at 7.17 per trial, despite achieving an average TCR of 0.895. Qwen required 53.42 seconds and 6.04 calls per trial, with an average TCR of 0.886. These results indicate that lower estimated API expenditure does not necessarily translate into lower researcher-facing waiting time. Longer sequential tool-use trajectories can increase total response time even when token-level model pricing is comparatively low.

No evaluated configuration simultaneously maximised task completion, minimised estimated API cost, and minimised latency. The results therefore support a deployment-oriented interpretation of model selection. Where completion reliability is the dominant requirement, Claude Opus provided the strongest observed performance. Where cost or response time is more constrained, other configurations may be preferable, subject to the lower completion rates or longer tool-use trajectories observed in this benchmark.

5.4 Qualitative Error Analysis

This section reports a qualitative error analysis of failed task–model trials. The purpose of the analysis was to identify recurring failure patterns that were not captured by aggregate task completion rates alone. 57 failed trials from 630 non-Llama task-model trials were manually inspected using trace-level logs, including evaluator decisions, final answers, tool-call sequences, parameter values, and raw tool outputs. The resulting taxonomy is shown in Table 5.8.

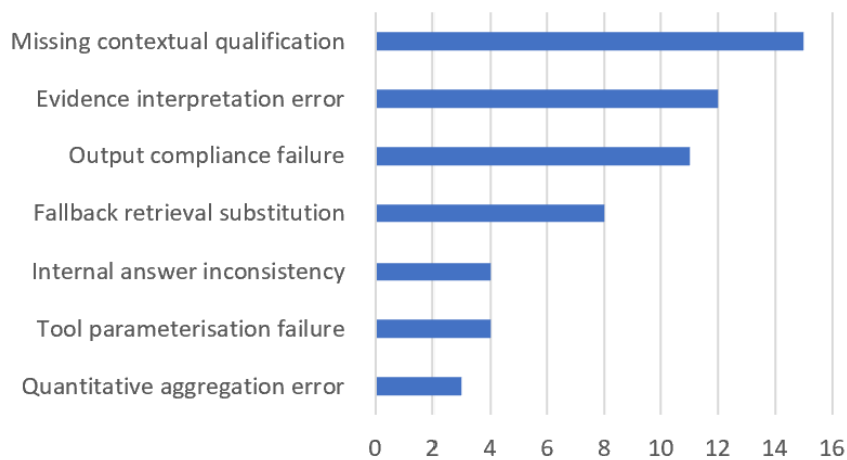


FIGURE 5.4: Distribution of primary failure categories across failed task–model trials

5.4.1 General Findings

Figure 5.4 shows the distribution of primary failure categories across failed task–model trials. Each failed trial was assigned one primary error category based on the dominant reason for the non-pass judgment. Some trials exhibited multiple issues, but this figure reports only the primary coded cause. Several patterns were associated with particular

models or task types, although these observations should be interpreted descriptively due to the limited number of failed trials.

Output compliance failures were most visible in Qwen’s failed trials, where 7 out of 11 failures involved no extractable JSON object in the final response. Since the benchmark parser attempted to recover the first JSON object appearing anywhere in the response, these cases suggest a stronger form of response-protocol failure rather than parser fragility. However, given the small number of failed trials, this should be interpreted as a descriptive pattern within this experimental configuration rather than a stable model-level characteristic. The remaining four failures were less severe and occurred across different models, in which the model omitted the required unit for numerical output.

Missing contextual qualification errors were concentrated in category E tasks, particularly E6 and E7, and appeared across most models. These failures occurred when the agent’s answer was mostly correct but omitted a required caveat, limitation, comparison, or contextual condition from the ground truth.

Fallback retrieval substitution was concentrated in D3 and D4. Both tasks intentionally involved cases where a tool call returned unavailable or empty data, and required reporting it faithfully in the final answer in the ground truth. In these cases, several agents attempted to recover by modifying the query, changing parameters, or using alternative retrieval strategies. Under the benchmark criteria, this was incorrect when the agent presented the fallback result as if it directly answered the original task. However, this pattern is better interpreted as a task-sensitive observation about missing-data handling than as a general model failure mode.

Quantitative aggregation errors and internal answer inconsistencies appeared in textual-answer tasks that require calculation over numbers. In these cases, agents retrieved the relevant evidence, but when reporting the numerical results, they made mistakes on the final calculation. These errors were not observed in the scalar or time-series answer formats, which enforce stricter output structures.

Evidence interpretation errors generally took two forms. First, multiple models made errors during textual tasks requiring trend analysis. Second, agents often incorrectly treated missing data as zero. This typically occurred when tasks requested data over a

continuous time range, but the tool’s output contained gaps. While substituting zero for missing data is a plausible assumption, it is highly problematic in real-world data systems because it distorts the underlying meaning of the retrieved evidence.

Tool parametrisation failures were relatively rare as direct causes of failed trials, appearing in only three failed cases. These included overly permissive clustering thresholds and ambiguous text-search Lucene queries (for example, search “hashtag:bushfire” and “#bushfire” gives different results), which did not change the broad conclusion of the response but gave different numerical values that contradicted the ground truth.

Llama 4 responses were treated as an outlier and analysed separately. Unlike the other models, its dominant failure mode was Output compliance: 89 of its 98 failed trials did not produce an extractable JSON object, despite the structured output requirement. This pattern may reflect weaker structured-output control under the benchmark prompt and serving configuration, rather than a single failure mechanism. As a result, detailed categorisation using the main error taxonomy was less informative, since most failures occurred before substantive task evaluation could be applied. Among the small subset of failed trials that did produce valid structured output, errors more often involved failure to invoke required tools, semantically misaligned parameter choices, or substantive misinterpretation of retrieved evidence. This suggests that the model’s difficulty was not limited to output formatting alone. Rather, Llama 4 exhibited failures across both the response-protocol layer and the downstream analytical layer. This pattern is consistent with recent work [57] showing that structured-output reliability and function-calling instruction adherence remain open challenges for LLM agents, and that even well-formed tool-using responses can still fail because of semantic misalignment or output-interpretation errors.

Overall, the most frequent agent-level failure patterns were not simple failures of intent recognition, tool selection, or schema-valid tool-call generation. Instead, In many failed trials, agents had already retrieved relevant data but failed during subsequent interpretation, quantitative aggregation, contextual qualification, and final-answer construction. This suggests that reliable MCP-based research agents require more than successful tool invocation. They must also preserve the evidential basis, limitations, and provenance of retrieved outputs when translating tool results into researcher-facing answers.

TABLE 5.8: Error taxonomy for failed benchmark trials.

Error taxonomy	Example	Description	Typical location
Output compliance failure	No extractable JSON object in the final response; malformed JSON; missing unit.	The agent may have solved the task substantively, but failed to return the required output format.	Across all categories
Tool parameterisation failure	Overly permissive threshold; ambiguous search term.	The agent selected an appropriate tool, but used parameters that changed, weakened, or broadened the result.	Across all categories
Fallback retrieval substitution	Use of alternative tools after an unavailable or empty result.	The agent received an empty, unavailable, or failed result, then attempted to recover by changing tools or parameters. However, it did not clearly report that the answer was based on an alternative method, and instead presented the result as if it directly answered the original task.	D4,D3
Quantitative aggregation error	Calculation error; numerical precision loss.	The agent retrieved relevant data but miscalculated, rounded incorrectly, or misreported the numerical result.	Tasks requiring textual answers
Missing contextual qualification	Omitted limitation; missed comparison.	The answer was partly correct, but missed a key condition, caveat, limitation, or explanatory detail needed for correct interpretation.	E7,E6
Evidence interpretation error	Incorrect trend interpretation, Missing or incomplete data reported as zero.	The agent misread the retrieved evidence or drew an unsupported conclusion from tool outputs.	C1
Internal answer inconsistency	Contradiction between reported evidence and final summary.	The answer contains claims that conflict with the agent’s own retrieved evidence, intermediate calculations, or final summary.	Tasks requiring textual answers

5.4.2 Example of task failures

Example 1: Fallback Retrieval Substitution (Task D4)

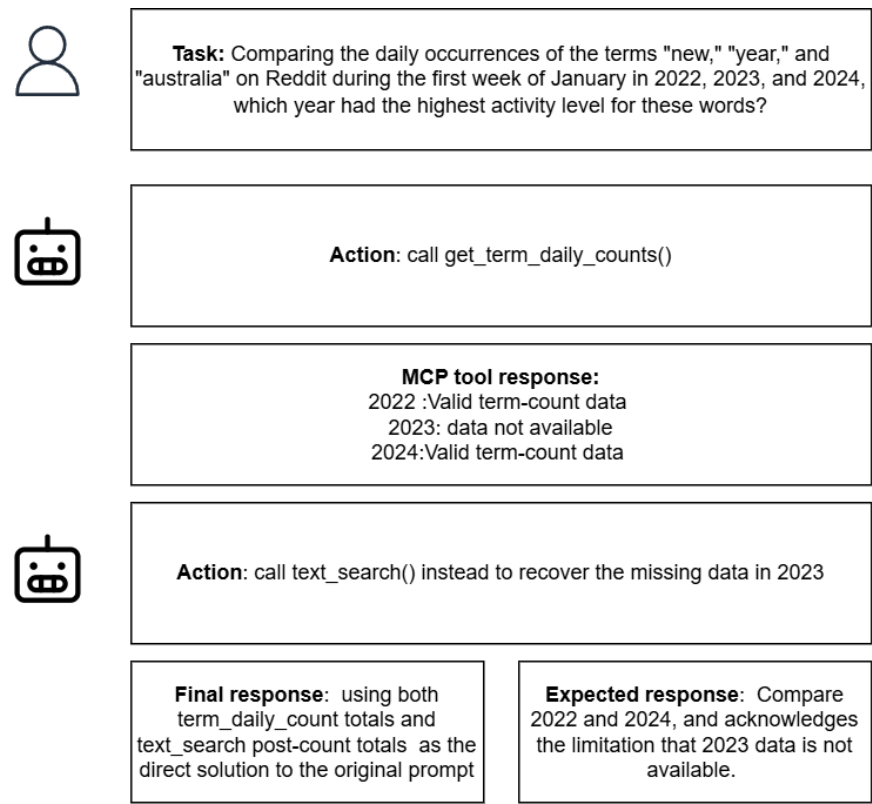


FIGURE 5.5: Task D4

Task D4 exposed a recurring failure mode in which agents attempted to compensate for unavailable data by substituting alternative retrieval strategies. When the required 2023 term-count endpoint could not be accessed, several agents reformulated the problem and issued a large number of text-search queries instead. While this behaviour demonstrated robustness in the face of retrieval failure, it also changed the evidential basis of the answer. Counts derived from post-search results were treated as equivalent to the original term-count statistics, despite being generated through a different retrieval process.

This substitution introduced two problems. First, it increased operational cost through additional MCP calls, resulting in higher token consumption and latency. More importantly, it weakened the traceability of the final result. The answer was no longer directly supported by the dataset requested by the user, but by a reconstructed approximation derived from multiple secondary queries. Although the agent ultimately identified 2024 as the year with the highest activity, the supporting figures contained internal inconsistencies and could not be directly verified against the original source.

From the perspective of natural language interfaces to big data systems, this failure is significant because it affects data provenance rather than retrieval success alone. Researchers may reasonably assume that reported statistics correspond directly to the underlying database records. When agents silently substitute alternative evidence sources, correct high-level conclusions can be accompanied by inaccurate or unverifiable supporting data. This explains why Task D4 achieved relatively low performance despite many agents reaching the correct headline answer.

Example 2: Missing Contextual Qualification (Task E7)

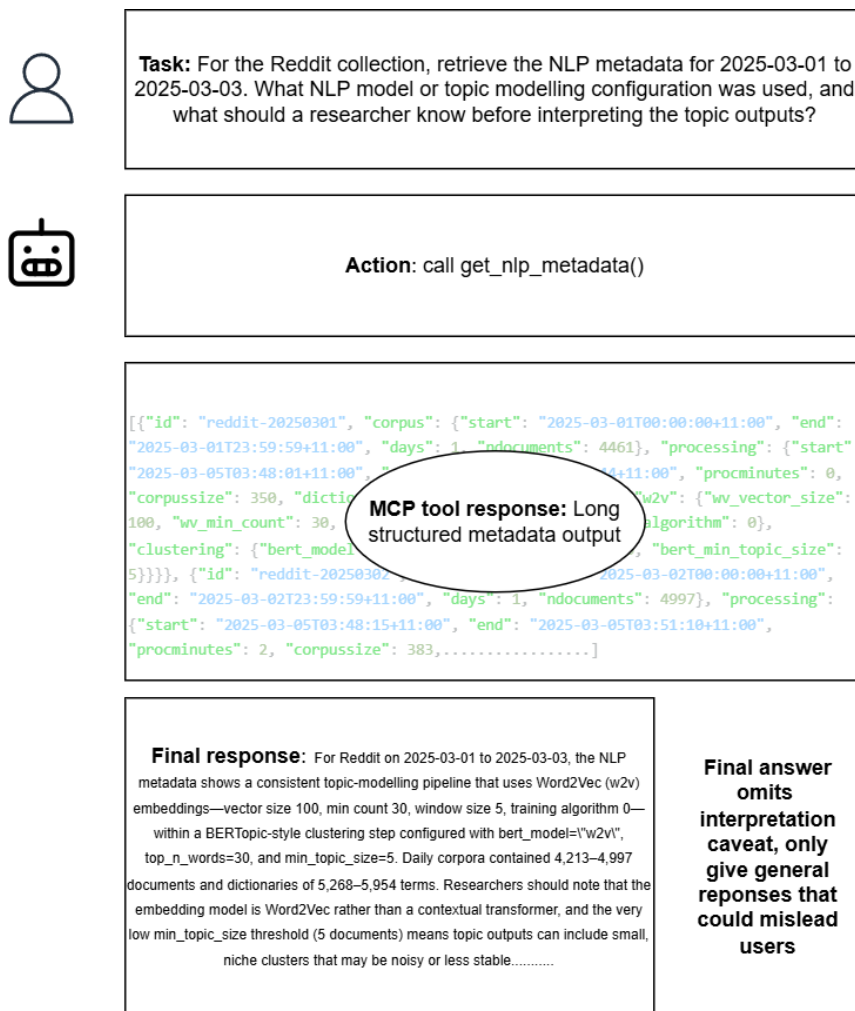


FIGURE 5.6: Task E7

Task E7 illustrates a failure mode in which the agent successfully retrieved the required metadata but failed to provide the contextual information necessary for correct interpretation. The query asked not only for the NLP and topic-modelling configuration used in

the dataset, but also for information that researchers should consider when interpreting the resulting topic outputs.

The agent correctly invoked the ‘get_nlp_metadata’ tool with appropriate parameters and retrieved the relevant daily metadata records. Its response accurately described several characteristics of the NLP processing pipeline and included general methodological cautions regarding topic modelling. However, it omitted a critical task-specific qualification: the processed corpus sizes were substantially smaller than the corresponding raw document counts.

This omission is important because it affects the validity of downstream interpretation. Without this contextual information, users may incorrectly assume that the generated topics represent the complete daily Reddit corpus rather than a filtered subset of documents. The failure therefore did not arise from incorrect tool selection or parameter usage, but from incomplete synthesis of the retrieved evidence.

This case highlights a broader limitation observed across several tasks in the benchmark. Modern MCP-enabled agents frequently demonstrated high tool-use competence, yet still failed to communicate all information required for accurate interpretation. For researcher-facing natural language interfaces, the ability to contextualise retrieved data may therefore be as important as the ability to retrieve it.

Example 3: Evidence Interpretation Error (D7)



Task: Compare Reddit activity between January and September 2024. How much did total post volume change, and did weekday/weekend posting patterns remain similar across the two months?



Action:

To get weekday pattern, call:
`aggregate_seasonality(reddit, daysofweek, Jan)`
`aggregate_seasonality(reddit, daysofweek, Sep)`

To get monthly total, call:
`aggregate_by_time(reddit, month, Jan ~Sep)`

MCP tool response:

Weekday post count

```
[{"time": "friday", "count": 22145}, {"time": "monday", ...}]
[{"time": "friday", "count": 13662}, {"time": "monday", ...}]
```

Monthly post count:

```
[{"time": "2024-01", "count": 171016}, {"time": "2024-02", ...}]
```



Reason:

I will calculate and compare raw weekday total vs raw weekend total to answer the query

Final response : Reddit post volume decreased from 171016 posts in January 2024 to 109334 posts in September 2024, a drop of 61682 posts, or about 36.07%. Weekday/weekend patterns were broadly similar because weekdays remained the majority in both months, but September had a higher weekend share and a stronger Sunday concentration than January.....

FIGURE 5.7: Task D7

Case D7 illustrates a post-retrieval evidence interpretation failure. The text judge marked this answer as partially correct, which collapsed to a non-pass under binary scoring. In this example, the agent successfully retrieved the correct monthly totals and day-of-week counts, executing tool calls that were both valid and semantically aligned with the task query. Furthermore, it accurately calculated that Reddit volume decreased from 171,016 posts in January 2024 to 109,334 posts in September 2024—a reduction of 61,682 posts.

However, the agent incorrectly concluded that weekday and weekend posting patterns were “broadly similar” simply because absolute weekday totals exceeded weekend totals in both months. This comparison is analytically flawed, as it does not account for the unequal number of weekdays versus weekend days. A more appropriate approach would normalise the count by the number of days which reveals a different conclusion: January

shows slightly higher average weekday activity, while September shows higher average weekend-day activity. Although the agent possessed the correct evidence, its application of an inappropriate comparative baseline led to a misleading conclusion for the second half of the task.

This example demonstrates how LLM agents can commit data interpretation errors even when the underlying retrieval process is flawless. In user-facing big data interfaces, this creates a risk: because the agent reports precise retrieved values, percentages, and raw counts, the output appears highly empirical and trustworthy to the user at first glance. The case therefore highlights the need to evaluate LLM agents beyond successful tool invocation and data retrieval. For analytical data access, correctness also depends on whether the agent applies suitable normalisation, comparison units, and contextual reasoning when transforming retrieved outputs into a user-facing conclusion.

5.5 Supplementary case study of Open-Ended Tasks

The primary benchmark evaluates whether MCP-enabled agents can complete analytical tasks with verifiable expected outcomes. However, one intended benefit of an LLM-based interface is its ability to support exploratory data analysis, where useful responses may extend beyond a predefined answer.

During pilot testing and benchmark development, several open-ended task formulations revealed a recurring pattern in which agents performed analytical operations beyond the minimum reference pathway. Such extensions were difficult to assess through reference-answer-based judging, because useful contextual findings or exploratory observations could fall outside a fixed expected answer. Several initial task formulations were therefore narrowed to a more confined scope. While this was necessary for the primary benchmark, it also reduced part of the exploratory character that motivates natural-language interfaces for research data systems.

The supplementary analysis revisits selected earlier, less constrained task formulations to examine whether an MCP-enabled agent can produce grounded analytical value when the task is less tightly specified. As discussed earlier in this chapter, this same behaviour

may also contribute to lower Tool Appropriateness scores, even where the final task outcome is successful. To characterise these behaviours and assess whether additional interactions produced analytical value rather than verbosity alone, a small supplementary trace analysis was conducted.

The supplementary analysis is also motivated by a broader limitation of outcome-only evaluation for exploratory data work.

5.5.1 Study Design

Three open-ended prompts, derived from bounded benchmark tasks but retaining their earlier broader formulation, were selected for qualitative trajectory analysis.

They covered different combinations of task demand profile and data-source scope: E1O as an analytical single-source task, D3O as a synthesis multi-source task, and D7O as a synthesis single-source task. These profiles were selected because the main benchmark showed lower completion or repeatability in tasks requiring analytical interpretation and synthesis, making them useful cases for examining exploratory tool-use behaviour in greater detail. The selected prompts therefore focus on cases where exploratory tool orchestration is both more likely to be useful and more difficult to evaluate using pass/fail outcome and Tool Appropriateness scoring alone. The task prompt can be found in Appendix [A.1](#).

Claude Opus 4.6 was selected for this study because it achieved the strongest overall benchmark performance and displayed exploratory behaviour in the main benchmark, particularly on analytical and synthesis tasks. It therefore provided a useful best-case probe of whether a high-performing MCP-enabled agent could add exploratory value beyond the fixed-answer tasks used in the main benchmark. To examine consistency across runs, each selected task was executed in three independent trials using the same procedure as the main benchmark.

The analysis proceeded in two stages. First, each tool-call sequence was divided into episodes. An episode was defined as a contiguous cluster of tool calls serving a shared local analytical objective. Second, each individual tool call was treated as a coding unit and assigned to one of the following categories:

- **Core retrieval (CR):** Directly retrieves evidence required to answer the main task. Without this action, the final answer would lack essential evidence. Examples include querying target collection, date range, keyword, count, metadata, or aggregation explicitly requested by the task.
- **Context expansion (CE):** Retrieves contextual information needed to interpret the core result properly. It may not answer the main question alone, but it improves qualification, limitation, or explanation. Examples include checking dataset coverage, metadata, NLP model configuration, source availability, known data gaps.
- **Exploratory retrieval (ER):** Additional calls that broaden analysis in a task-relevant way. It is not strictly required, but it can support open-ended insight generation. e.g., adding an extra analytical angle beyond the minimum answer, comparing related terms, adjacent time windows, extra collections, or checking trends.
- **Visual generation (VG):** Uses a visualisation tool to represent retrieved evidence. e.g. generating a chart of daily term frequency
- **Misdirected / Redundant (MR):** Tool calls that do not add useful task-relevant evidence. For example, repeating prior calls without new information, drifting away from the task, or using tool parameters that do not match the task's intent

The coded traces were then compared across the three tasks and repeated trials with respect to three features: (1) the sequence of episodes through which retrieval and interpretation actions unfolded; (2) the relative amount of activity devoted to core retrieval and supplementary investigation; and (3) whether supplementary calls materially informed the final answer: its output was explicitly reported, used to qualify a finding, or provided evidence for an interpretive claim or caveat in the final response.

5.5.2 Findings

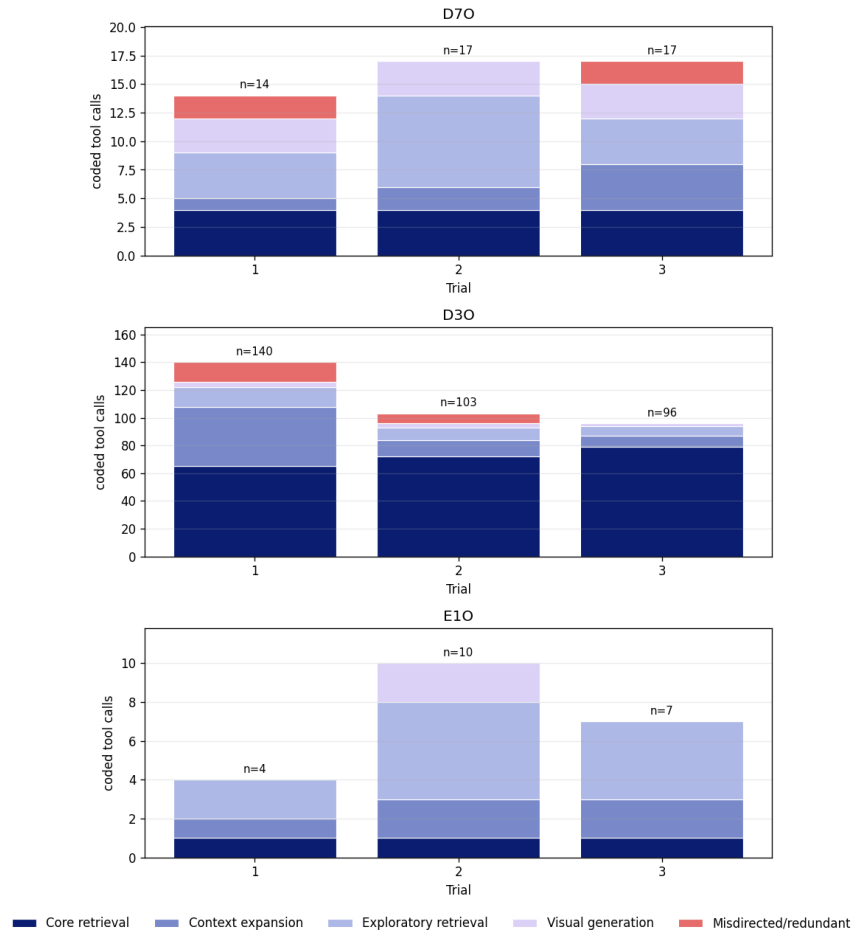


FIGURE 5.8: Distribution of tool-call codes across the three independent trials of each open-ended task. Bars show the number of individually coded tool calls; total coded calls are shown above each bar.

Figure 5.8 shows the distribution of coded tool calls across the three trials of each open-ended task. For all three open-ended tasks, the traces showed substantial tool use beyond the core retrieval (dark blue) required to answer the prompt. Across the nine traces, 177 tool calls were coded as context expansion, exploratory retrieval, or visual generation, compared with 25 coded as misdirected or redundant. This suggests that the agent’s additional tool use was generally directed towards extending, qualifying, or contextualising the initial evidence, rather than reflecting widespread erroneous behaviour.

The level of expansion differed substantially by task. D3O involved the largest number of coded calls in all three trials, consistent with the broader analytical burden of comparing multiple heterogeneous collections over a year-long period. D7O showed a more moderate but still substantial amount of non-core activity, despite being a single-source temporal

comparison. E1O involved fewer calls overall, but still showed divergence in the way the agent investigated and interpreted the open-ended question. Because total call volume varied substantially across tasks, absolute call counts should be interpreted as indicators of within-task expansion rather than directly comparable measures of agent efficiency across tasks.

Across all three tasks, variation was more evident in the supporting evidence, contextualisation, and interpretive framing than in the central empirical findings. Repeated trials generally converged on similar high-level conclusions while differing in the analytical pathways used to justify them.

Task-level trace patterns

D7O: Stable temporal comparison with variable exploratory branches.

In D7O, all three repeated trials produced a highly consistent core account of the January–September comparison. The initial episodes were particularly similar: each retrieved the relevant monthly and daily totals, identified a September data-harvesting gap, and two trials noted an anomalously low day in January. However, the trials differed in how they operationalised the open-ended request for “additional features.” One extended the analysis to hourly activity patterns; another compared prominent terms and collection metadata; and a third examined the availability of NLP topic outputs. These extensions were generally relevant and, in particular, helped reveal that incomplete harvesting or downstream processing could affect interpretation of the observed difference. At the same time, the outputs illustrate a limitation of exploratory tool use: additional analysis introduced minor and inconsistent arithmetic errors. In two trials, the September clean-period average was miscalculated. Overall, in these D7O trials, exploratory branches identified anomalies and processing limitations that were absent from the minimum retrieval pathway. The specific exploratory branch pursued, however, varied across runs and also introduced opportunities for arithmetic error.

D3O: Extensive cross-collection exploration

D3O showed the heaviest overall tool use, reflecting the higher analytical burden of cross-collection synthesis. Despite this larger call volume, explicitly misdirected or redundant activity remained limited; most additional calls were devoted to context expansion and

exploratory retrieval. Core retrieval in D3O was primarily conducted through repeated `text_search` calls, which established the relative volume of “bushfire” activity across collections.

Across the three D3O trials, the agent consistently converged on the same core empirical finding: GDELT dominated 2024 “bushfire” mentions. Each trial then moved beyond this minimum comparison to identify contextual features that could affect interpretation, including Bluesky’s late-year increase, the absence of Twitter coverage, and Reddit’s comparatively stable and less event-driven pattern. However, the focus of this exploratory activity varied considerably across trials. Trial #1 focused most strongly on data validity and semantic interpretation. It examined GDELT’s language composition and used term co-occurrence evidence to assess whether the retrieved records still referred to Australian bushfires. Trial #2 adopted a broader descriptive approach, examining collection-wide weekday publishing patterns and providing a more detailed monthly activity breakdown. Trial #3 offered the clearest cross-collection comparison, placing greater emphasis on differences in collection size. The three trials therefore diverged markedly in how they operationalised the open-ended request for “researcher interpretation,” despite converging on the same central comparison.

E1O: Interpretive variation around “characteristic” terms

E1O involved fewer tool calls overall, but still displayed clear interpretive branching. All three trials broadly agreed on the most prominent terms used during the first week of January 2025. However, they differed in how they addressed the more open-ended question of which terms were especially characteristic of the period. One trial relied primarily on topic-cluster evidence; another supplemented this with more fine-grained daily inspection and visualisation, resulting in the highest tool-call count; and the third focused more narrowly on a particular day and its associated political discourse. The variation therefore lay less in retrieval of the underlying evidence than in how the agent chose to define and substantiate “characteristic” in its final synthesis. This indicates that, even in a relatively bounded single-collection task, the open-ended wording was associated with materially different analytical pathways across repeated trials, even though the underlying factual target remained similar.

Furthermore, the contrast between E1 and E1O shows that relatively small changes in prompt wording can alter the perceived scope of tool use. The benchmark version of E1

asked for "high-frequency terms" and a "summary of themes" inferred from those terms. Across three trials, the same model, Opus 4.6, consistently relied on `get_all_terms` and `analyze_terms_in_collection`. E1O retained the same collection and time period, but asked which terms were "especially characteristic" of the period. The evidential basis for "characteristic" was less clearly specified, appearing to invite temporal, topical, and visual corroboration. Across the three E1O trials, the agent consequently used additional daily-count, topic, NLP-term, and chart-generation tools. This comparison is consistent with the interpretation that open-ended wording broadens the agent's perceived analytical remit, even where the underlying factual target remains largely unchanged.

Behaviour pattern

Together, the three cases show a recognisable pattern of open-ended tool use by the selected MCP-enabled agent.

Stable core findings, divergent interpretive pathways

Across D7O, D3O, and E1O, the agent generally retrieved the central evidence required by the prompt and reached broadly consistent high-level conclusions across repeated trials. The main variation emerged after this minimally sufficient evidence had been obtained. Different trials pursued different investigative pathways and consequently foregrounded different contextual features, caveats, and supporting evidence. For open-ended tasks, the agent therefore appeared to construct an interpretive frame around the retrieved evidence rather than follow a fixed retrieval sequence.

Exploratory retrieval introduced additional contextual information

Additional retrieval often produced information that materially improved interpretation of the core result. In D7O, exploratory branches identified data-harvesting gaps, anomalous daily values, and possible limitations in downstream NLP outputs. In D3O, they surfaced collection-coverage differences and platform-specific activity patterns. In E1O, additional daily and topical inspection helped distinguish between common terms and terms specific to the selected date. Across these three case studies, open-ended prompts were associated with additional contextual investigation, particularly where the wording requested interpretation, additional features, or period-specific characteristics. Several

of these branches supplied useful qualifications, although the study does not estimate how often this behaviour would be beneficial across a wider task set or model population.

Exploratory depth had variable marginal value

The value of additional exploration was uneven across trials. Some branches directly resolved an uncertainty or supplied an important qualification, while others extended the analysis beyond the likely practical scope of the prompt. For example, one D3O trial devoted a substantial sequence of calls to GDELT language composition and co-occurrence patterns. This produced a potentially important finding about how apparent “bushfire” volume should be interpreted, but also increased latency and analytical complexity for a question that could have been answered at a broader comparative level. This behaviour is best characterised as variable scope discipline or exploratory over-extension. Explicitly misdirected or redundant calls remained relatively uncommon in the coded traces.

Additional analysis also created new opportunities for error

Broader investigation did not uniformly improve answer quality. In D7O, two trials introduced minor arithmetic inaccuracies when calculating the September clean-period average, despite reaching the same overall conclusion about the January–September difference. This observation is consistent with the main-benchmark error analysis, which identified quantitative aggregation errors in text-answer tasks requiring substantial arithmetic reasoning. The cases illustrate a trade-off: broader exploration can improve coverage of caveats and contextual features, while additional analytical branches may also create more points at which calculation, interpretation, or synthesis errors can occur.

Available tool affordances appeared to influence exploration

Across the nine traces, the agent sometimes invoked NLP, sentiment, temporal, or visualisation tools that were not required for a minimally sufficient answer. For example, 7 of 9 trials used `generate_chart` to present numerical findings, despite none of the prompts explicitly requesting a visual output. These observations show that the available tool set formed part of the practical space of exploration available to the agent. However, the present design does not isolate whether tool availability itself caused these choices, because tool descriptions, prompt interpretation, and stochastic variation may also have contributed.

Chapter 6

Discussion

6.1 RQ1: Designing an Execution-Grounded Benchmark for Live Research Data Systems

RQ1 concerns how performance claims can remain interpretable when agents operate over a live research system whose data, service behaviour, and analytical outputs may change over time. This is therefore more than a question of constructing a task set. Recent work on agentic benchmarking distinguishes between *task validity*, where successful completion genuinely reflects the target capability, and *outcome validity*, where the evaluation procedure correctly identifies successful completion [40]. This distinction is particularly important in the present setting. A failed trial may reflect deficient reasoning, inappropriate tool use, or an inability to interpret returned data. It may also reflect unavailable data, altered backend behaviour, a reference pathway that no longer produces the expected output, or limitations in the evaluation procedure itself. The central design challenge was therefore to make these sources of variation as distinguishable as possible, so that benchmark outcomes could support interpretable claims about agent capability.

Constrained realism as a design response: To explicitly isolate reasoning and tool-use capabilities, the model-facing tool layer was held constant throughout the evaluation: The available tool descriptions, parameter schemas, and expected interaction format were fixed. This helped reduce the source of uncontrolled variation. Concurrently, to preserve ecological validity, the backend remained completely live. By exposing

agents to authentic operational conditions—such as incomplete data coverage, evolving historical records, and ongoing processing changes—the evaluation retains the genuine friction of a researcher-facing deployment. Finally, the benchmark retained only tasks for which a stable, evidence-grounded reference state could be established. This limited the openness of the task set, but it ensured that a failed trial could be assessed against a known executable basis instead of an unclear situation or unverified backend condition. Together, these choices preserved live-system realism while providing a clear basis for interpreting benchmark results.

Ground truth as an auditable reference state: The live nature of AIO also changes the status of ground truth. In contrast to frozen benchmarks, where expected answers can be treated as fixed labels, the underlying data and analytical outputs in AIO may change over time through new harvesting, historical backfill, or revised processing pipelines. Consequently, ground truth becomes an auditable reference state tied to a particular system condition at the time of execution rather than a permanently fixed answer. By retaining the reference calls and re-executing them against the live system, the benchmark can distinguish more credibly among legitimate changes in the data environment, reference pathways that have become invalid, and genuine agent errors. This distinction is essential because each case has different implications for performance interpretation: a changed backend state should not automatically be counted as a model failure, while an invalid reference pathway should not be silently replaced with an alternative route merely to preserve the task. For live research infrastructures, benchmark validity therefore depends not only on storing an expected answer, but also on retaining the executable pathway, system condition, and audit record through which that answer was established.

Beyond minimum-path evaluation. A further design implication concerns how benchmarks treat deviations from a minimum reference pathway. Reference plans remain necessary for establishing that a task has an executable evidential basis and for identifying calls that are required for a minimally sufficient answer. However, they should not be interpreted as complete descriptions of all analytically valuable behaviour. In researcher-facing tasks, additional calls may be used to verify coverage, inspect limitations, test alternative interpretations, or obtain contextual information that changes how the core result should be communicated. Benchmark evaluation should therefore

distinguish between at least three levels of performance: outcome correctness, evidential process quality, and optional exploratory value. Outcome correctness concerns whether the final answer is supported by the required evidence. Evidential process quality concerns whether the agent preserves provenance, relevant limitations, and appropriate analytical conditions while constructing that answer. Optional exploration concerns whether additional activity contributes useful contextual insight beyond the minimally sufficient pathway. The first two dimensions can often be operationalised through reference plans and trace-to-answer inspection. The third is harder to score automatically and may require episode-level annotation or expert review. The empirical consequences of this distinction are examined further in RQ3.

6.2 RQ2: Task Performance and Repeated-Run Reliability

Overall, the results suggest that contemporary MCP agents are generally capable of retrieving information successfully within a bounded research environment. Performance differences emerged less from evidence access than from evidence interpretation and integration, and high average task completion did not always translate into reliable performance across repeated executions.

Several models achieved high overall task completion rates, with Claude Opus 4.6 performing best and Claude Sonnet 4.6 and DeepSeek V4 Pro forming a second tier. GPT-5.5, Kimi K2.6, and Qwen 3.6 Plus also completed the majority of benchmark tasks. These results suggest that contemporary proprietary and open-weight models can successfully conduct many forms of structured social-data retrieval and analysis when operating within a bounded MCP environment. Within the evaluated configurations, the primary distinction was therefore not between proprietary and open-weight models, but between individual agent configurations and task demands. Llama 4 was a notable exception, exhibiting near-complete failure across most task categories.

The task-demand taxonomy provided a more informative descriptive lens for the observed performance differences than the user-facing research categories. Categories such as Discovery, Dataset Overview, Comparative Analysis, and Exploratory NLP Analysis

often overlapped in the underlying operations required to produce an answer. Consequently, category-level differences should be interpreted cautiously, as more challenging categories frequently involve broader source coverage, additional reasoning steps, or greater integration demands.

The task-demand taxonomy provides a clearer view of where models succeeded and struggled. Across the evaluated non-Llama models, Direct Retrieval and Batch Retrieval were the strongest demand profiles. Claude Opus, Claude Sonnet, GPT-5.5, Kimi, and Qwen all achieved perfect TCR on Batch Retrieval tasks, while Direct Retrieval scores ranged from 0.905 to 1.000. These results suggest that repeated tool use was not, by itself, a major source of difficulty. Even when tasks required more tool calls and larger working contexts, the procedure was typically explicit and repetitive, involving the application of the same retrieval pattern across dates, terms, or collections before aggregating the results. A clearer separation emerged once tasks required interpretation rather than retrieval alone. Analytic Interpretation and Synthesis tasks produced substantially greater variation between models. While Claude Opus remained close to ceiling performance across all demand profiles, GPT-5.5, Kimi, and Qwen showed lower performance on Analytic Interpretation tasks, and several models also declined on Synthesis tasks. These demands required agents not only to retrieve evidence but also to evaluate, integrate, and communicate findings appropriately, creating additional opportunities for analytical error.

The relatively high completion rates on these retrieval tasks should be considered in the context of the benchmark’s design and evaluation focus. Recent MCP benchmarks such as LiveMCPBench evaluate agents across large and heterogeneous tool ecosystems, requiring them to discover relevant capabilities across 70 MCP servers and more than 500 tools [42]. LiveMCP-101 similarly introduces distractor tools and broader tool-selection demands [49]. These benchmarks assess an agent’s ability to navigate unfamiliar environments, select appropriate tools, and coordinate actions across diverse resources.

This study addresses a different but complementary aspect of MCP-agent performance. Because agents operated within a single research-oriented infrastructure consisting of a frozen set of semantically related tools, this design isolates a different stage of the workflow. It removes uncertainty about tool availability. Viewed in this context, the strong task completion rates indicate that contemporary agents can effectively execute

structured retrieval. However, substantial variation remained on tasks requiring interpretation, comparison, and synthesis. The results therefore indicate that, even when tool selection and discovery are relatively constrained, higher-level analytical reasoning continues to differentiate agent performance.

Repeated-run evaluation further qualifies this positive result. All non-Llama models achieved pass@3 values of 0.971 or 1.000, meaning that nearly every benchmark task could be completed correctly at least once across three trials. Their all-three success rates were lower, ranging from 0.771 to 0.914. Claude Opus was the most repeatable configuration, completing 32 of 35 tasks successfully in all three trials, but even this model showed mixed outcomes on three tasks. Thus, high pass@3 indicates that many tasks fall within the models’ capability envelope, whereas lower pass^3 indicates that this capability is not always exposed reliably in a single interaction.

This distinction is particularly important for researcher-facing natural-language interfaces. In conventional benchmark settings, at-least-once success may suggest that a task is solvable through repeated sampling, reranking, or external selection. A researcher using an AIO interface, however, will typically issue a query only once and rely directly on the returned answer. They are unlikely to submit the same request multiple times and independently determine which response is correct. This leads to an important practical implication: although repeated sampling may improve the likelihood of obtaining a correct response, it does not by itself ensure reliability. Voting-based architectures may help aggregate candidate outputs, but they still cannot guarantee a dependable answer. The system therefore requires a principled mechanism to select, verify, or qualify competing outputs before presenting results to users.

This pattern aligns with the motivation of tau-bench, which distinguishes between the ability to solve a task at least once and the ability to solve it consistently across repeated executions [38]. The absolute results in this study are stronger than those reported in many multi-turn transactional settings, but the underlying reliability pattern is similar. tau-bench evaluates customer-service tasks involving simulated users, policy constraints, stateful environments, and action-taking tools. In contrast, this work consists of bounded, read-only analytical tasks. The persistence of a pass@3 – pass^3 gap despite the removal of many sources of environmental complexity suggests that reliability problems cannot be attributed solely to multi-turn interaction, policy constraints,

or stateful environments. Even in bounded analytical workflows, variability can emerge from how agents interpret tool outputs, maintain intermediate reasoning, and construct final answers.

The DraCor MCP case study provides a related domain-level comparison. Its evaluation similarly found that agents could often select appropriate tools and complete research-corpus tasks, while outcomes varied across repetitions [32]. Together with the present results, this suggests that tool-access competence can coexist with instability in research-oriented analytical workflows. The next section examines this issue at trace level, considering whether successful tool invocation, valid parameters, and appropriate pathways are sufficient to explain end-to-end analytical reliability.

6.3 RQ3: Tool Use, Failure Patterns, and Exploratory Behaviour

The trace-level findings show why the distinction between minimum-path compliance and evidential process quality matters in practice. Across several failed trials, agents generated schema-valid and broadly task-relevant calls, yet produced misleading final answers because they omitted contextual limitations, altered the evidential basis through fallback retrieval, applied unsuitable comparative baselines, or failed to integrate relevant retrieved information into the final synthesis. These patterns indicate that end-to-end analytical reliability depends on more than whether an agent reaches the required tools.

6.3.1 Process-Level diagnostics are necessary but insufficient

Process-level diagnostics were necessary for interpreting agent behaviour, but they explained only a limited share of end-to-end analytical reliability. Under the standardised, schema-rich MCP interface, Parameter Schema Validity and Constraint Compliance were near ceiling among the stronger configurations. Executable calls and adherence to documented operating limits were therefore common, leaving substantial variation in Task Completion Rate (TCR) unexplained. The remaining performance gap centred on higher-level reasoning: identifying sufficient evidence, applying appropriate analytical assumptions, and carrying relevant limitations into the final response.

Consistent with MCP-Bench’s framing [41], these results shift the primary evaluative emphasis from formal call generation towards trajectory-level reasoning. Among stronger models, tool execution showed limited variation, whereas performance diverged in evidence sufficiency, synthesis of complex observations, and preservation of analytical rigour in final responses.

This pattern is illustrated by the divergence between Tool Appropriateness (TA) and Task Completion Rate (TCR). TA measures alignment with a manually specified core/helper tool set, yet model-level results show that reference-pathway alignment did not reliably predict analytical success. GPT-5.5 achieved the highest TA without achieving the highest TCR, suggesting that close adherence to the expected retrieval pathway did not ensure successful evidence synthesis. DeepSeek retained high TCR despite recording the lowest TA among the stronger non-Llama models, indicating that effective task completion could occur through trajectories that received lower reference-pathway scores. Llama recorded mid-range TA despite issuing relatively few calls, while also showing premature termination, missing evidence, and output-compliance failures.

This helps identify important limitations in TA as currently operationalised. The metric groups exploratory, recovery-oriented, redundant, and genuinely irrelevant calls within a simplified weighting scheme, while leaving omissions of necessary calls and completeness of evidence retrieval largely unmeasured. TA therefore provides a useful descriptive measure of reference-pathway alignment, but it should not serve as a standalone proxy for overall tool-use quality. Evaluation of researcher-facing systems should additionally assess evidence coverage and the quality of final synthesis, since an apparently aligned trajectory can still produce an incomplete or misleading analysis.

6.3.2 Many Consequential Failures Occurred After Retrieval

A major challenge in MCP-agent evaluation concerns tool discovery: agents may need to identify relevant capabilities from large, heterogeneous, and partially distracting tool inventories. Recent benchmarks such as LiveMCPBench and LiveMCP-101 foreground this challenge through multi-server environments, large tool pools, and distractor tools. In the present study, however, the tool space was deliberately constrained. The qualitative error analysis suggests that for agents operating in a more bounded setting, the bottleneck shifts from discovery to evidence-to-claim transformation.

This evidence-to-claim transformation failed in several related ways.

- **Loss of Evidential Provenance:** Agents may change the evidential basis of an answer without preserving that change in the final claim. Where this substitution was not disclosed, a researcher could reasonably infer that the reported value reflected the requested method when it in fact depended on a different evidential basis. This represents a provenance problem that has implications for interface and tool-harness design, discussed later in this chapter.
- **Missing contextual qualification:** Agents could retrieve relevant contextual information without carrying its interpretive significance into the final answer. In several NLP-oriented tasks, agents accurately described the requested topics or term patterns while omitting limitations that altered their meaning or scope, such as noisy topic clusters, processed subsets of the underlying corpus, or the fact that returned term frequencies represented stemmed forms rather than natural-language words.
- **Disconnection between Local and Global Synthesis:** In one D7O trace, for instance, the agent explicitly identified that September contained only 29 available daily records and noted late-month collection gaps during its intermediate calculations. However, the final JSON summary still divided the monthly total by 30 days and interpreted the resulting difference as a substantive decrease in posting activity. The relevant evidence and coverage limitation were therefore available in the trace, but the constraint was not preserved when the agent generated its final analytical claim.

These cases indicate that successful retrieval alone is insufficient for reliable research-facing assistance.

In this study, *evidence-to-claim transformation* refers to the set of operations through which retrieved observations are converted into a final researcher-facing claim. The flow of information can be represented as follows:

Retrieved observation → Local evidence extraction → Analytical transformation →
Final claim

Failures often occur when information that had been correctly retrieved or locally recognised was not reliably maintained across the later stages of this sequence. The first pattern concerns an unrecorded change in evidential provenance, while the latter two reveal a more direct disconnect between local evidence extraction and global synthesis. In both cases, the final answer ceased to provide a transparent representation of the evidence and constraints available to the agent during execution.

This interpretation aligns with recent work on evidence extraction and tool-response processing. Tan and D’Souza [58] found that LLMs could perform moderately on isolated property retrieval but degraded sharply when required to preserve relational and numerical bindings across multiple pieces of evidence. Although the present study concerns live tool use rather than cross-document meta-analysis, a comparable difficulty appeared in maintaining the relationship between retrieved counts, coverage conditions, and the higher-level claim derived from them. Similarly, Kate et al. [57] identify tool-response processing as a distinct challenge for LLM agents operating over structured JSON outputs. In their benchmark, prompting models to generate executable parsing code was generally more accurate than direct answer generation for filtering and aggregation tasks. This suggests that transformations involving strict filtering, aggregation, or numerical calculation may be more reliably handled through executable components than through unconstrained narrative synthesis alone.

One plausible contributing mechanism is context-management pressure. Successful tool use frequently returned lengthy structured observations, which accumulated alongside prior outputs and intermediate calculations in the agent’s conversation history. The final synthesis stage consequently required the agent to retain, select, and reconcile a growing body of evidence rather than merely answer from one observation. Existing work on long-context and multi-step agents suggests that expanding context can degrade the effective use of relevant information even when that information has been retrieved [59, 60]. However, the present study did not manipulate context length or tool-output volume, so this explanation remains provisional.

These findings suggest that final-answer generation should be treated as a distinct analytical transformation stage rather than as a neutral presentation layer. Research-facing systems would benefit from retaining a persistent structured evidence representation between retrieval and prose synthesis, such that each summary claim remains traceable

to its underlying observations, denominators, coverage conditions, and transformation steps. Derived statistics and comparison bases should, where possible, be computed or validated outside free-form narrative generation. The final response could then draw from a structured record linking source observations, calculated values, and interpretive claims. Such a design would make inconsistencies between accurate local evidence and inaccurate global summaries more detectable before an answer is presented to a researcher.

6.3.3 Exploratory Behaviour

Recent process-oriented work on exploratory data agents provides a useful contrast for interpreting these findings. DataClaw evaluates agents in low-guidance, cross-domain real-world data settings, where relevant sources, schemas, and noise conditions are initially uncertain. In such environments, exploratory actions can produce meaningful partial progress without reliably yielding a correct final outcome [61]. The present study examines a more bounded stage of the same broader problem. The selected agent operated over a single AIO MCP server with a frozen, semantically related tool surface, and the supplementary tasks were designed such that their core evidence was generally accessible through known analytical capabilities. Across D7O, D3O, and E1O, the agent typically obtained the central evidence and reached broadly similar high-level conclusions before substantial divergence emerged across trials. Subsequent calls primarily served contextual elaboration: checking collection coverage, inspecting adjacent temporal patterns, seeking methodological context, or adding support for a richer interpretation. The central challenge was therefore less often discovery of an unknown source than deciding whether further retrieval was likely to resolve a material interpretive uncertainty.

The value of this additional activity was uneven. In several cases, exploratory retrieval provided crucial context, such as data gaps, anomalies, and limitations in NLP outputs, that significantly improved the final interpretation. Such insights are critical for preventing overconfident claims in real-world data systems. Conversely, other cases resulted in non-integrated exploration. In these instances, the agent acquired potentially relevant information but did not incorporate it into the final answer, use it to qualify a claim, or revise its analytical interpretation.

A successful topic-analysis trajectory from the main benchmark illustrates this pattern. The agent obtained the core evidence required by the prompt after an initial collection-discovery call and two `get_nlp_topics` calls, one for each target collection. It then issued six additional, unprompted calls. Two `get_topic_groupings` calls sought evidence of cross-day topic continuity; two `get_collection_summary` calls attempted to explain differences in topic size through whole-collection volume; and two `get_nlp_metadata` calls sought processing context, one of which returned an error. These actions were not uniformly irrational: topic grouping and NLP metadata could potentially improve interpretation. However, their marginal contribution was uneven. The topic-grouping evidence was not necessary to answer the prompt, the collection summaries concerned historically unaligned coverage periods and therefore did not provide a secure basis for explaining observed differences, and the successfully retrieved metadata was not incorporated into the final answer. This example shows the issue was not exploration itself — it was the absence of a reliable mechanism for determining whether an additional observation should alter, qualify, or be excluded from the final analytical claim.

This pattern connects to recent work on unnecessary tool use, which shows tool-using agents may continue to invoke tools even when the expected value of additional calls is low, thereby increasing execution cost and latency without proportionate improvement in task quality [62]. In researcher-facing systems, however, the relevant decision is more specific than whether a tool is strictly necessary for answering the prompt. An additional call may have high value when it establishes a coverage limitation, identifies a data anomaly, or tests an assumption underlying a comparison. Its value is low when it only expands the conversation history without reducing uncertainty or changing the evidential basis of the answer. The resulting design requirement is therefore scope discipline. Agents should remain able to investigate beyond a minimum retrieval pathway when interpretation warrants it, while making the intended value of each additional call explicit. Intermediate evidence ledgers, compressed tool-output summaries, or pre-call checks specifying the uncertainty to be resolved could support this behaviour. Such mechanisms could retain the contextual benefits of exploration while reducing unnecessary latency, context accumulation, and opportunities for faulty synthesis.

Collectively, these failure patterns suggest that the main weakness in the evaluated agent architecture extended beyond tool selection and execution. The system lacked a reliable mechanism to carry analytical constraints—such as retrieval pathways and anomalous

values—from retrieved observations into the final claim. Although agents were often able to retrieve these details correctly, they did not consistently preserve them during synthesis. To support practical real-world use, systems may benefit from a traceable transformation process that connects retrieved evidence to researcher-facing claims and makes intermediate reasoning explicit. This observation motivates the architectural and interaction-design implications discussed below.

6.4 Implications for Research-Agent Design and Deployment

6.4.1 One-shot capability is insufficient for researcher-facing deployment

The findings support a staged view of deployment. MCP-enabled agents appear most suitable for bounded retrieval, metadata inspection, and regular aggregation tasks, where the requested evidence, transformation rules, and expected output are relatively explicit. Their use in comparative, NLP-oriented, and interpretive tasks requires stronger safeguards, because these tasks depend on the preservation of contextual qualifications and analytical assumptions beyond the initial retrieval step.

The observed difference between pass@3 and pass^3 highlights an important consideration for researcher-facing systems. Success across multiple attempts does not necessarily translate into reliable performance within a single interaction. In practice, researchers typically depend on one returned answer rather than generating and evaluating several alternatives. This suggests an opportunity for interface and system design: a research agent may benefit from exposing uncertainty, verification status, and links between retrieved evidence and generated claims. Such mechanisms could help users interpret outputs more appropriately and support more dependable use in research settings.

The exploratory analysis also suggests that researcher-facing systems may benefit from an explicit scope-setting stage. Before execution, an interface could distinguish between a concise descriptive answer, a broader exploratory analysis, and a targeted investigation of identified uncertainties. Such a distinction would allow users to make the trade-off between speed, cost, analytical breadth, and evidential qualification visible. It would

also reduce the risk that agents either terminate before inspecting material context or extend tool use well beyond the practical needs of the query.

6.4.2 From prompt instructions to enforceable harness controls

Several observed failure patterns should be addressed at the harness level. In the study configuration, the shared system prompt instructed agents to handle unavailable results cautiously and return structured outputs, yet these behaviours still occurred. These findings suggest that clearer prompt wording alone is unlikely to resolve the problem. Policy-critical distinctions need to be represented as executable state and enforced by the client harness at the points where retrieval is transformed into a final answer.

First, tool failure and fallback retrieval should be represented as explicit state transitions. When a requested tool call returns an error, unavailable result, or incomplete response, the client should retain that outcome as part of the task state. When a fallback route has been used, the interface should require disclosure that the evidential basis has changed. This would prevent an agent from presenting a reconstructed approximation as though it were directly equivalent to the originally requested statistic.

Second, missingness should remain a typed analytical condition throughout the workflow. The client-side adaptor could normalise retrieved results into structured fields such as expected temporal coverage, observed dates, missing dates, tool-error status, and collection-coverage status. Calculations should then be derived from this coverage-aware state instead of values reconstructed by the language model from conversational history.

Third, structured output compliance should be treated as a distinct operational requirement. The benchmark showed that substantively relevant tool use can still yield an unusable response when the final output cannot be parsed or lacks required fields. Schema validation, repair procedures, and explicit interface-level error messages should therefore sit outside the generative model. This division reduces the extent to which deployment reliability depends on the model following formatting instructions perfectly in every run.

6.4.3 Research agents require a structured evidence state beyond conversational history

The results point to a broader architectural implication. In the evaluated system, retrieved tool outputs were returned to the model through the conversation history, from which the model later generated a final answer. This design supports flexible synthesis, but it provides no explicit guarantee that the constraints attached to retrieved evidence will remain available when a final claim is written. The observed errors in provenance preservation, coverage handling, aggregation, and comparative interpretation are consistent with this limitation.

Research-oriented MCP clients should therefore maintain a structured evidence state separate from the conversational context. Such an evidence ledger could record, for each material observation, the source tool and retrieval pathway, observed value, temporal coverage, missingness or anomalies, denominator, derived statistic, and any caveat required for interpretation. Final claims could then be generated with access to this structured state and checked against it before presentation.

This design also suggests a clearer division of responsibility between executable components and generative synthesis. Transformations involving strict arithmetic should preferably be implemented through deterministic computation or specialised analytical tools, whether on the MCP server or in the client harness. The language model can then focus on explaining and contextualising verified results. A secondary validation step, implemented through programmatic checks or a separate verification component, could test whether numerical claims, source references, and required qualifications in the final answer remain consistent with the structured evidence state.

6.5 Limitations and future work

This study provides an execution-grounded evaluation of MCP-enabled LLM agents operating over a live big data system. Its findings should nevertheless be interpreted within several methodological and practical limitations, which we synthesise into three core areas: benchmark construction, evaluation design, and operational variance.

Benchmark Scope and Single-Annotator Bias First, the benchmark comprises 35 single-turn tasks. While sufficient to identify initial behavioural patterns and trace-level execution strategies, this sample size cannot support robust statistical claims regarding fine-grained differences between models, task categories, or individual tools. The distribution of task-demand profiles inherently reflects available AIO capabilities rather than a fully balanced factorial design. Furthermore, the dataset—including reference execution plans, ground-truth records, and the qualitative error taxonomy—was constructed by a single researcher. Although simple implementation errors were mitigated through dashboard cross-checks, repeated MCP executions, and manual audits, the benchmark inevitably reflects one researcher’s interpretation of plausible workflows. Consequently, reference-plan-based metrics may overly favor anticipated analytical strategies.

The evaluation is also restricted to single-turn requests, bypassing the multi-step conversational interactions—such as clarifying ambiguous goals, revising constraints, or progressively challenging intermediate results—that characterise practical research. Future work must expand to larger, multi-turn datasets developed via inter-annotator agreement analysis, co-design workshops with social-data researchers, and permitted historical query logs to ensure prompts reflect genuine analytical needs rather than only the operations that the toolset makes easy to formalize.

Evaluation Framework Rigidity and Metric Tensions Second, the evaluation is conditional on a fixed agent architecture and a single judge model. All models received uniform tool snapshots, basic execution loops, step limits, and structured JSON output instructions. While this standardisation improves cross-model comparability, it does not capture performance variations under alternative frameworks, dynamic tool discovery, or model-specific prompting strategies. The structured JSON requirement, while vital for automated evaluation, may have inadvertently increased response-protocol burden or encouraged answers tailored to the evaluator rather than a natural, user-facing interaction. Similarly, our expected-output-conditioned prompting introduces format conditioning. We mitigated the risk of direct answer leakage by restricting prompts to schema requirements without revealing gold values, but complex analytical responses remain difficult to classify consistently under a strict binary evaluation threshold.

Furthermore, evaluating textual responses introduces vulnerabilities associated with the LLM-as-a-judge paradigm. By prompting a single model to score agent outputs against a ground truth without human-calibrated alignment strategies, the evaluation risks several known biases. First, relying on a single evaluation pass leaves the scoring vulnerable to latent model variance; while setting the temperature to 0.0 improves reproducibility, API-hosted frontier models cannot guarantee strict determinism. Second, our asymmetric manual validation strategy—inspecting all textual failures but only a random sample of passes—provides high confidence in identifying false negatives but risks overlooking false positives where the judge may have been overly lenient.

Moreover, our evaluation metrics highlight a tension between strict task adherence and analytical exploration. The "Tool Appropriateness" metric, grounded in reference-plan relevance, effectively evaluates bounded tasks but struggles to adequately reward exploratory retrieval. An agent that executes an additional, unprompted metadata query or contextual comparison might materially improve the final insight but incur a penalty for deviating from the core toolset. Future iterations require multiple independent judges, ablation studies on unconstrained natural-language conditions, and new evaluation criteria that separately assess evidential relevance, novelty, and insight quality alongside binary task completion.

Operational Variance and the Researcher Experience Finally, conducting evaluations in a live, remote AIO environment introduces external stochasticity. Variations such as remote MCP-service availability, underlying data updates, temporary tool failures, and provider rate limits inevitably affect observed outcomes. Because trials affected by identifiable infrastructure failures were rerun under the same configuration, our reported reliability measures reflect performance under an "operationally available" service configuration rather than the full user-facing reliability of the deployed system. Additionally, running only three trials per task highlights repeated-run variability but is insufficient for establishing robust variance estimates, uncertainty intervals, or trace-consistency measures.

Crucially, this study evaluates the technical and operational execution of a researcher-facing interface rather than the user experience itself. We provide no empirical evidence regarding researcher trust, perceived usability, accessibility, or adoption relative to the existing AIO dashboard. Future longitudinal studies should therefore combine expanded

benchmark trials with dedicated human-computer interaction (HCI) research, evaluating whether provenance-aware agent interfaces actively assist users in formulating questions, interpreting limitations, and completing complex analytical workflows more effectively than traditional platforms.

Chapter 7

Conclusion

This thesis evaluated MCP-enabled large language model agents as natural-language interfaces to the Australian Internet Observatory, a live and heterogeneous social-data research system. It developed an execution-grounded benchmark of 35 bounded researcher-oriented tasks, combining reference tool plans, structured expected outcomes, live-system auditing, repeated trials, process-level diagnostics, and qualitative trace analysis.

The findings show that contemporary agents can complete many structured retrieval, metadata, and regular aggregation tasks when operating over a fixed, semantically related MCP tool environment. However, high rates of schema-valid and operationally compliant tool calls did not consistently produce reliable research-facing answers. Performance was less stable for tasks requiring interpretation of analytical outputs, integration across multiple results, contextual qualification, and final-answer construction. Repeated-run results further showed that the ability to solve a task at least once did not guarantee dependable performance in a single researcher interaction.

The qualitative analyses indicate that several consequential failures occurred after relevant evidence had already been retrieved. Agents could omit material limitations, apply unsuitable comparison bases, miscalculate derived values, treat missingness as zero, or silently substitute an alternative retrieval pathway without preserving the changed evidential basis in the final answer. The supplementary open-ended analysis also showed that additional exploration could uncover useful contextual information, while increasing latency, context burden, and opportunities for faulty synthesis.

These results suggest that reliable researcher-facing MCP agents require safeguards beyond prompt instructions and tool-schema validation. Future systems should preserve provenance, missingness, coverage conditions, and derived calculations in a structured evidence state separate from conversational history. Deterministic or externally validated components should support arithmetic and other strict transformations, while final generated claims should remain traceable to their underlying observations and qualifications.

The study is limited by its single-system setting, 35-task benchmark, single-annotator construction process, three trials per task, and evaluation within one common agent architecture. It therefore does not establish general performance across all MCP ecosystems or real researcher workflows. Nevertheless, it provides an empirical basis for evaluating a critical stage of MCP-enabled analytical assistance: the transformation of live tool outputs into accurate, interpretable, and evidence-grounded research-facing answers.

Data and Code Availability

The source code, benchmark materials, experimental configuration files, and supplementary results supporting this thesis are available in the project repository:

<https://github.com/yanrx030/mcp-aio-agent-benchmark>

The repository includes the agent harness, benchmark task set, agent and evaluator prompts, live-system audit materials, scoring scripts, and extended result tables. Access to materials that depend on institutional systems, protected endpoints, or restricted datasets may be subject to the relevant access conditions and cannot be redistributed publicly.

Appendix A

Benchmark Task Set

This appendix records the benchmark task inventory used in the main experiment. It preserves the task prompts and the task-level labels required to interpret the reported results. For readability, full reference-call arguments, audit payloads, and time-sensitive expected outputs are retained in the project repository

TABLE A.1: Full benchmark task inventory.

ID	Benchmark task	Scope	Answer	Demand	Core tool(s)	Ref. steps
<i>Category A: Collection and coverage inspection</i>						
A1	how many posts does twitter collection have?	Single	Scalar	DR	get_collection.summary	1
A2	Compare the harvesting date span of all available collections. Which collection has the longest harvesting period, and which has the largest post count?	Multi	Text	BR	get_collections, get_collection.summary	8
A3	Compare Reddit, Mastodon, and Twitter by total post count and harvesting date range. Summarise which collection is largest and which has the most recent data.	Multi	Text	BR	get_collections, get_collection.summary	3
A4	List all available collections and their startdates.	Multi	Text	BR	get_collections, get_collection.summary	8
A5	Which available collections have harvesting data extending into 2026, and which collections appear to have stopped earlier?	Multi	Text	BR	get_collections, get_collection.summary	8
A6	Rank all available collections by total post count from largest to smallest.	Multi	Text	BR	get_collections, get_collection.summary	8
A7	If a researcher wants to study long-term online activity over the widest available date range, which collection would be the best starting point based on the collection summaries, and why?	Multi	Text	BR	get_collections, get_collection.summary	8
<i>Category B: Constrained retrieval</i>						
B1	Count the number of Bluesky posts from February 2024 that mention either bulk billing or Medicare.	Single	Scalar	DR	text_search	1
B2	How many Bluesky posts in 2024 mentioned "cost of living" and did not mention mortgage?	Single	Scalar	DR	text_search	1
B3	How many Twitter posts used the hashtag #auspol during first quarter 2023?	Single	Scalar	DR	text_search	1
B4	How many English-language Mastodon posts mention unimelb in 2025?	Single	Scalar	DR	text_search	1
B5	During January 2024, how many Reddit posts were recorded in total?	Single	Scalar	DR	aggregate_by_time	1
B6	For Bluesky from 2025-02-01 to 2025-02-03, retrieve the topic clusters and report the size of the largest cluster.	Single	Scalar	DR	get_nlp.topics	1
B7	In 2020, which available collection had the highest number of posts or records mentioning the keyword inflation, give me the count?	Multi	Scalar	SY	text_search, get_collections	8
<i>Category C: Temporal patterns and visualisation</i>						
C1	Find out when the Twitter collection started, and then show the daily activity for its first 10 days of data	Single	Text	SY	get_collection.summary, aggregate_by_time	2
C2	Count the number of #melbourne posts on Mastodon for January 2024, grouped by week. Use a strict 7-day 'basket' logic starting on Jan 1st, so that the reporting periods begin on 01-01, 01-08, 01-15, 01-22, and 01-29	Single	Time series	BR	text_search	5

Continued on next page

Table A.1 continued

ID	Benchmark task	Scope	Answer	Demand	Core tool(s)	Ref. steps
C3	For the Reddit dataset, was posting activity generally higher on weekends or week-days during 2023?	Single	Text	AI	aggregate_seasonality	1
C4	I'm interested in Mastodon user behaviour; does post volume typically peak at certain hours of the day over Feb 2024?	Single	Text	AI	aggregate_seasonality	1
C5	Can you draw a bar chart on the monthly post counts for the Reddit collection throughout the entire year of 2025?	Single	Chart	SY	aggregate_by_time,generate_chart	2
C6	How did the post count of the Youtube collection fluctuate on a monthly basis during 2024? draw a line graph	Single	Chart	SY	aggregate_by_time,generate_chart	2
C7	Draw a line chart showing the daily sentiment trend of the Mastodon collection during May 2024.	Single	Chart	SY	aggregate_by_time,generate_chart	2
<i>Category D: Cross-collection comparison and synthesis</i>						
D1	for the first five days of 2025, show and compare the daily volume of posts mentioning 'new year' across reddit and Bluesky collection.	Multi	Time series	SY	text_search	10
D2	Compare the monthly post volume of Reddit and Mastodon during 2024. Which platform had the higher total volume, and did their monthly trends follow a similar pattern?	Multi	Text	SY	aggregate_by_time	2
D3	which collection had the highest number of posts mentioning bushfire in 2024? What insights do you have about this result?	Multi	Text	SY	get_collections,text_search	8
D4	Comparing the daily occurrences of the terms "new," "year," and "australia" on Reddit during the first week of January in 2022, 2023, and 2024, which year had the highest activity level for these words?	Single	Text	SY	get_term_daily_counts	3
D5	For the Mastodon dataset, was posting activity generally higher on Sundays than on Mondays from the start of the Mastodon collection through 2024?	Single	Text	SY	aggregate_seasonality	5
D6	Compare the daily sentiment trend of Mastodon and gdel during January 2025. Did the two platforms follow a similar daily pattern?	Multi	Text	SY	aggregate_by_time	2
D7	Compare Reddit activity between January and September 2024. How much did total post volume change, and did weekday/weekend posting patterns remain similar across the two months?	Single	Text	SY	aggregate_by_time,aggregate_seasonality	3
<i>Category E: NLP-derived output interpretation</i>						
E1	Over the first week in Jan 2025, in the Bluesky collection, identify the highest-frequency terms. Summarise the main themes that can be inferred from these returned terms.	Single	Text	AI	get_all_terms,analyze_terms_in_collection	1
E2	For Bluesky, during the first week of Feb 2025, what topic clusters are detected, and which cluster appears to be the largest or most prominent?	Single	Text	AI	get_nlp_topics	1
E3	For reddit collection, can you show me how discussion themes are grouped together from 24 Dec 2023 to 25 Dec 2023, and point out the strongest topic groupings?	Single	Text	AI	get_topic_groupings	1

Continued on next page

Table A.1 continued

ID	Benchmark task	Scope	Answer	Demand	Core tool(s)	Ref. steps
E4	For the reddit collection between 2022-04-20 and 2022-04-25, identify the top stemmed terms and summarise the main themes suggested by those terms.	Single	Text	AI	<code>get_all_terms,</code> <code>analyze_terms_in_collection</code>	1
E5	In the Twitter collection, how did daily usage of the stemmed terms tax,inflat, and economi change between 2022-02-01 and 2022-02-14?	Single	Text	AI	<code>get_term_daily_counts</code>	1
E6	Compare the topic clusters found in the Bluesky and reddit collections during 2025-05-01 to 2025-05-07. Are the dominant topic areas similar or different across the two platforms?	Multi	Text	SY	<code>get_nlp_topics</code>	2
E7	For the Reddit collection, retrieve the NLP metadata for 2025-03-01 to 2025-03-03. What NLP model or topic modelling configuration was used, and what should a researcher know before interpreting the topic outputs?	Single	Text	AI	<code>get_nlp_metadata</code>	1

Note. Scope denotes whether a task requires evidence from a single or multiple collections. Demand: DR = Direct Retrieval; BR = Batch Retrieval; AI = Analytic Interpretation; SY = Synthesis. Ref. steps gives the number of calls in the manually constructed reference plan. The temporal requirement and complexity labels were retained in the machine-readable benchmark but are not reproduced here because the required period and operational burden are already apparent from the prompt, source scope, demand profile, and reference-step count.

A.1 Open-Ended Supplementary Tasks

TABLE A.2: Derivation of the open-ended tasks used in the supplementary exploratory case study.

Open-ended ID	Bounded benchmark task	Broadened supplementary task	What was broadened
E1O	E1. Over the first week in Jan 2025, in the Bluesky collection, identify the highest-frequency terms. Summarise the main themes that can be inferred from these returned terms.	Over the first week in Jan 2025, in the Bluesky collection, what are the most prominent terms people are using, and which ones seem especially characteristic of that period?	The task moves from reporting high-frequency terms and inferred themes to judging which terms are <i>characteristic</i> of the period. This leaves the evidential basis for salience less specified and can invite temporal, topical, or visual corroboration.
D3O	D3. Which collection had the highest number of posts mentioning bushfire in 2024? What insights do you have about this result?	Investigate discussion of “bushfire” across the available collections in 2024. Which collections show the strongest activity, when does activity appear concentrated, and what additional patterns are worth noting for a researcher interpreting this result?	The task expands a single highest-volume comparison into an investigation of relative activity across collections, temporal concentration, and contextual patterns relevant to research interpretation.
D7O	D7. Compare Reddit activity between January and September 2024. How much did total post volume change, and did weekday/weekend posting patterns remain similar across the two months?	Explore how Reddit posting activity changed between January and September 2024. Compare overall volume, daily variation, and weekday/weekend patterns, and identify any additional features that would matter when interpreting the difference between the two months.	The task retains the volume and weekday/weekend comparison, while explicitly inviting daily-pattern analysis and further contextual qualifications that may affect interpretation of the month-to-month difference.

Note. The bounded tasks were part of the 35-task main benchmark and were assessed using task-specific reference plans and expected outcomes. The broadened prompts were used only in the supplementary qualitative case study; they were designed to examine how the agent scoped and extended researcher-oriented analytical requests.

Appendix B

Prompt Templates

This appendix reproduces the prompt templates used by the experimental agent and evaluator. Runtime-specific inputs—including the task prompt, tool definitions, ground-truth answer, and agent answer—were inserted programmatically at execution time and are described in Chapter 4.

B.1 Agent Prompt Configuration

B.1.1 Base system prompt

Prompt identifier: MAIN_SYSTEM_PROMPT.

You are an agent interacting with external tools. Your job is to solve the user's task
↪ as accurately as possible using the available tools when needed.

STRICT OPERATING RULES:

1. TOOL USE: Use tools for all facts, counts, or charts unless the required information
↪ is explicitly provided in the task. Do not guess. If a tool call fails, attempt a
↪ valid alternative or report the failure.
2. DATA INTEGRITY: Preserve numerical precision from tool outputs. Do not infer or
↪ fabricate missing data points, buckets, or labels. If a requested data is only
↪ partially covered by tool outputs, explicitly report the unavailable parts rather
↪ than silently omitting them.
3. DECOMPOSITION: If a task exceeds tool constraints, attempt to decompose it into
↪ smaller steps or consider valid alternative.
4. SOURCE GROUNDING: Base the final answer ONLY on tool results or provided task info.

STRICT OUTPUT RULES:

1. FINAL OUTPUT: Your final response must be EXACTLY one valid JSON object. The JSON
↪ must follow the response schema provided in the instructions.
2. NO MARKDOWN: Do not use ```json or ``` blocks. Do not include any text before or
↪ after the JSON.
3. GROUNDING: Every number and fact must originate from a tool output. Do not include
↪ unsupported claims.

4. TYPES: Numeric values must be JSON numbers (10), not strings ("10").

STATUS DEFINITIONS:

- "success": Task fully completed with tool-backed data.
- "partial": Task partially completed; some data missing or tool limits reached.
- "unsupported": Tool capabilities do not match the user request.
- "failed": Tool error or unexpected output prevented a result.

B.1.2 Scalar-response schema prompt

Prompt identifier: SCALAR_PROMPT.

Return the final answer as exactly one JSON object using this schema:

```
{
  "status": "success | partial | unsupported | failed",
  "answer": {
    "value": number | null,
    "unit": "string | null",
    "comment": "string | null"
  }
}
```

Requirements:

- Use "success" only if you produced a concrete scalar result.
- Put the main numeric answer in "value".
- Percentages use unit="percent" and value in 0-100 scale.
- Use "unit" for units such as "posts", "percent", or null if no unit applies.
- Use "comment" for a short description of what the scalar represents
- If no valid scalar result can be obtained, set "value" to null and explain via status.
- Keep all calculations grounded in tool outputs or task-provided information.
- Do not include extra fields.

B.1.3 Time-series-response schema prompt

Prompt identifier: TIMESERIES_PROMPT.

Return the final answer as exactly one JSON object using this schema:

```
{
  "status": "success | partial | unsupported | failed",
  "answer": {
    "x_axis_type": "time",
    "granularity": "string | null",
    "value_unit": "string | null",
    "series": [
      {
        "label": "string | null",
        "points": [
          {
            "x": "string",
            "value": number
          }
        ]
      }
    ]
  },
  "comment": "string | null"
}
```

```
}
}
```

Requirements:

- Use "success" only if you obtained a valid ordered time series.
- "x_axis_type" must be exactly "time".
- "granularity" should describe the time unit if known, such as "hour", "day", "month".
- "series" may contain one or more series.
- Each item in "series" should use "label" to describe what that series represents.
- Within each series, "points" must be ordered from earliest to latest.
- Unless explicitly specified otherwise, a week refers to a consecutive 7-day bucket
 - ↪ anchored at the task or tool start date. For weekly series, use YYYY-MM-DD as the x
 - ↪ label, where the label is the bucket start date.
- Use canonical time labels based on granularity:
 - month -> YYYY-MM
 - day -> YYYY-MM-DD
 - hour -> YYYY-MM-DDTHH:00:00Z
 - week -> YYYY-MM-DD, where the label is the bucket start date
- Use one consistent time-label format across all series.
- If no valid series can be produced, return an empty "series" array and use the
 - ↪ appropriate non-success status.
- Use "comment" for a short summary of the result.
- Do not include extra fields.

B.1.4 Text-response schema prompt

Prompt identifier: TEXT_PROMPT.

Return the final answer as exactly one JSON object using this schema:

```
{
  "status": "success | partial | unsupported | failed",
  "answer": {
    "summary": string,
    "key_points": [string]
  }
}
```

Requirements:

- Use "success" only if you obtained enough evidence to provide a grounded textual
 - ↪ answer.
- "summary" should be a concise answer to the user's task.
- "key_points" is a list of discrete factual findings. Each point MUST include specific
 - ↪ facts from the tool output.
- Keep all claims grounded in tool outputs or task-provided information.
- Do not include extra fields.
- If the result is unsupported or failed, "summary" should briefly explain why.

B.1.5 Chart-response schema prompt

Prompt identifier: CHART_PROMPT.

Return the final answer as exactly one JSON object using this schema:

```
{
  "status": "success | partial | unsupported | failed",
```

```

"answer": {
  "chart_type": "string | null",
  "title": "string | null",
  "labels": ["string"],
  "datasets": [
    {
      "label": "string | null",
      "data": [number]
    }
  ],
  "summary": "string",
  "render_url": "string | null"
}
}

```

Requirements:

- Use "success" only if a complete and valid chart specification grounded in tool outputs was produced.
- "chart_type" should be a short type such as "bar", "line", or "pie". If no chart type is explicitly requested, choose a simple suitable type.
- If the task explicitly requests a chart type, use that chart type when supported.
- Represent the plotted data using "labels" and "datasets".
- For each dataset, the length of "data" must exactly match the length of "labels".
- Keep the order of "labels" consistent with the plotted x-axis.
- For temporal charts, use canonical time labels where possible:
 - month -> YYYY-MM
 - day -> YYYY-MM-DD
 - hour -> YYYY-MM-DDTHH:00:00Z
 - week -> YYYY-MM-DD, where the label is the bucket start date
- Unless explicitly specified otherwise, a week refers to a consecutive 7-day bucket anchored at the task or tool start date.
- If chart generation is unsupported or fails, return:
 - "chart_type": null
 - "title": null
 - "labels": []
 - "datasets": []
 - "summary": "a brief explanation"
 - "render_url": null

B.2 Judge Prompt Configuration

B.2.1 Semantic correctness evaluator prompt

Prompt identifier: Evaluator_PROMPT.

You are evaluating whether an agent's final answer is factually consistent with the
 ↪ ground truth answer for a benchmark task.

You will be given:

1. The original task prompt
2. The ground truth answer
3. The agent's final answer

Judge semantic correctness, not writing style.

Rules:

- Do not require exact wording, phrasing, ordering, or formatting.
- Additional details are allowed if they are consistent with the ground truth.
- More specific numerical details are allowed unless they contradict the ground truth.
- The answer must preserve the main conclusion, comparisons, rankings, and limitations.
- Missing data, 404 errors, unavailable data, or incomplete comparisons are important
 ↪ limitations.
- Do not convert missing/unavailable data into zero unless the ground truth explicitly
 ↪ says zero.
- If the same main conclusion is preserved and there are no substantive contradictions
 ↪ or missing limitations, score 2.
- If the main conclusion is correct but an important limitation or supporting fact is
 ↪ missing or misstated, score 1.
- If the main conclusion is wrong, reversed, or absent, score 0.
- Use -1 only when the answer is empty, irrelevant, non-interpretable, or refuses to
 ↪ answer.

Scoring:

2 = Correct
 1 = Partially correct
 0 = Incorrect
 -1 = Invalid

Return exactly one JSON object:

```
{
  "score": -1 | 0 | 1 | 2,
  "main_conclusion_match": true | false,
  "has_substantive_contradiction": true | false,
  "missing_or_changed_limitations": true | false,
  "reason": "brief explanation"
}
```

Appendix C

Illustrative Evaluation Record

This appendix presents one abbreviated record from the evaluation pipeline. It illustrates a *partially correct* outcome: the agent reached the correct overall comparison, but misstated a supporting ranking. To preserve readability, tool outputs are summarised rather than reproduced in full; the agent final response and the judge decision are reproduced as structured JSON.

C.1 Task and audited reference answer

Task ID: C3

Answer type: Text

Model configuration: MoonshotAI Kimi K2.6

Recorded outcome: Partially correct (judge score: 1)

Benchmark prompt

For the Reddit dataset, was posting activity generally higher on weekends or weekdays during 2023?

Audited reference answer

Posting activity on Reddit was generally higher on weekdays than on weekends during 2023. The weekday total was 1,199,919 posts across Monday to

Friday, compared with 460,064 posts across Saturday and Sunday. Average posting volume was approximately 239,984 posts per weekday and 230,032 posts per weekend day. Thursday had the highest daily volume (243,426 posts), followed by Wednesday (242,988 posts), while Saturday had the lowest volume (226,939 posts).

C.2 Abbreviated agent execution trace

The agent completed three tool calls. Parameter-schema validity and constraint compliance were both recorded as 1.00 for the trial.

1. **Helper call:** `get_collections`.

The agent inspected the available collections and identified the Reddit dataset.

2. **Helper call:** `get_collection_summary`.

The agent retrieved contextual metadata for the Reddit collection.

3. **Core call:** `aggregate_seasonality`.

The tool returned weekday and weekend posting volumes for 2023. The relevant evidence showed higher average activity on weekdays, with Thursday (243,426 posts) above Wednesday (242,988 posts), and Saturday the lowest day (226,939 posts).

C.3 Agent final response

```
{
  "status": "success",
  "answer": {
    "summary": "During 2023, posting activity in the Reddit dataset was generally
↪ higher on weekdays than on weekends.",
    "key_points": [
      "The average weekday post count was approximately 239,984, while the average
↪ weekend day post count was approximately 230,032.",
      "Wednesday had the highest single-day count with 242,988 posts, followed closely
↪ by Thursday with 243,426 posts.",
      "Saturday had the lowest count among all days with 226,939 posts.",
```

```
    "Total weekday posts (Monday-Friday) summed to 1,199,919, compared to 460,064  
    ↪ total weekend posts (Saturday-Sunday)."  
  ]  
}  
}
```

C.4 Judge decision

The judge assigned a partial-correctness score after parsing the returned JSON. The displayed decision is formatted without the code fence present in the raw model output.

```
{  
  "score": 1,  
  "main_conclusion_match": true,  
  "has_substantive_contradiction": true,  
  "missing_or_changed_limitations": false,  
  "reason": "The main conclusion is correct, and all numerical data matches the ground  
  ↪ truth. However, the agent's answer incorrectly states that Wednesday had the  
  ↪ highest post count, when the ground truth (and the agent's own provided numbers)  
  ↪ shows that Thursday had the highest count. This is a misstatement of a  
  ↪ supporting fact."  
}
```

Interpretation. The trial demonstrates the distinction between preserving a task’s main conclusion and accurately representing every supporting claim. Although the agent correctly identified weekday activity as higher overall, the reversed ordering of the two highest-volume days warranted a partial-correctness score rather than a fully correct score.

Appendix D

Ground-Truth Audit Protocol for Live-System Evaluation

Before each experimental run, the stored benchmark reference state was audited against the current outputs of the live AIO MCP service. The audit was designed to determine whether the stored normalised ground truth remained a valid answer to the original task prompt at the time of execution. It therefore assessed the validity of the expected answer, rather than performing a superficial comparison of old and newly returned raw payloads.

D.1 Purpose and Scope

The protocol was applied only to the benchmark reference state. It was not part of the evaluated agents' decision-making process, and it did not permit alternative retrieval pathways to repair or replace a reference plan. For each task, the stored `ref_tool_call` sequence constituted the fixed reference plan. The plan was re-executed in its original order and with its original tool names and arguments. Where the current output no longer supported the stored ground truth, the task was flagged for review before the associated experimental run.

D.2 Repository Dependencies and Implementation Context

The audit relied on a small set of project-repository components. These details are included to document reproducibility without treating the audit procedure as a coding-agent workflow.

Component	Role in the audit
Benchmark task records	Provided each task identifier, original prompt, answer type, stored raw reference output, fixed <code>ref_tool_call</code> sequence, and stored normalised ground truth.
MCP execution wrapper	Sent each explicit tool name and argument object in the stored reference plan to the AIO MCP service and returned a normalised payload for audit. The wrapper collected outputs only; it did not generate alternative plans or interpret task answers.
AIO MCP service and environment configuration	Supplied the live tool endpoint, required credentials, and run-time configuration needed to execute the stored reference calls.
Answer-type schemas	Supported consistent interpretation of scalar, time-series, chart, and text ground-truth representations during audit and later evaluation.
Audit log	Recorded the audit timestamp, re-executed outputs, status decision, supporting notes, any proposed reference update, and whether human review was required.

TABLE D.1: Repository components required for the ground-truth audit.

The complete implementation, including the execution wrapper and machine-readable audit records, is provided with the project materials described in the Data and Code Availability statement. The thesis reports the procedure and dependencies needed to understand and reproduce the audit logic; implementation-specific command-line instructions and coding-agent compatibility notes are retained in the repository documentation.

D.3 Audit Workflow

1. **Validate the task record.** The audit first checked that the task identifier, prompt, answer type, stored raw output, fixed reference plan, and stored ground truth were available. Records with missing fields were marked for manual review rather than completed through inference.

2. **Re-execute the fixed reference plan.** Each stored reference call was executed in the original order, preserving its tool name and arguments. The audit did not modify `ref_tool_call`, substitute a different tool, or invoke an alternative retrieval route. A failed or incomplete execution was retained as evidence of an infrastructure or service issue.
3. **Assess ground-truth validity.** The current MCP outputs were evaluated against the original task prompt and the stored normalised ground truth. The assessment considered whether the expected answer remained supported, and whether changed values, rankings, dates, labels, trends, coverage conditions, or limitations would materially change that answer. Harmless wording and formatting differences were ignored.
4. **Classify the audit result.** Each task was assigned one of the statuses defined in Table D.2.
5. **Resolve flagged cases.** Valid references were retained. Stale or uncertain cases were reviewed before use in an experimental run. Tool or API failures were treated as environment-level issues rather than evidence of agent failure; affected trials were excluded or re-run according to the experimental procedure described in the methodology chapter.

Audit status	Operational definition
<code>valid</code>	The stored ground truth remained supported by the current MCP output and required no update.
<code>stale_ground_truth</code>	The current MCP output supported a materially different task answer, so the stored ground truth required an auditable refresh before evaluation.
<code>tool_or_api_error</code>	The current output could not be trusted because execution failed, returned an error or incomplete response, or produced an obviously invalid payload.
<code>uncertain</code>	A task-relevant discrepancy was observed, but the cause or correct reference update could not be established from the current execution alone and required human review.

TABLE D.2: Ground-truth audit status categories.

D.4 Audit Record and Reference-State Control

For each audited task, the audit log retained the task identifier, timestamp, current MCP output, stored ground truth, status decision, reasoning notes, proposed update where relevant, and a human-review flag. The stored benchmark task file was preserved as the original record. Any approved reference refresh was documented in the audit materials rather than silently replacing the prior reference state. This preserves traceability between the live-system condition observed before a run and the reference state used to evaluate the resulting agent output.

Appendix E

Additional Results

E.1 Tool appropriateness by model and task demand profile, calculated over successful trials only.

TABLE E.1: Dashes indicate that no successful trials were available for that model-profile combination.

Model	A	BR	D	S
Claude Opus 4.6	0.674	0.986	0.960	0.782
Claude Sonnet 4.6	0.648	0.988	0.879	0.759
DeepSeek V4 Pro	0.533	0.893	0.767	0.699
GPT-5.5	0.895	0.988	0.842	0.928
Kimi K2.6	0.825	0.989	0.843	0.830
Llama 4	–	1.000	1.000	–
Qwen 3.6 Plus	0.855	0.976	0.763	0.771

E.2 Task-Level Completion Outcomes Across Repeated Trials

TABLE E.2: Task-level completion counts across three independent trials. Each cell reports the number of successful trials out of three. AI = Analytic Interpretation; BR = Batch Retrieval; DR = Direct Retrieval; SY = Synthesis.

Task	Intent	Profile	Opus	Sonnet	DeepSeek	GPT-5.5	Kimi	Qwen	Llama
A1	A	DR	3/3	3/3	3/3	3/3	3/3	3/3	3/3
A2	A	BR	3/3	3/3	3/3	3/3	3/3	3/3	0/3
A3	A	BR	3/3	3/3	2/3	3/3	3/3	3/3	0/3
A4	A	BR	3/3	3/3	3/3	3/3	3/3	3/3	0/3

TABLE E.2: Task-level completion counts across three independent trials (continued).

Task	Intent	Profile	Opus	Sonnet	DeepSeek	GPT-5.5	Kimi	Qwen	Llama
A5	A	BR	3/3	3/3	2/3	3/3	3/3	3/3	2/3
A6	A	BR	3/3	3/3	3/3	3/3	3/3	3/3	1/3
A7	A	BR	3/3	3/3	3/3	3/3	3/3	3/3	0/3
B1	B	DR	3/3	3/3	3/3	3/3	3/3	3/3	0/3
B2	B	DR	3/3	3/3	3/3	3/3	3/3	3/3	0/3
B3	B	DR	3/3	3/3	3/3	3/3	3/3	3/3	1/3
B4	B	DR	3/3	3/3	3/3	3/3	3/3	3/3	0/3
B5	B	DR	3/3	3/3	3/3	3/3	3/3	2/3	0/3
B6	B	DR	3/3	3/3	3/3	1/3	2/3	3/3	0/3
B7	B	SY	3/3	3/3	1/3	3/3	3/3	3/3	0/3
C1	C	SY	3/3	2/3	2/3	3/3	3/3	0/3	0/3
C2	C	BR	3/3	3/3	3/3	3/3	3/3	3/3	0/3
C3	C	AI	3/3	3/3	3/3	3/3	2/3	3/3	0/3
C4	C	AI	3/3	3/3	2/3	3/3	3/3	3/3	0/3
C5	C	SY	3/3	2/3	3/3	3/3	3/3	3/3	0/3
C6	C	SY	3/3	3/3	3/3	3/3	3/3	3/3	0/3
C7	C	SY	3/3	3/3	3/3	3/3	3/3	3/3	0/3
D1	D	SY	3/3	3/3	3/3	3/3	3/3	3/3	0/3
D2	D	SY	3/3	1/3	3/3	3/3	2/3	3/3	0/3
D3	D	SY	3/3	3/3	2/3	1/3	2/3	3/3	0/3
D4	D	SY	3/3	1/3	3/3	2/3	0/3	2/3	0/3
D5	D	SY	2/3	3/3	3/3	3/3	3/3	3/3	0/3
D6	D	SY	3/3	3/3	3/3	2/3	3/3	2/3	0/3
D7	D	SY	3/3	2/3	2/3	1/3	3/3	3/3	0/3
E1	E	AI	3/3	3/3	3/3	1/3	3/3	1/3	0/3
E2	E	AI	2/3	3/3	2/3	3/3	3/3	3/3	0/3
E3	E	AI	3/3	3/3	3/3	1/3	3/3	3/3	0/3
E4	E	AI	3/3	3/3	3/3	3/3	3/3	2/3	0/3
E5	E	AI	3/3	2/3	3/3	3/3	2/3	3/3	0/3
E6	E	SY	2/3	3/3	3/3	3/3	1/3	2/3	0/3
E7	E	AI	3/3	2/3	3/3	2/3	1/3	1/3	0/3

Note. 3/3 represent the task succeed in 3 of 3 trials

Bibliography

- [1] Anthropic. Model context protocol, 2025. URL <https://modelcontextprotocol.io/docs/getting-started/intro>.
- [2] Adriana Alvarado Garcia, Tianling Yang, and Milagros Miceli. What knowledge do we produce from social media data and how? *Proc. ACM Hum.-Comput. Interact.*, 9(1), January 2025. doi: 10.1145/3701216. URL <https://doi.org/10.1145/3701216>.
- [3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017.
- [4] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wensen Cheng, Qi Zhang, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, and Tao Gui. The rise and potential of large language model based agents: A survey, 2023. URL <https://arxiv.org/abs/2309.07864>.
- [5] Zhiruo Wang, Zhoujun Cheng, Hao Zhu, Daniel Fried, and Graham Neubig. What are tools anyway? a survey from the language model perspective, 2024. URL <https://arxiv.org/abs/2403.15452>.
- [6] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL <https://arxiv.org/abs/2210.03629>.
- [7] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023. URL <https://arxiv.org/abs/2302.04761>.
- [8] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023. URL <https://arxiv.org/abs/2307.16789>.
- [9] OpenAI. Gpt-5.5 system card, 2026. URL <https://openai.com/index/gpt-5-5-system-card/>.
- [10] Meta AI. The llama 4 herd: The beginning of a new era of natively multimodal ai innovation, 2025. URL <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>.
- [11] Harrison Chase. Langchain. <https://github.com/langchain-ai/langchain>, 2022.
- [12] Jerry Liu and LlamaIndex Team. Llamaindex: A data framework for llm applications, 2024.

- [13] Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Jirong Wen. Tool learning with large language models: a survey. *Frontiers of Computer Science*, 19(8), January 2025. ISSN 2095-2236. doi: 10.1007/s11704-024-40678-2. URL <http://dx.doi.org/10.1007/s11704-024-40678-2>.
- [14] Aditi Singh, Abul Ehtesham, Saket Kumar, and Tala Talaei Khoei. A survey of the model context protocol (mcp): Standardizing context to enhance large language models (llms). *Preprints*, April 2025. doi: 10.20944/preprints202504.0245.v1. URL <https://doi.org/10.20944/preprints202504.0245.v1>.
- [15] Martin Treiber. Model context protocol (mcp): A new standard for tool and data integration, Apr 2025. URL <https://www.ikangai.com/model-context-protocol-mcp-a-new-standard-for-tool-and-data-integration/>.
- [16] Anthropic. Architecture overview. URL <https://modelcontextprotocol.io/docs/learn/architecture>.
- [17] Australian Internet Observatory — internetobservatory.org.au. <https://internetobservatory.org.au/>. [Accessed 21-09-2025].
- [18] AIReD. Aired website. <https://www.aio.eresearch.unimelb.edu.au/>, 2025. URL <https://www.aio.eresearch.unimelb.edu.au/>.
- [19] Weixu Zhang, Yifei Wang, Yuanfeng Song, Victor Junqiu Wei, Yuxing Tian, Yiyan Qi, Jonathan H. Chan, Raymond Chi-Wing Wong, and Haiqin Yang. Natural language interfaces for tabular data querying and visualization: A survey, 2024. URL <https://arxiv.org/abs/2310.17894>.
- [20] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Wang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain text-to-SQL. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3911–3921, Brussels, Belgium, oct-nov 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1425. URL <https://aclanthology.org/D18-1425>.
- [21] Tao Yu, Rui Zhang, Heyang Er, Suyi Li, Eric Xue, Bo Pang, Xi Victoria Lin, Yi Tan, Tian-Jun Shi, Zihan Li, Youxuan Jiang, Michihiro Yasunaga, Sung-Yun Shim, Tao Chen, Alexander Fabbri, Zifan Li, Lu Chen, Yu Zhang, Shreya Dixit, Vincent Zhang, Caiming Xiong, Dragomir Radev, Richard Socher, and Wen-tau Xiong. CoSQL: A conversational text-to-SQL challenge towards cross-domain natural language interfaces to databases. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3013–3024, Hong Kong, China, nov 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1293. URL <https://aclanthology.org/D19-1293>.
- [22] Jinyang Li, Binyuan Hui, Ge Qu, Binhua Li, Jiayi Yang, Bowen Li, Le Zong, Hongyu Kang, Yuxiang Lu, Nan Huo, Yusu Piao, Yaguan Qian, Yisfan He, Wenjie Zhang, Jian-Guang Lou, Yong-Liang Yang, and Fei-Yue Wang. BIRD: Can large language models truly understand user intentions? a case study on text-to-SQL, 2023.
- [23] Mengyi Liu and Jianqiu Xu. Nli4db: A systematic review of natural language interfaces for databases, 2025. URL <https://arxiv.org/abs/2503.02435>.
- [24] Jinwei Lu, Yuanfeng Song, Zhiqian Qin, Haodi Zhang, Chen Zhang, and Raymond Chi-Wing Wong. Bridging the gap: Enabling natural language queries for nosql databases through text-to-nosql translation, 2025. URL <https://arxiv.org/abs/2502.11201>.

- [25] Makbule Gulcin Ozsoy, Leila Messallem, Jon Besga, and Gianandrea Minneci. Text2cypher: Bridging natural language and graph databases, 2024. URL <https://arxiv.org/abs/2412.10064>.
- [26] Yuanyuan Liang, Tingyu Xie, Gan Peng, Zihao Huang, Yunshi Lan, and Weining Qian. Nat-nl2gql: A novel multi-agent framework for translating natural language to graph query language, 2024. URL <https://arxiv.org/abs/2412.10434>.
- [27] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. Vega-lite: A grammar of interactive graphics. *IEEE Transactions on Visualization & Computer Graphics (Proc. InfoVis)*, 2017. doi: 10.1109/tvcg.2016.2599030. URL <http://idl.cs.washington.edu/papers/vega-lite>.
- [28] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi: 10.1109/MCSE.2007.55.
- [29] Arpit Narechania, Arjun Srinivasan, and John Stasko. Nl4dv: A toolkit for generating analytic specifications for data visualization from natural language queries. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):369–379, February 2021. ISSN 2160-9306. doi: 10.1109/tvcg.2020.3030378. URL <http://dx.doi.org/10.1109/TVCG.2020.3030378>.
- [30] Victor Dibia. Lida: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models, 2023. URL <https://arxiv.org/abs/2303.02927>.
- [31] Rafi Al Attrach, Pedro Moreira, Rajna Fani, Renato Umeton, and Leo Anthony Celi. Conversational llms simplify secure clinical data access, understanding, and analysis, 2025. URL <https://arxiv.org/abs/2507.01053>.
- [32] Peer Trilcke, Ingo Börner, Henny Sluyter-Gäthje, Daniil Skorinkin, Frank Fischer, and Carsten Milling. Agentic dracor and the art of docstring engineering: Evaluating mcp-empowered llm usage of the dracor api, 2025. URL <https://arxiv.org/abs/2508.13774>.
- [33] Frank Fischer, Ingo Börner, Mathias Göbel, Angelika Hechtel, Christopher Kittel, Carsten Milling, and Peer Trilcke. Programmable Corpora: Introducing DraCor, an Infrastructure for the Research on European Drama. In *Proceedings of DH2019: "Complexities", Utrecht, July 9–12, 2019*. Utrecht University, 2019. doi: 10.5281/zenodo.4284002.
- [34] Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. Survey on evaluation of llm-based agents, 2025. URL <https://arxiv.org/abs/2503.16416>.
- [35] Mahmoud Mohammadi, Yipeng Li, Jane Lo, and Wendy Yip. Evaluation and benchmarking of llm agents: A survey. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, KDD '25, page 6129–6139. ACM, August 2025. doi: 10.1145/3711896.3736570. URL <http://dx.doi.org/10.1145/3711896.3736570>.
- [36] Qiantong Xu, Fenglu Hong, Bo Li, Changran Hu, Zhengyu Chen, and Jian Zhang. On the tool manipulation capability of open-source large language models, 2023.
- [37] Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025.
- [38] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains, 2024. URL <https://arxiv.org/abs/2406.12045>.

- [39] Yixing Jiang, Kameron C. Black, Gloria Geng, Danny Park, James Zou, Andrew Y. Ng, and Jonathan H. Chen. Medagentbench: A realistic virtual ehr environment to benchmark medical llm agents, 2025. URL <https://arxiv.org/abs/2501.14654>.
- [40] Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Rahul Gupta, Jwala Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Jasjeet Sekhon, Jacob Steinhardt, Antony Kellermann, Sarah Schwettmann, Matei Zaharia, Ion Stoica, Percy Liang, and Daniel Kang. Establishing best practices for building rigorous agentic benchmarks, 2025. URL <https://arxiv.org/abs/2507.02825>.
- [41] Zhenting Wang, Qi Chang, Hemani Patel, Shashank Biju, Cheng-En Wu, Quan Liu, Aolin Ding, Alireza Rezazadeh, Ankit Shah, Yujia Bao, and Eugene Siow. Mcp-bench: Benchmarking tool-using llm agents with complex real-world tasks via mcp servers, 2025. URL <https://arxiv.org/abs/2508.20453>.
- [42] Guozhao Mo, Wenliang Zhong, Jiawei Chen, Xuanang Chen, Yaojie Lu, Hongyu Lin, Ben He, Xianpei Han, and Le Sun. Livemcpbench: Can agents navigate an ocean of mcp tools?, 2025. URL <https://arxiv.org/abs/2508.01780>.
- [43] Shiqing Fan, Xichen Ding, Liang Zhang, and Linjian Mo. Mcptoolbench++: A large scale ai agent model context protocol mcp tool use benchmark, 2025. URL <https://arxiv.org/abs/2508.07575>.
- [44] Wei Song, Haonan Zhong, Ziqi Ding, Jingling Xue, and Yuekang Li. Help or hurdle? rethinking model context protocol-augmented large language models, 2025. URL <https://arxiv.org/abs/2508.12566>.
- [45] Xuanqi Gao, Siyi Xie, Juan Zhai, Shqing Ma, and Chao Shen. Mcp-radar: A multi-dimensional benchmark for evaluating tool use capabilities in large language models, 2025. URL <https://arxiv.org/abs/2505.16700>.
- [46] Zhiwei Liu, Jielin Qiu, Shiyu Wang, Jianguo Zhang, Zuxin Liu, Roshan Ram, Haolin Chen, Weiran Yao, Shelby Heinecke, Silvio Savarese, Huan Wang, and Caiming Xiong. Mcpeval: Automatic mcp-based deep evaluation for ai agent models, 2025. URL <https://arxiv.org/abs/2507.12806>.
- [47] Zijian Wu, Xiangyan Liu, xinyuan zhang, Lingjun Chen, Fanqing Meng, Lingxiao Du, Yiran Zhao, Fanshi Zhang, Yaoqi Ye, Jiawei Wang, Zirui Wang, Jinjie Ni, Yufan Yang, Arvin Xu, and Michael Qizhe Shieh. MCP-Mark: A benchmark for stress-testing realistic and comprehensive MCP use. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=uobR0wBsJm>.
- [48] Zikang Guo, Benfeng Xu, Chiwei Zhu, Wentao Hong, Xiaorui Wang, and Zhendong Mao. Mcp-agentbench: Evaluating real-world language agent performance with mcp-mediated tools, 2025. URL <https://arxiv.org/abs/2509.09734>.
- [49] Ming Yin, Dinghan Shen, Silei Xu, Jianbing Han, Sixun Dong, Mian Zhang, Yebowen Hu, Shujian Liu, Simin Ma, Song Wang, Sathish Reddy Indurthi, Xun Wang, Yiran Chen, and Kaiqiang Song. Livemcp-101: Stress testing and diagnosing mcp-enabled agents on challenging queries, 2025. URL <https://arxiv.org/abs/2508.15760>.
- [50] Ziyang Luo, Zhiqi Shen, Wenzhuo Yang, Zirui Zhao, Prathyusha Jwalapuram, Amrita Saha, Doyen Sahoo, Silvio Savarese, Caiming Xiong, and Junnan Li. Mcp-universe: Benchmarking large language models with real-world model context protocol servers, 2025. URL <https://arxiv.org/abs/2508.14704>.
- [51] OpenRouter. Using mcp servers with openrouter — openrouter — documentation. URL <https://openrouter.ai/docs/cookbook/coding-agents/mcp-servers>.
- [52] Anthropic. Introducing claude 4, 2026. URL <https://www.anthropic.com/news/claude-4>.

- [53] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- [54] Kimi Team, Yifan Bai, Yiping Bao, Y. Charles, Cheng Chen, Guanduo Chen, Haiting Chen, Huarong Chen, Jiahao Chen, Ningxin Chen, Ruijie Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jialei Cui, Hao Ding, Mengnan Dong, Angang Du, Chenzhuang Du, Dikang Du, Yulun Du, Yu Fan, Yichen Feng, Kelin Fu, Bofei Gao, Chenxiao Gao, Hongcheng Gao, Peizhong Gao, Tong Gao, Yuyao Ge, Shangyi Geng, Qizheng Gu, Xinran Gu, Longyu Guan, Haiqing Guo, Jianhang Guo, Xiaoru Hao, Tianhong He, Weiran He, Wenyang He, Yunjia He, Chao Hong, Hao Hu, Yangyang Hu, Zhenxing Hu, Weixiao Huang, Zhiqi Huang, Zihao Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yongsheng Kang, Guokun Lai, Cheng Li, Fang Li, Haoyang Li, Ming Li, Wentao Li, Yang Li, Yanhao Li, Yiwei Li, Zhaowei Li, Zheming Li, Hongzhan Lin, Xiaohan Lin, Zongyu Lin, Chengyin Liu, Chenyu Liu, Hongzhang Liu, Jingyuan Liu, Junqi Liu, Liang Liu, Shaowei Liu, T. Y. Liu, Tianwei Liu, Weizhou Liu, Yangyang Liu, Yibo Liu, Yiping Liu, Yue Liu, Zhengying Liu, Enzhe Lu, Haoyu Lu, Lijun Lu, Yashuo Luo, Shengling Ma, Xinyu Ma, Yingwei Ma, Shaoguang Mao, Jie Mei, Xin Men, Yibo Miao, Siyuan Pan, Yebo Peng, Ruoyu Qin, Zeyu Qin, Bowen Qu, Zeyu Shang, Lidong Shi, Shengyuan Shi, Feifan Song, Jianlin Su, Zhengyuan Su, Lin Sui, Xinjie Sun, Flood Sung, Yunpeng Tai, Heyi Tang, Jiawen Tao, Qifeng Teng, Chaoran Tian, Chensi Wang, Dinglu Wang, Feng Wang, Hailong Wang, Haiming Wang, Jianzhou Wang, Jiaxing Wang, Jinhong Wang, Shengjie Wang, Shuyi Wang, Si Wang, Xinyuan Wang, Yao Wang, Yejie Wang, Yiqin Wang, Yuxin Wang, Yuzhi Wang, Zhaoji Wang, Zhengtao Wang, Zhengtao Wang, Zhexu Wang, Chu Wei, Qianqian Wei, Haoning Wu, Wenhao Wu, Xingzhe Wu, Yuxin Wu, Chenjun Xiao, Jin Xie, Xiaotong Xie, Weimin Xiong, Boyu Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinran Xu, Yangchuan Xu, Ziyao Xu, Jing Xu, Jing Xu, Junjie Yan, Yuzi Yan, Hao Yang, Xiaofei Yang, Yi Yang, Ying Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Xingcheng Yao, Wenjie Ye, Zhuorui Ye, Bohong Yin, Longhui Yu, Enming Yuan, Hongbang Yuan, Mengjie Yuan, Siyu Yuan, Haobing Zhan, Dehao Zhang, Hao Zhang, Wanlu Zhang, Xiaobin Zhang, Yadong Zhang, Yangkun Zhang, Yichi Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Haotian Zhao, Yikai Zhao, Zijia Zhao, Huabin Zheng, Shaojie Zheng, Longguang Zhong, Jianren Zhou, Xinyu Zhou, Zaida Zhou, Jinguo Zhu, Zhen Zhu, Weiyu Zhuang, and Xinxing Zu. Kimi k2: Open agentic intelligence, 2026. URL <https://arxiv.org/abs/2507.20534>.
- [55] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026. URL <https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro>.
- [56] Gemini Team Google. Gemini 2.5: Our most intelligent ai model, 2025. URL <https://blog.google/innovation-and-ai/models-and-research/google-deepmind/gemini-model-thinking-updates-march-2025/>.
- [57] Kiran Kate, Yara Rizk, Poulami Ghosh, Ashu Gulati, Tathagata Chakraborti, Zidane Wright, and Mayank Agarwal. How good are LLMs at processing tool outputs? In Vera Demberg, Kentaro Inui, and Lluís Marquez, editors, *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2918–2941, Rabat, Morocco, March 2026. Association for Computational Linguistics. ISBN 979-8-89176-380-7. doi: 10.18653/v1/2026.eacl-long.134. URL <https://aclanthology.org/2026.eacl-long.134/>.

-
- [58] Zhiyin Tan and Jennifer D’Souza. Diagnosing structural failures in llm-based evidence extraction for meta-analysis, 2026. URL <https://arxiv.org/abs/2602.10881>.
- [59] Yufeng Du, Minyang Tian, Srikanth Ronanki, Subendhu Rongali, Sravan Bodapati, Aram Galstyan, Azton Wells, Roy Schwartz, Eliu A Huerta, and Hao Peng. Context length alone hurts llm performance despite perfect retrieval, 2025. URL <https://arxiv.org/abs/2510.05381>.
- [60] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 02 2024. ISSN 2307-387X. doi: 10.1162/tacl_a_00638. URL https://doi.org/10.1162/tacl_a_00638.
- [61] Qiaohong Zhang, Weihao Ye, Jialong Chen, Yi Luo, BoYuan Li, Bowen Deng, Zibin Zheng, Jianhao Lin, Wei-Shi Zheng, and Chuan Chen. Dataclawbench: An agent benchmark for exploratory real-world financial data analysis, 2026. URL <https://arxiv.org/abs/2605.02503>.
- [62] Boyan Liu, Gongming Zhao, and Hongli Xu. Utility-guided agent orchestration for efficient llm tool use, 2026. URL <https://arxiv.org/abs/2603.19896>.